Agentic AI Framework for Context-Aware Semantic Understanding and Intelligent Assistance of Dynamic Web Forms

(Smart Form Auto-Filling System)

Submitted in partial fulfillment of the requirement for the award of
Degree of Bachelor of Technology in
Computer Science

Submitted To



SVKM's NMIMS,
Mukesh Patel School of Technology Management & Engineering,
Shirpur Campus (M.H.)

Submitted By:

Ashmit Patil (E246) Enrollment No – 70552300048
Sanket Mali (E265) Enrollment No – 70552300067
Pinak Salunke (E247) Enrollment No – 70552300049

Under The Supervision Of:

Dr. Nitin Choubey
(Professor, Department of Computer Science)

DEPARTMENT OF COMPUTER SCIENCE
Mukesh Patel School of Technology Management & Engineering
ACADEMIC SESSION: 2026-27

CERTIFICATE

This is to certify that the project entitled **"Agentic AI Framework for Context-Aware Semantic Understanding and Intelligent Assistance of Dynamic Web Forms"** has been done by

"Ashmit Patil" (Roll No.: 246; Enrollment No.: 70552300048)

"Sanket Mali" (Roll No.: 265; Enrollment No.: 70552300067)

"Pinak Salunke" (Roll No.: 247; Enrollment No.: 70552300049)

under my guidance and supervision and has been submitted in partial fulfillment of the degree of "Bachelor of Technology in Computer Science" of SVKM's NMIMS (Deemed-to-be University), MPSTME Shirpur Campus (M.H.), India.

Project Mentor	**Examiner**
(Dr. Nitin Choubey, Internal Guide)	

Date:

Place: Shirpur

H.O.D.
(Dr. Nitin S. Choubey)

DEPARTMENT OF COMPUTER SCIENCE

Mukesh Patel School of Technology Management & Engineering

CERTIFICATE OF APPROVAL

The project entitled "Agentic AI Framework for Context-Aware Semantic Understanding and Intelligent Assistance of Dynamic Web Forms" submitted by Ashmit Patil (70552300048), Sanket Mali (70552300067) and Pinak Salunke (70552300049) of B.Tech (Computer Science), Academic Session 2026-27, MPSTME Shirpur Campus, SVKM's NMIMS, has been examined and is approved for the award of the degree of Bachelor of Technology in Computer Science.

The examiners record their approval below after the project presentation and viva-voce examination held on _____.

_____	_____
Internal Examiner	**External Examiner**
Name: _____	Name: _____

Date: _____

Place: Shirpur

_____	_____
Project Guide	**Head of Department**
(Dr. Nitin Choubey)	(Dr. Nitin S. Choubey)

DECLARATION

We, the undersigned students of B.Tech (Computer Science), Academic Session 2026-27, MPSTME Shirpur Campus, SVKM's NMIMS, declare that the project report entitled "Agentic AI Framework for Context-Aware Semantic Understanding and Intelligent Assistance of Dynamic Web Forms" is a record of the work carried out by us under the guidance of Dr. Nitin Choubey. The software, diagrams, test results and screenshots in this report are produced from the implementation developed by the project team.

To the best of our knowledge, this report has not been submitted, in part or in full, to any other university or institution for the award of any degree or diploma. Information obtained from published or unpublished sources has been acknowledged in the text and listed in the References.

_____	_____
Ashmit Patil	**Sanket Mali**
Enrollment No. 70552300048	Enrollment No. 70552300067
_____	_____
Pinak Salunke	
Enrollment No. 70552300049	

Date: _____

Place: Shirpur

ACKNOWLEDGEMENT

After the completion of this capstone project work, words are not enough to express our feelings about all those who helped us to reach our goal; feeling above this is our indebtedness to The Almighty for providing us this moment in life.

It is a great pleasure and a moment of immense satisfaction for us to express our profound gratitude to Dr. Nitin Choubey, Professor, Computer Science Department, MPSTME, Shirpur, whose constant encouragement enabled us to work enthusiastically. Their perpetual motivation, patience and expertise in discussion during the progress of the project work have benefited us to an extent which is beyond expression. Their depth and breadth of knowledge of the Computer Science field made us realise that theoretical knowledge always helps to develop efficient operational software, which is a blend of all core subjects of the field. We are highly indebted to them for their invaluable guidance and ever-ready support in the successful completion of this project in time.

We express our sincere thanks and gratitude to Dr. Nitin S. Choubey, Head of Department, Computer Science, MPSTME, Shirpur, for providing the necessary infrastructure and help to complete the project work successfully.

We also extend our deepest gratitude to Dr. Venkatadri Marriboyina, Associate Dean, SVKM's NMIMS, Shirpur Campus, for providing all the necessary facilities and an encouraging environment to bring out the best of our endeavours.

We sincerely wish to express our grateful thanks to all members of the staff of the Computer Science department and to all those who have imparted technical knowledge of computer technology to us during the various stages of B.Tech. Computer Science. We also thank our friends, who contributed directly or indirectly to this capstone project work, and the open-source communities whose tools made it possible.

Ashmit Patil
Sanket Mali
Pinak Salunke

ABSTRACT

Filling web forms is a repetitive task in which people re-type the same identity, education, address and employment details on portals for admissions, banking, government services and recruitment. Native browser autofill only replays values typed earlier, cannot read a user's documents, records no evidence for a value, and does not warn when two documents disagree. This report presents the design, implementation and verification of the Smart Form Auto-Filling System, a privacy-first assistant that builds a profile from the user's own documents and uses it to recommend values for a web form, which the user reviews before anything is filled.

The system has three parts: a FastAPI backend with MongoDB and ChromaDB, a React dashboard, and a Chrome Manifest V3 extension. Uploaded PDF, DOCX, TXT and image files are parsed; facts are extracted by deterministic rules over a 91-concept YAML ontology and, optionally, by a large language model, then merged with provenance (origin, evidence text, page, confidence). Disagreements between documents are computed as conflicts for the user to resolve. A seven-node LangGraph pipeline classifies each form field, resolves a value from the profile or from retrieved document evidence, and applies confidence and sensitivity rules. Prompt-injection defences, tenant isolation by user identifier, token revocation, redacted logging and an on-demand extension permission model protect the user's data. The extension never reads field values, never fills password, payment or one-time-code fields and never submits a form.

The implementation is verified by 196 automated tests, all of which passed in the recorded run: 146 backend tests with a mock LLM, 3 integration tests against a real ChromaDB service, 22 dashboard tests, 24 extension tests and one Playwright browser test on an isolated Docker Compose stack. Real-provider LLM tests were not run. The main contribution is an integrated, evidence-preserving and human-in-the-loop pipeline from document to filled form that is deployable with a single Compose file.

Keywords: Web form autofill, document information extraction, retrieval-augmented generation, LangGraph, field ontology, prompt injection, Chrome Manifest V3, privacy.

TABLE OF CONTENTS

Chapter 1 Introduction **1**
1.1 BACKGROUND OF THE PROJECT TOPIC 1
1.2 MOTIVATION AND SCOPE OF THE PROJECT 2
1.3 PROBLEM STATEMENT 3
1.4 AIM AND OBJECTIVES 3
1.5 SALIENT CONTRIBUTIONS 4
1.6 ORGANISATION OF THE REPORT 5
CHAPTER SUMMARY 5
Chapter 2 Literature Survey **7**
2.1 INTRODUCTION TO THE OVERALL TOPIC 7
Background concepts used in this project 8
2.2 EXHAUSTIVE LITERATURE SURVEY 8
2.2.1 Web Agents and Benchmarks 8
2.2.2 LLM Agent Frameworks, Knowledge and Memory 9
2.2.3 Form Understanding, Autofill and Form Security 9
2.2.4 Supporting Techniques 10
2.3 COMPARATIVE ANALYSIS 12
2.4 RESEARCH AND DEVELOPMENT GAP 12
2.5 PROPOSED APPROACH 13
CHAPTER SUMMARY 13
Chapter 3 Problem Definition and Proposed Solution **14**
3.1 PROBLEM DEFINITION 14
3.2 CHALLENGES 15
3.3 EXISTING SYSTEM AND ITS LIMITATIONS 15
3.4 PROPOSED SOLUTION OVERVIEW 16
Workflow summary 17
Advantages of the proposed solution 17
3.5 REQUIREMENTS 18
Functional requirements 18
Non-functional requirements 19
3.6 TECHNOLOGY STACK AND JUSTIFICATION 19
3.7 FEASIBILITY ANALYSIS 20
3.8 IMPLEMENTATION STATUS AND LIMITATIONS 20
CHAPTER SUMMARY 21
Chapter 4 Design **22**

4.1 BLOCK DIAGRAM AND SYSTEM ARCHITECTURE 22
4.2 HARDWARE AND SOFTWARE DESCRIPTION 23
Deployment architecture 23
Hardware 24
Software 24
4.3 BACKEND MODULE DESIGN 25
Configuration 25
Document processing 26
Ontology 26
4.4 DATA FLOW AND CONTEXT 27
Context diagram (Level 0) 27
Data flow diagram (Level 1) 28
Data flow diagram (Level 2) 29
4.5 BEHAVIOURAL DESIGN 30
Use case diagram 30
Activity diagrams 31
Sequence diagrams 33
4.6 DATABASE DESIGN 39
MongoDB collections 39
ChromaDB index 41
Status vocabularies 41
4.7 STRUCTURAL DESIGN: CLASS DIAGRAMS 43
4.8 CORE ALGORITHMS 45
Fact extraction and merging 45
Form-fill pipeline 46
Deterministic field classification 48
Decision rules 49
Conflict resolution 51
Retrieval-augmented question answering 51
4.9 SECURITY DESIGN 52
4.10 API DESIGN 55
4.11 CLIENT DESIGN 57
Chrome extension 57
Dashboard 58
CHAPTER SUMMARY 59
Chapter 5 Result Analysis: Snapshots 60
5.1 REGISTRATION AND LOGIN 60
5.2 DASHBOARD 61

5.3 DOCUMENT UPLOAD AND PROCESSING 62
5.4 PROFILE WITH PROVENANCE 63
5.5 CONFLICT DETECTION AND RESOLUTION 63
5.6 EXTENSION: SCAN 64
5.7 EXTENSION: REVIEW BEFORE FILL 65
5.8 GROUNDED QUESTION ANSWERING 66
5.9 SETTINGS AND SERVICE HEALTH 67
5.10 OBSERVATIONS 68
CHAPTER SUMMARY 68
Chapter 6 Testing 70
6.1 TESTING STRATEGY 70
6.2 BACKEND TESTS 71
6.3 DASHBOARD AND EXTENSION TESTS 73
6.4 INTEGRATION AND BROWSER END-TO-END TESTS 73
6.5 TEST CASES 74
6.6 TEST RESULTS 75
6.7 SECURITY-RELATED VERIFICATION 76
6.8 DISCUSSION 76
CHAPTER SUMMARY 77
Chapter 7 Conclusion and Future Work 78
7.1 CONCLUSION 78
Achievement of objectives 78
Principal findings 79
7.2 LIMITATIONS 80
7.3 FUTURE WORK 80
CHAPTER SUMMARY 81
References 82
Appendix I: Source Code Excerpts 86
I.1 Prompt-injection helpers 86
I.2 LangGraph construction 87
I.3 Decision node 87
I.4 Concept view and conflict logic 88
I.5 Merging candidate facts 89
I.6 Field eligibility in the extension 89
I.7 Writing a value into a page 90
I.8 Settings validators 91
Appendix II: Ontology Excerpt 92

Appendix III: Configuration Reference **94**
Appendix IV: List of Technical Components **96**
Backend (Python 3.11) 96
Dashboard and extension 96
Services and tools 97
Appendix V: Installation and Test Commands **98**
Appendix VI: List of Papers Presented and Published **99**

LIST OF FIGURES

Figure 3.1: Proposed solution workflow: Phase A prepares the profile once; Phase B assists every web form.....17

Figure 4.1: System architecture of the Smart Form Auto-Filling System.....23

Figure 4.2: Docker Compose deployment architecture.....24

Figure 4.3: Backend package structure and allowed dependencies.....25

Figure 4.4: Context diagram (DFD Level 0).....28

Figure 4.5: Data flow diagram, Level 1.....29

Figure 4.6: Data flow diagram, Level 2: Extract and Store Facts (process 3.0) ..30

Figure 4.7: Use case diagram 31

Figure 4.8: Activity diagram: document upload and processing 32

Figure 4.9: Activity diagram: filling a web form through the extension 33

Figure 4.10: Sequence diagram: user registration.....34

Figure 4.11: Sequence diagram: login, authenticated request and logout.....35

Figure 4.12: Sequence diagram: document upload, processing and status polling 36

Figure 4.13: Sequence diagram: form analysis, review and fill.....37

Figure 4.14: Sequence diagram: grounded document question answering.....38

Figure 4.15: Sequence diagram: conflict detection and resolution 39

Figure 4.16: Entity relationship diagram of the MongoDB collections and the ChromaDB chunk index.....40

Figure 4.17: State diagram of a document's processing status.....42

Figure 4.18: State diagram of a structured fact's status 43

Figure 4.19: Class diagram: form-fill agent, classifier, fact service and ontology 44

Figure 4.20: Class diagram: LLM providers, document pipeline and retrieval services 45

Figure 4.21: LangGraph state graph of the form-fill pipeline.....47

Figure 4.22: Flowchart of deterministic field classification 49

Figure 4.23: Flowchart of the decide node.....50

Figure 4.24: Retrieval-augmented question answering pipeline 52

Figure 4.25: Defence-in-depth layers of the implementation.....53

Figure 4.26: Prompt-injection defence applied to every untrusted input.....55

Figure 4.27: State diagram of the extension popup flow.....58

Figure 5.1: Login screen of the dashboard 61

Figure 5.2: Dashboard overview.....62

Figure 5.3: Documents page with upload area and processing status 62

Figure 5.4: Profile page listing extracted facts with provenance.....63
Figure 5.5: Conflicts page with candidate values from different documents.....64
Figure 5.6: Extension popup after scanning a page.....65
Figure 5.7: Extension popup: review before fill.....66
Figure 5.8: Ask page: grounded answer with cited source 67
Figure 5.9: Settings page with live service health 68
Figure 6.1: Continuous integration pipeline and test layers 71

LIST OF TABLES

Table 1.1: Application domains of the system 2
Table 2.1: Summary of the surveyed literature and its relevance 11
Table 2.2: Comparison of related approaches with the proposed system 12
Table 3.1: Existing practice versus the proposed system 15
Table 3.2: Functional requirements and where they are implemented 18
Table 3.3: Non-functional requirements 19
Table 3.4: Technology stack and justification 19
Table 3.5: Implementation status of features 20
Table 4.1: Hardware requirements 24
Table 4.2: Configured thresholds used by the pipeline 26
Table 4.3: Field ontology: concepts per domain 27
Table 4.4: MongoDB collections and their indexes 40
Table 4.5: Status vocabularies used by the system 41
Table 4.6: Security controls and where they are implemented 53
Table 4.7: Principal REST endpoints 56
Table 6.1: Backend test inventory (146 tests) 71
Table 6.2: Dashboard test inventory (22 tests) 73
Table 6.3: Extension test inventory (24 tests) 73
Table 6.4: Representative test cases 74
Table 6.5: Test results (run of 13 October 2026) 75
Table 7.1: Current limitations and proposed work 80
Table IV.1: Backend dependencies 96
Table IV.2: Dashboard and extension framework versions 96

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
bcrypt	Password hashing function based on the Blowfish cipher
CI	Continuous Integration
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
DFD	Data Flow Diagram
DOM	Document Object Model
E2E	End to End
ER	Entity Relationship
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
KYC	Know Your Customer
LLM	Large Language Model
MV3	Manifest Version 3 (Chrome extension platform)
NER	Named Entity Recognition
OCR	Optical Character Recognition
OTP	One-Time Password
PII	Personally Identifiable Information
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SPA	Single-Page Application
TTL	Time To Live
UI	User Interface
UML	Unified Modelling Language
XSS	Cross-Site Scripting
YAML	YAML Ain’t Markup Language

Chapter 1
Introduction

This chapter introduces the Smart Form Auto-Filling System. It explains why filling web forms from personal documents is a problem worth solving, what the project set out to do and what it deliberately leaves out, and how the remaining chapters of the report are organised. Every statement about the system in this report describes the implementation in the project repository, and features that are optional, partial or future are labelled as such.

1.1 BACKGROUND OF THE PROJECT TOPIC

Almost every service that a student or a working adult uses today starts with a web form. Admission portals ask for marks, board names and addresses; banks and insurers ask for identity numbers and income; government portals ask for the same name and date of birth that other portals asked for yesterday; and employers ask applicants to retype a résumé that they have already uploaded. The facts that these forms request are largely the same, and most of them already exist in documents the user holds: a mark sheet, an identity card, a résumé, a salary slip.

Browsers try to reduce this effort with built-in autofill, which stores values a user has typed before and offers them again when a field looks similar. This helps with a postal address or an e-mail address, but it has three structural limits. It can only replay values that the user once typed into a form, so it cannot use the many facts that sit in uploaded documents. It matches fields mostly by HTML attributes and simple heuristics, so a label such as “Name of the guardian” or “Year of passing, Class XII” is easily mismatched. And it keeps no record of where a value came from or how reliable it is. There is also a security dimension: Lin, Ilia and Polakis showed that browser autofill can be abused by pages that include hidden fields, which makes indiscriminate filling a privacy risk [16].

Recent advances in language models make a different approach feasible. Models can read unstructured documents and return structured facts, and retrieval-augmented generation (RAG) [20] lets a model answer from a user's own text rather than from general knowledge. Research on web agents [1], [2], [4] has shown that models can operate real web pages, but it has largely focused on task automation, for which the agent is given broad control of a browser. Form filling needs the opposite design: a narrow, auditable assistant that proposes values from the user's verified data, explains each proposal, and leaves the decision to the user. This project builds such an assistant.

1.2 MOTIVATION AND SCOPE OF THE PROJECT

The motivation is practical. Re-entering the same details is slow, and it is also error-prone: a transposed digit in an identity number or a wrongly formatted date can lead to a rejected application. Where a person has two documents that give different values for the same attribute, for example two spellings of a name or two addresses, a typical autofill tool simply picks one without telling the user. The project therefore aims at three things at once: reduce typing, keep every filled value traceable to evidence, and keep the user in control.

The system is aimed at individuals who fill forms on behalf of themselves. Table 1.1 lists the kinds of portal where the same facts recur. The system does not depend on any one portal: it works on the structure of an HTML form and on a fixed vocabulary of concepts.

Table 1.1: Application domains of the system

Domain	Typical form	Facts that recur
Education	College and scholarship applications	Name, date of birth, board, marks, address, parent details
Banking and insurance	Account opening, KYC update, policy application	Identity numbers, address, income, nominee
Government services	Registration and certificate portals	Name, date of birth, address, identity numbers
Employment	Recruitment portals	Education, experience, skills, links, contact details
Healthcare	Hospital registration	Name, date of birth, blood group, emergency contact

The scope of the implemented system is defined by five functional pillars:

1. **Document intake and processing.** Upload of PDF, DOCX, TXT, PNG and JPG files, parsing, optional OCR, and background processing with visible progress.
2. **Fact extraction with provenance.** A deterministic extractor and an optional LLM extractor produce facts whose origin, evidence text, source page and confidence are stored.
3. **Profile and conflict management.** The user sees one resolved value per concept, can edit or confirm values, and resolves disagreements between documents.
4. **Semantic form assistance.** A browser extension describes the fields of the current page; a LangGraph pipeline maps fields to concepts and proposes values for review.
5. **Grounded question answering.** A RAG module answers questions about the user's own documents and cites the source passage.

The following are explicitly **out of scope**: automatic submission of forms, filling of password, payment-card, one-time-code and CAPTCHA fields, browsers other than Chrome, mobile clients, and a multi-user or multi-tenant administration console. Scanned documents are supported only when the optional PaddleOCR engine is installed.

1.3 PROBLEM STATEMENT

Existing autofill tools cannot build a user profile from the user's documents, cannot explain or evidence the values they insert, handle conflicting sources poorly, and in some designs act on hidden or sensitive fields. The problem addressed by this project is to design and implement a system that (a) extracts structured, evidence-backed facts from personal documents, (b) maps the fields of an arbitrary web form to those facts by meaning rather than by exact attribute names, (c) detects and surfaces disagreements between sources, and (d) fills values only after explicit user approval, without reading existing field values, touching sensitive controls or submitting the form.

1.4 AIM AND OBJECTIVES

The aim of the project is to build a privacy-first, human-in-the-loop form-filling assistant driven by the user's own documents. The objectives are:

1. to parse PDF, DOCX, TXT and image documents and extract facts that conform to a fixed ontology of personal concepts;
2. to store, for every fact, its source document, page, evidence text, extraction method and confidence;
3. to compute cross-document conflicts from the active facts and let the user resolve them;
4. to classify the fields of a web form into ontology concepts using deterministic rules first and a language model only for the remainder;
5. to resolve each field's value from the profile and, when absent, from retrieved document evidence that the model must quote;
6. to apply confidence and sensitivity rules so that nothing uncertain or sensitive is filled without confirmation;
7. to defend against prompt injection and to isolate every user's data from every other user's; and
8. to verify the implementation with automated unit, integration and browser end-to-end tests.

1.5 SALIENT CONTRIBUTIONS

The project's contribution is the integration of several known techniques into one auditable pipeline whose design choices favour privacy and traceability:

- **Ontology-driven semantics:** A YAML ontology of 91 concepts with 295 aliases, autocomplete tokens, cardinality, data type and sensitivity flags is the single vocabulary shared by extraction, classification and filling.
- **Dual-track extraction with provenance:** Deterministic rules run first; an LLM track is optional; results are merged so that agreement raises confidence slightly and disagreement becomes a user-visible conflict.
- **Batch LangGraph pipeline:** One seven-node graph serves every recommendation endpoint. Classification is deterministic first, one batched LLM call handles only ambiguous fields, and values from document retrieval must be quoted from the evidence.

- **Conflict computation without a second source of truth:** Conflicts are derived from active facts, so deleting or reprocessing a document cannot leave a stale conflict.
- **Privacy-first Manifest V3 extension:** On-demand access through activeTab, no reading of field values, skipping of sensitive controls, a review step before every fill, and no submission.
- **Injection-aware design:** Page text, document text and retrieved chunks are treated as untrusted data in every model call.
- **Reproducible deployment and verification:** A single Docker Compose project and 196 automated tests, including a Playwright test of the built extension in real Chromium.

Some of these properties echo individual ideas in the surveyed literature. The project's own claim is limited to the combination, which Chapter 2 shows is not offered by the works reviewed.

1.6 ORGANISATION OF THE REPORT

The report follows the college's seven-chapter format. Chapter 2 surveys the literature on web agents, language-model memory and retrieval, form understanding and automated form filling, and ends with the gap this project addresses. Chapter 3 states the problem formally, compares existing practice with the proposed solution, lists functional and non-functional requirements, and records which features are implemented, optional or future. Chapter 4 documents the design through architecture, data-flow, behavioural and structural diagrams, the database design, the principal algorithms and the security design. Chapter 5 presents snapshots of the running system. Chapter 6 describes the testing strategy and records the results. Chapter 7 concludes and lists future work. The References and five appendices follow.

CHAPTER SUMMARY

Form filling is a repetitive task whose data already exists in the user's documents, but native autofill cannot use those documents, cannot evidence a value and cannot arbitrate between sources. The project builds an assistant that extracts evidence-backed facts, maps form fields to them by meaning, surfaces conflicts, and fills only

what the user approves. The scope, objectives and contributions were stated, and the organisation of the report was outlined.

Chapter 2
Literature Survey

This chapter reviews published work that bears on document-driven form filling. It first sets out the background concepts, then reviews nineteen papers grouped by theme, compares the approaches with the proposed system, and concludes with the gap that motivates the project. Bibliographic details of every paper were checked against the publisher, proceedings or arXiv record; where only a preprint record could be confirmed, the reference says so.

2.1 INTRODUCTION TO THE OVERALL TOPIC

Automatic form filling sits at the junction of four research areas. **Information extraction** turns unstructured documents into structured facts; recent surveys of form understanding show a move from layout rules to transformer models [15]. **Retrieval-augmented generation** grounds a language model's output in retrieved passages [20], using dense sentence embeddings such as Sentence-BERT [21] and compact models such as MiniLM [22] to find relevant text. **Web agents** use language or vision-language models to act on web pages [1], [2], [3], [4], and surveys organise them into perception, reasoning, planning and action [10], [11]. Finally, **security and privacy** research examines how autofill and LLM-integrated applications can be abused [16], [23].

A form-filling assistant must use all four areas, and it has constraints that general web agents do not. The values it inserts are personal; they must be correct and attributable; the page it reads is controlled by a third party and may be hostile; and the user must remain the decision-maker. These constraints shape both the choice of techniques and the gap identified at the end of the chapter.

Background concepts used in this project

- **Autofill:** Browser feature that stores values previously typed and replays them in fields matched by attributes such as name and autocomplete tokens. It is the baseline against which the project is positioned.
- **Ontology:** A controlled vocabulary of concepts (for example personal.date_of_birth) with aliases, data types and flags. The project uses it so that extraction, classification and filling share meaning.
- **Retrieval-augmented generation:** A language model answers from passages retrieved from an index rather than from its parameters alone [20].
- **Agent orchestration:** A graph of steps with shared state, here implemented with LangGraph [31].
- **Prompt injection:** Instructions hidden in data that a model reads, which can make an LLM-integrated application act against its user [23], [26].

2.2 EXHAUSTIVE LITERATURE SURVEY

2.2.1 Web Agents and Benchmarks

WebArena [1] provides a self-hostable web environment containing realistic functional websites and a set of long-horizon tasks, against which language-model agents are scored by whether the task's goal is reached. It showed that agents were far from human performance on realistic web tasks. Mind2Web [2] contributes a dataset of more than two thousand open-ended tasks collected from 137 real websites across 31 domains, and a two-stage approach in which a small model first filters page elements before a larger model chooses the action. Together these benchmarks establish how web agents are evaluated, but neither addresses personal data or user approval.

SeeAct [3] studies whether GPT-4V can act as a web agent. With a human mapping the model's plan onto page elements it completed 51.1% of tasks on live sites, and the authors identify **grounding**, linking a textual plan to the correct element, as the main bottleneck. WebVoyager [4] builds an end-to-end multimodal agent that works on fifteen live websites and reports a 59.1% task success rate. CogAgent [5] is an 18-billion-parameter vision-language model that reads screenshots at 1120 by 1120 resolution, and SeeClick [6] introduces GUI-grounding pre-training and the ScreenSpot benchmark. Ferret-UI [7] extends grounded UI understanding to mobile

screens with an any-resolution design. WorkArena [8] measures agents on common enterprise knowledge-work tasks on the ServiceNow platform and introduces the BrowserGym environment, concluding that agents are promising but short of full automation. LCoW [9] learns a module that contextualises complex web pages so that a decision-making model can act on them more reliably.

Relevance and limits. These works show that models can find the right element on a page and that grounding is hard. The project borrows the lesson that precise grounding matters, but it avoids the problem for form fields by using the DOM directly: each field is a descriptor with a stable identifier and the fill action targets that identifier. Unlike an autonomous agent, the project's extension takes no action that the user has not approved.

2.2.2 LLM Agent Frameworks, Knowledge and Memory

Wang et al. [11] survey LLM-based autonomous agents and propose a unified framework covering profile, memory, planning and action modules, together with applications and evaluation methods. A companion survey on the memory mechanism of LLM agents [14] organises memory by source, form and operation, which is a useful lens for the project's own memory: the profile of facts is persistent, evidence-linked memory, and saved answers are explicitly user-initiated memories. Pan et al. [12] present a roadmap for combining large language models with knowledge graphs, arguing that explicit, structured knowledge complements the black-box parametric knowledge of a model. HippoRAG [13] proposes a retrieval framework inspired by long-term memory that combines a language model, a knowledge graph and a graph-ranking step to improve retrieval for knowledge integration.

Relevance and limits. The project's ontology plays the role of a small, explicit knowledge structure: each fact is a typed, sourced assertion, which is the benefit that Pan et al. attribute to structured knowledge. The project does not build a knowledge graph or multi-hop retrieval; its questions are short factual lookups, for which single-step dense retrieval with a similarity cutoff is adequate.

2.2.3 Form Understanding, Autofill and Form Security

Abdallah et al. [15] review transformer and language-model approaches to understanding forms in scanned documents, including key-information extraction, and the benchmark datasets used to evaluate them. This is the extraction side of the

problem; the review supports the project's use of an optional OCR step and of a language model to read unstructured documents, but it treats forms as inputs to be read, not as targets to be filled.

Lin, Ilia and Polakis [16] analyse the privacy threats of browser form autofill and show that pages can exploit hidden fields and the autofill preview mechanism to obtain personal information, in all Chromium-based browsers they tested. This directly motivates three design rules in the project: sensitive and hidden fields are skipped, the extension never reads existing field values, and nothing is filled until the user approves it.

FormLM [17] recommends question and option ideas for the *creation* of online forms by augmenting a pre-trained language model with form structure, trained on a dataset of 62 thousand forms. LAFF [18] suggests values for categorical fields in data-entry forms by learning dependencies between fields with Bayesian networks from historical input, reporting a mean reciprocal rank above 0.73. FormNexus [19] derives semantic insights from the text, DOM structure and visual proximity of form elements, stores them in a form entity-relation graph, and uses an LLM in a feedback loop to infer input constraints for test generation.

Relevance and limits. FormLM models forms from the designer's side; LAFF predicts categorical values from earlier entries rather than from documents; FormNexus targets test generation. All three show that field semantics can be inferred from text and structure, which the project does with ontology aliases and nearby-text cues. None of them builds a personal profile from documents or involves the user in a review step.

2.2.4 Supporting Techniques

Lewis et al. [20] introduced retrieval-augmented generation for knowledge-intensive tasks, pairing a sequence-to-sequence model with a dense index. Sentence-BERT [21] showed that siamese BERT networks produce sentence embeddings that can be compared by cosine similarity at a fraction of the cost of pairwise BERT scoring, and MiniLM [22] compresses transformers by distilling self-attention, which is the family of model that the project's default embedding model (all-MiniLM-L6-v2, 384 dimensions) belongs to. Greshake et al. [23] demonstrate indirect prompt injection against LLM-integrated applications, in which instructions placed in retrieved or web

content hijack the application; the OWASP guidance [26] lists prompt injection first among its risks for LLM applications. These works justify the project's RAG design and its treatment of every page, document and chunk as untrusted input.

Table 2.1 summarises the nineteen surveyed works, the main idea of each and the reason it does not by itself solve the problem of this project.

Table 2.1: Summary of the surveyed literature and its relevance

Ref.	Work (venue, year)	Main idea	Limitation for this project
[1]	WebArena (ICLR 2024)	Realistic, self-hostable web tasks	Task automation; no personal data or user approval
[2]	Mind2Web (NeurIPS D&B 2023)	2,000+ tasks, 137 sites, two-stage agent	Benchmark of agent actions, not form values
[3]	SeeAct (ICML 2024)	GPT-4V web agent; grounding bottleneck	Visual grounding needed; autonomous action
[4]	WebVoyager (ACL 2024)	End-to-end multimodal agent, 15 live sites	Autonomous; evaluation by model judge
[5]	CogAgent (CVPR 2024)	18B screenshot-based VLM	Heavy model; screenshot-only
[6]	SeeClick (ACL 2024)	GUI-grounding pre-training, ScreenSpot	Element location, not field meaning
[7]	Ferret-UI (ECCV 2024)	Mobile UI understanding	Mobile screens, not web forms
[8]	WorkArena (ICML 2024)	Enterprise knowledge-work benchmark	Benchmark; agents short of full automation
[9]	LCoW (ICLR 2025)	Learned page contextualisation	Aids agents; no profile or evidence
[10]	GUI Agents survey (ACL Findings 2025)	Perception, reasoning, planning, acting	Survey only
[11]	LLM agents survey (Front. Comput. Sci. 2024)	Unified agent framework	Survey only
[12]	LLMs + KGs roadmap (IEEE TKDE 2024)	Structured knowledge for LLMs	No form-filling application
[13]	HippoRAG (NeurIPS 2024)	Graph-based long-term memory retrieval	Multi-hop retrieval not needed here
[14]	Agent memory survey (arXiv 2024)	Memory by source, form, operation	Survey only
[15]	Form understanding review (arXiv 2024)	Transformers for scanned forms	Reads forms; does not fill them
[16]	Fill in the Blanks (CCS 2020)	Privacy threats of browser autofill	Analysis of threats, not a solution
[17]	FormLM (EMNLP 2022)	Recommends form creation ideas	Form design, not filling
[18]	LAFF (ACM TOSEM)	Bayesian suggestion of categorical values	Uses earlier entries, not documents
[19]	FormNexus (ISSTA 2024)	LLM constraint inference for form tests	Testing, not assistance

2.3 COMPARATIVE ANALYSIS

Table 2.2 compares four families of approach on the properties that matter to this project. The comparison is based on the capabilities described in the cited papers and on documented browser behaviour; “not reported” means the reviewed papers do not describe the property.

Table 2.2: Comparison of related approaches with the proposed system

Property	**Browser autofill**	**Web agents [1]-[10]**	**Form-field prediction [17]-[19]**	**Proposed system**
Reads user documents	No	Not reported	No	Yes (PDF, DOCX, TXT, image)
Stores evidence for each value	No	Not reported	No	Yes (source, page, text, confidence)
Handles conflicting sources	No	Not reported	No	Yes (computed conflicts, user resolves)
Matches fields by meaning	Heuristics	Model-based	Yes	Ontology aliases, then LLM
User approval before action	Partial	Typically autonomous	Not applicable	Mandatory, per field
Avoids sensitive / hidden fields	Known weaknesses [16]	Not reported	Not applicable	Skipped in extension and backend
Prompt-injection defence	Not applicable	Not reported	Not reported	Tagged data, patterns, quote checks

2.4 RESEARCH AND DEVELOPMENT GAP

The surveyed work leaves the following gap. Web agents [1], [3], [4] demonstrate action on pages, but they are designed for autonomy rather than for review of personal data. Form-oriented research [17], [18], [19] infers field semantics or values for form design, value suggestion and testing, but not from a user’s own documents. Extraction research [15] reads documents but does not connect them to forms. Security research [16], [23] documents risks but does not offer an integrated assistant that avoids them. **Among the works reviewed, none combines document-derived profile building with evidence, cross-document conflict handling, semantic field matching, injection-aware model use and mandatory user review in one deployable system.** This project addresses that combination.

2.5 PROPOSED APPROACH

The proposed approach keeps the language model in a supporting role. Facts enter the system through a deterministic extractor and an optional LLM extractor and are stored with evidence. A field is classified by rules first; the model sees only ambiguous fields, in one batch, with sanitised descriptions. A value comes from the stored profile, and only if the profile lacks it does the system retrieve document passages and ask the model for a value that it must quote from a cited passage. Every proposal reaches the user with a reason and a source, and the extension writes only approved values and then verifies that the page kept them. Chapters 3 and 4 detail this design.

CHAPTER SUMMARY

The literature shows that models can understand and act on web pages, that grounding is the main difficulty, that autofill carries privacy risks, and that retrieval and structured knowledge improve factual reliability. No reviewed work provides evidence-preserving, conflict-aware, user-approved filling from personal documents. That gap defines the problem addressed in Chapter 3.

Chapter 3
Problem Definition and Proposed Solution

This chapter turns the gap identified in Chapter 2 into a precise problem, states the requirements that a solution must meet, and describes the proposed solution at the level of workflow and technology choice. It closes with an honest account of which features are implemented, which are optional, and which belong to future work.

3.1 PROBLEM DEFINITION

Let D be the set of documents uploaded by a user u. Let K be a finite ontology of concepts, each with a data type, a cardinality (single or multiple valued), and flags marking it sensitive or not fillable. The system must derive from D a set of facts $F(u)$, where each fact is a tuple (concept, value, origin, source document, page, evidence text, confidence), and must, for any web form with fields X presented to the user, produce for each field x in X a recommendation $r(x)$ consisting of a status, an optional value, a confidence, a reason and a source.

A solution is acceptable only if it satisfies the following constraints:

1. **Evidence.** A value is proposed only if it is stored in $F(u)$ or appears verbatim in a retrieved passage of D that the system can cite.
2. **Conflict transparency.** If $F(u)$ contains two or more distinct values for a single-valued concept, no value is filled automatically; the candidates are shown.
3. **Sensitivity.** Passwords, payment details, one-time codes and CAPTCHAs are never filled. Concepts flagged sensitive, and values the system only inferred, require confirmation.
4. **Consent.** The page is changed only after the user approves each field, and the form is never submitted by the system.

5. **Isolation.** No operation on one user's data may read, return or affect another user's data.
6. **Untrusted input.** Text from web pages and documents is treated as data, never as instructions to the language model.

3.2 CHALLENGES

- **Heterogeneous forms:** Labels, placeholders, names and identifiers vary between sites, and the same concept may be phrased as "DOB", "Date of Birth" or "Birth date". Rule-based matching must be precise, and ambiguity must be resolved without guessing.
- **Context-dependent fields:** A field labelled "Name" inside a "Father's details" section is not the user's name. The system must recognise this and avoid filling the user's own value.
- **Conflicting sources:** Two documents may spell a name or write an address differently. Choosing silently would hide an error.
- **Language-model risks:** Models can invent values and can be manipulated by text they read [23]. Every model output must be validated and grounded.
- **Browser security model:** A Manifest V3 extension has limited, permission-gated access to pages [32]. Tokens must not reach scripts running in the page.
- **Framework-controlled inputs:** Pages built with React or similar libraries ignore naive assignments to the value property, so a fill must emulate user input and verify the outcome.
- **Unreadable documents:** Scanned files without OCR cannot yield text. The system must say so rather than treat empty output as a result.

3.3 EXISTING SYSTEM AND ITS LIMITATIONS

In current practice a person either types every value by hand, copies from a résumé or document, or relies on browser autofill. Table 3.1 contrasts this practice with the proposed system on the properties that matter to the problem definition.

Table 3.1: Existing practice versus the proposed system

Aspect	Existing practice (manual / browser autofill)	Proposed system

Aspect	Existing practice (manual / browser autofill)	Proposed system
Source of values	Typed by the user, or values saved from earlier forms	Facts extracted from the user's uploaded documents, plus user-entered facts
Field matching	Attribute names and heuristics	Ontology aliases, autocomplete tokens and nearby text, then one batched LLM call for the rest
Evidence for a value	None	Source document, page, evidence text, method, confidence
Conflicting values	One value is chosen without notice	Conflict is computed and shown; the user chooses
Sensitive and hidden fields	Filled by indiscriminate matching [16]	Skipped in the extension and blocked again in the backend
User control	Preview or one-click fill	Per-field review with reason and source; user can edit or skip
After filling	No verification	DOM re-read to confirm the page kept each value; results reported
Submission	User or page script	Never submitted by the system

3.4 PROPOSED SOLUTION OVERVIEW

The solution has two phases, shown in Figure 3.1. In **Phase A** the user uploads documents once. The backend parses each document, extracts facts and merges them, stores the facts in MongoDB with their provenance, and indexes document chunks in ChromaDB. In **Phase B**, on any web page, the extension describes the form's fields, the backend classifies them and proposes values, the user reviews the proposals, and the extension fills the approved values and verifies the result.

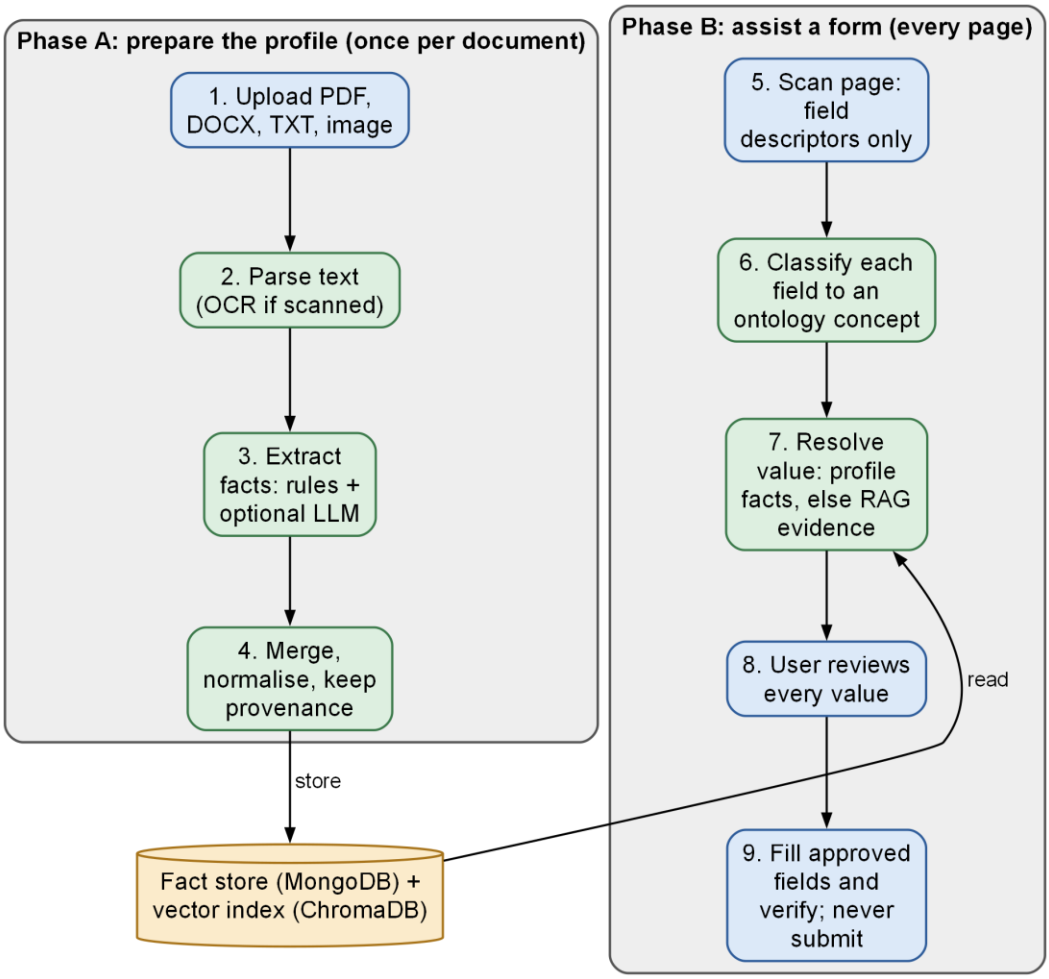


Figure 3.1: Proposed solution workflow: Phase A prepares the profile once; Phase B assists every web form

Workflow summary

Upload, parse, extract, merge, index; then scan, classify, resolve, decide, review, fill, verify. Upload and processing are asynchronous, so the dashboard polls the document's status. Form analysis is also asynchronous: the analyze endpoint returns immediately with status 202 and the popup polls the recommendations until they are ready. The same batch pipeline serves the extension, the synchronous recommendation endpoints and the agent endpoints, so behaviour is identical for every client (architecture decision AD-002).

Advantages of the proposed solution

- **Traceable values:** every proposal states where it came from and how certain the system is.

- **Less typing, fewer errors:** values are normalised and formatted to the target field (dates, options, radio groups).
- **Safe by construction:** sensitive and hidden controls are skipped in two places, and a fill never submits a form.
- **Predictable cost:** the language model is called only when rules cannot decide, and the default provider is a deterministic mock that makes no network call.
- **Portable deployment:** the whole stack starts with one Compose command.

3.5 REQUIREMENTS

Functional requirements

Table 3.2 lists the functional requirements and the module in which each is implemented.

Table 3.2: Functional requirements and where they are implemented

ID	Requirement	Implemented in
FR1	Register, log in, log out, delete account	services/auth.py; api/v1/auth.py
FR2	Upload PDF, DOCX, TXT, PNG, JPG with extension, magic-byte, size and duplicate checks	services/documents.py
FR3	Process documents in the background with visible stage and progress	processing/runner.py; processing/pipeline.py
FR4	Parse text; use OCR if installed; otherwise mark requires_review	processing/extractor.py
FR5	Extract facts by deterministic rules over the ontology	extraction/deterministic.py
FR6	Optionally extract facts with an LLM and merge both tracks	extraction/llm_extraction.py; extraction/merge.py
FR7	Store provenance: origin, raw value, evidence, page, method, confidence	models/documents.py (StructuredFact)
FR8	View, add, edit, confirm and delete profile facts	services/facts.py; services/profile.py
FR9	Detect cross-document conflicts and let the user resolve them	services/facts.py
FR10	Answer questions from the user's documents with cited sources	services/retrieval.py
FR11	Detect form fields in the page, skipping sensitive and hidden controls	extension/src/content/detector.ts
FR12	Classify fields to ontology concepts	agents/field_classifier.py
FR13	Recommend values with status, reason, source and candidates	agents/graph.py
FR14	Record approve, edit, reject, skip decisions; save answers on request	services/form_filling.py; popup AutofillFlow.tsx
FR15	Fill approved values and verify them; never submit	extension/src/content/autofill.ts
FR16	Delete all user data (vectors, facts, files) with or without the account	services/user_data.py

ID	Requirement	Implemented in
FR17	Report service health and form-session activity	services/health.py; dashboard Settings and Activity pages

Non-functional requirements

Table 3.3 lists the non-functional requirements and how the implementation meets them.

Table 3.3: Non-functional requirements

Category	Requirement	How it is met
Security	Authenticated, isolated, revocable access	bcrypt, JWT with jti revocation, user_id filter on every query (Section 4.9)
Privacy	Minimal collection from pages	Descriptors only, has_value flag instead of values, activeTab on demand
Reliability	Failures are explicit	Typed errors; no silent fallback to hash vectors or other providers
Responsiveness	Interactive analysis must not hang	Interactive LLM calls capped at 20 s with no retry; popup poll timeout 180 s
Maintainability	Verifiable changes	196 automated tests, ruff, black, type checks, CI workflow
Portability	Reproducible environment	Docker Compose; configuration through environment variables
Observability	Diagnosable without leaking data	Structured JSON logs with request ID and redaction
Usability	User can understand every proposal	Reason and source on each recommendation; Ready or Check status

3.6 TECHNOLOGY STACK AND JUSTIFICATION

Table 3.4 lists the technologies used in each layer and the reason for each choice; the architecture decisions cited (AD-003 to AD-008) are recorded in the repository.

Table 3.4: Technology stack and justification

Layer	Technology	Justification
API	FastAPI, Pydantic v2, Uvicorn [27]	Async I/O, typed request and response models, generated OpenAPI documentation
Fact store	MongoDB 8 via Motor [29]	Heterogeneous fact shapes; per-user queries; TTL indexes for revoked tokens
Vector index	ChromaDB over HTTP v2 [30]	Self-hostable; collection metadata records model and dimension (AD-004, AD-005)
Embeddings	all-MiniLM-L6-v2, 384 dimensions [21], [22]	Small, CPU-friendly default; mismatch with the collection is reported
Agent graph	LangGraph [31]	Explicit state graph with conditional edges; deterministic order of steps
LLM access	Google, OpenAI, Anthropic through httpx; mock default	No SDK lock-in; one interface; deterministic offline mock for tests (AD-007)

Layer	Technology	Justification
Jobs	Celery with Redis; inline asyncio mode	Docker processing outlives API requests; local runs need no queue (AD-003)
Documents	pdfplumber, pypdfium2, python-docx, Pillow, OpenCV; optional PaddleOCR	Parsing and image pre-processing; OCR optional to avoid a heavy dependency (AD-008)
Auth	bcrypt [25], PyJWT [24]	Salted adaptive hashing; short-lived bearer tokens
Dashboard	React 19, TypeScript, Vite, React Router [28]	Component model; typed API client
Extension	Chrome Manifest V3, React 18, Vite [32]	Current extension platform; on-demand injection with activeTab
Delivery	Docker Compose, Nginx [33]	Single-command stack; production-style static serving with security headers
Testing	pytest, Vitest, Testing Library, Playwright [34]	Unit, integration and real-browser tests

3.7 FEASIBILITY ANALYSIS

- **Technical:** every component is open source and runs on a single developer machine; the embedding model runs on CPU. The project was built and tested on a Windows 11 laptop with a 13th-generation Intel Core i5-13450HX (10 cores, 16 threads) and 15.7 GB of memory.
- **Economic:** the default configuration uses the mock provider and local storage, so running and testing the system incurs no API charges. A real LLM provider is optional and billed by that provider; real-provider tests are opt-in.
- **Operational:** the user uploads documents once and then uses a popup; no administrative role is needed.
- **Legal and ethical:** the system stores identity data. The security design (Section 4.9) and the documented retention and deletion behaviour are therefore part of feasibility, not an afterthought.

3.8 IMPLEMENTATION STATUS AND LIMITATIONS

Table 3.5 separates implemented features from optional, partial and future ones, as recorded in the repository documentation and verified by the tests of Chapter 6.

Table 3.5: Implementation status of features

Feature	Status	Note
Authentication, token revocation, rate limits	Implemented	30-minute JWT; logout and deletion revoke the token
Document upload and processing (PDF, DOCX, TXT, images)	Implemented	Inline mode locally; Celery with Redis in Compose

Feature	Status	Note
Deterministic extraction and merge	Implemented	Precision-first; heuristic name line marked inferred
LLM extraction and classification	Implemented, opt-in	Default provider is mock; real providers need a key
Conflict detection and resolution	Implemented	Computed from active facts
RAG retrieval and grounded answers	Implemented	Similarity cutoff 0.35; cited sources only
Seven-node LangGraph pipeline	Implemented	Used by all recommendation endpoints
Chrome extension scan, review, fill, verify	Implemented	Verified in a real Chromium test
Dashboard (11 routes)	Implemented	Login, register, dashboard, documents, document details, profile, discovered facts, conflicts, reviews, ask, activity, settings
OCR for scanned documents	Optional	Needs INSTALL_OCR=true; otherwise requires_review
Cloudinary storage	Optional	STORAGE_PROVIDER=cloudinary
Real-provider LLM tests	Opt-in, not run	May incur charges; synthetic data only
Other browsers, mobile, multi-step wizard re-scan	Future	Chapter 7

The principal limitations are: dependence on OCR for scans; a vocabulary limited to 91 concepts, so a field with no matching concept is left for the user; a static scan model in which fields added after the scan need a re-scan; in-process rate limiting that does not span multiple replicas; and a dashboard token kept in browser local storage, which is protected by a restrictive Content Security Policy and a short expiry but would be exposed by a successful script injection (architecture decision AD-010).

CHAPTER SUMMARY

The problem was defined formally with six acceptance constraints. Challenges were identified, existing practice was contrasted with the proposed two-phase solution, seventeen functional and eight non-functional requirements were tabulated, technology choices were justified, and the status of every feature was recorded together with the principal limitations.

Chapter 4
Design

This chapter documents the design of the Smart Form Auto-Filling System as implemented. It moves from the architecture and deployment, through the flow of data and the behaviour of the system, to the database, class structure, core algorithms, security design and application programming interface. Each diagram is explained in the text, and each was drawn from the corresponding source modules named alongside it.

4.1 BLOCK DIAGRAM AND SYSTEM ARCHITECTURE

Figure 4.1 shows the architecture. The system is a three-tier design with an external-services ring. The **presentation tier** has two clients: a React 19 dashboard served by Nginx, and a Chrome Manifest V3 extension whose popup (React 18) and content script cooperate through extension messages. The **application tier** is a FastAPI service. Every request first passes middleware that applies cross-origin rules, request-identifier logging with redaction, rate limiting and bearer-token authentication. Eleven routers expose the REST interface and delegate to services, which in turn use the document pipeline, the form-fill LangGraph pipeline and the field ontology. The **data tier** consists of MongoDB, which is the source of truth for users, documents, facts and form sessions; ChromaDB, which holds the vector index of document chunks; a file store (a local volume by default, or Cloudinary); and Redis, which is used only as the Celery broker in Compose.

The client never touches a database. All access to MongoDB, ChromaDB and the file store passes through services that receive the authenticated user identifier and scope every query by it. The external ring contains optional services: a language-model provider (Google, OpenAI or Anthropic, with a deterministic mock as the default), the sentence-transformer embedding model, an optional OCR engine and the Celery worker, which runs the same pipeline code as the API process.

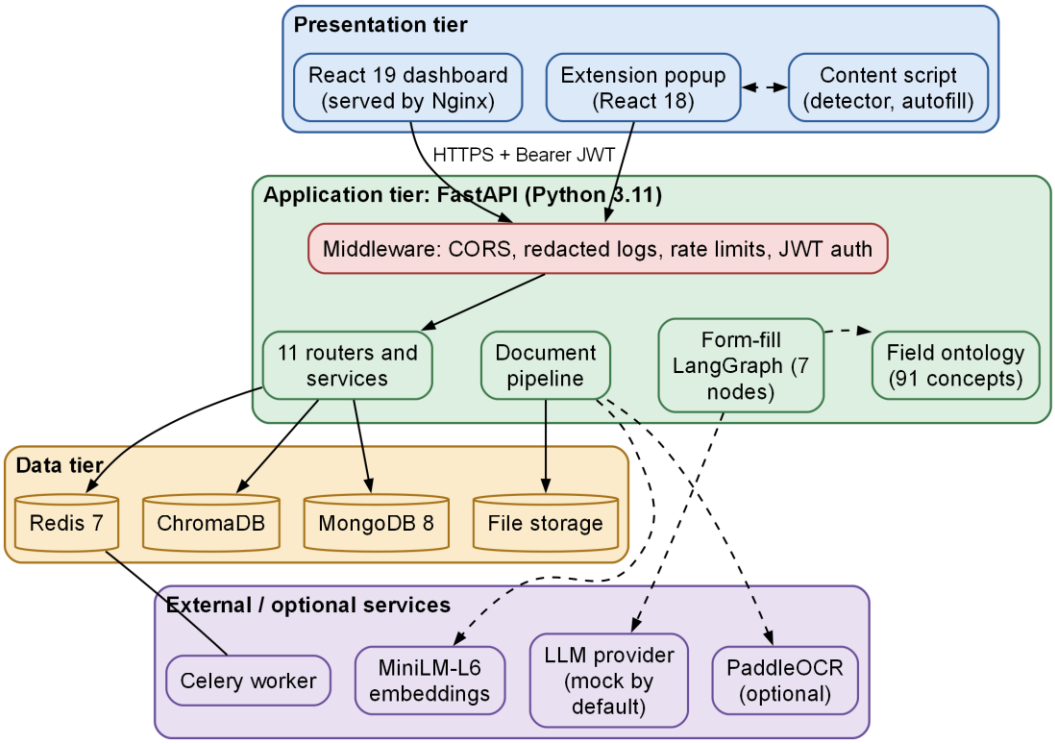


Figure 4.1: System architecture of the Smart Form Auto-Filling System

4.2 HARDWARE AND SOFTWARE DESCRIPTION

Deployment architecture

Figure 4.2 shows the Docker Compose deployment. Seven services share the network smart-form-internal: frontend (Nginx, host port 5173), backend (Uvicorn, port 8000), worker (Celery), mongodb (mongo:8.0), chromadb (host port 8001), redis (redis:7-alpine) and extension-builder, which builds the extension bundle. MongoDB, ChromaDB and Redis use named volumes. The browser reaches the dashboard through Nginx and the API directly with a bearer token. A separate Compose file, docker-compose.test.yml, defines an isolated project with its own ports and volumes so that the end-to-end test can run beside the development stack.

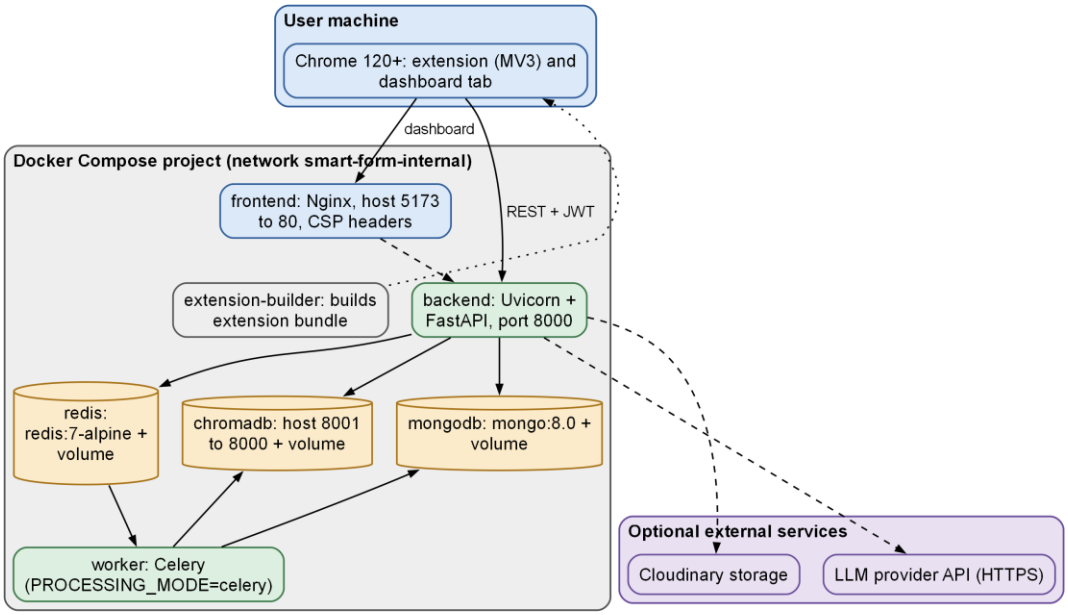


Figure 4.2: Docker Compose deployment architecture

Hardware

The system runs on a general-purpose computer; no GPU is required because the embedding model runs on the CPU.

Table 4.1 records the machine on which the system was developed and the test run of Chapter 6 was made. These are measured facts about the development machine, not minimum requirements: no benchmark of smaller machines was performed.

Table 4.1: Hardware requirements

Item	Value used for development and testing
Processor	13th Gen Intel Core i5-13450HX, 10 cores, 16 logical processors
Memory	15.7 GB
Operating system	Microsoft Windows 11 Home Single Language
Container runtime	Docker 29.4.0 with Compose v2
Python / Node.js	Python 3.11.9 / Node.js 22.14.0
Browser	Google Chrome 155.0.8059.40 and Playwright-managed Chromium
Disk	Named volumes for MongoDB, ChromaDB and Redis; local upload directory

Software

Backend dependencies are pinned to compatible ranges in requirements.txt: FastAPI, Pydantic 2, bcrypt, PyJWT, SlowAPI, httpx, Motor and PyMongo, LangGraph 1.x, pdfplumber, pypdfium2, python-docx, Pillow, OpenCV (headless), sentence-

transformers and Celery with Redis. PaddleOCR and PaddlePaddle are in a separate optional file. The dashboard uses React 19, React Router 7, TypeScript and Vite; the extension uses React 18, TypeScript and Vite with a second build for the content script. Appendix IV lists the components in full.

4.3 BACKEND MODULE DESIGN

Figure 4.3 shows the packages of the backend and the direction of their dependencies. Routers in app/api/v1 contain no business logic; they validate input, apply rate limits and call services. Services coordinate repositories, storage, the pipeline and the agent package. The agents package holds the form-fill graph, the field classifier, the LLM provider abstraction and value formatting. The processing and extraction packages implement document parsing, chunking, deterministic and LLM extraction and merging. The ontology package loads the YAML vocabulary. Cross-cutting security code (prompt-safety helpers, JWT and bcrypt, configuration, logging and rate limiting) is shown in red.

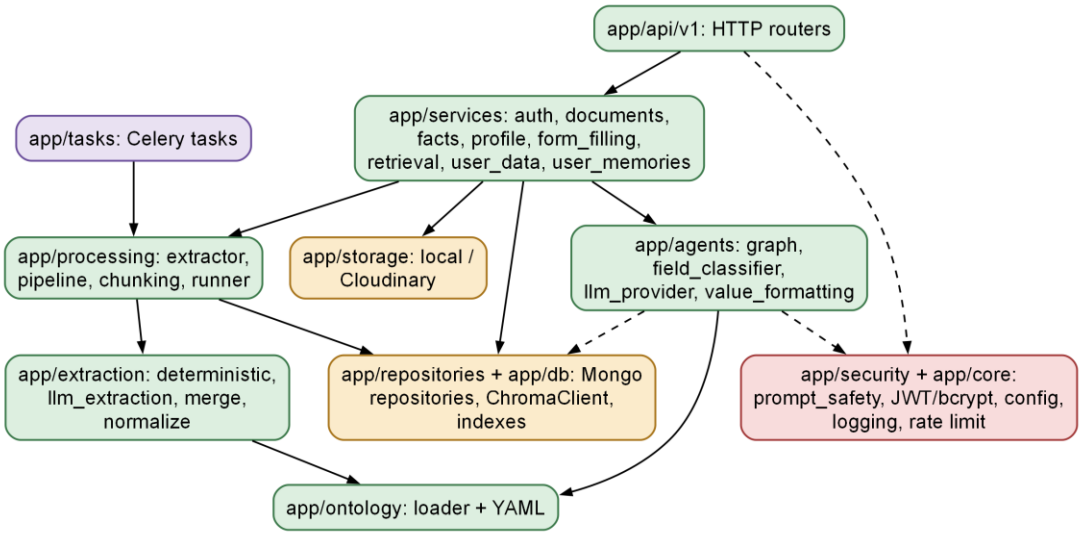


Figure 4.3: Backend package structure and allowed dependencies

Configuration

app/core/config.py defines a validated Settings class. At start-up it rejects a placeholder or short JWT secret, an unknown JWT algorithm or LLM provider, a real provider without an API key, a chunk overlap that is not smaller than the chunk size, and Cloudinary storage without credentials. The thresholds that govern the pipeline are all configurable and are listed in Table 4.2.

Table 4.2: Configured thresholds used by the pipeline

Setting	Default	Used for
CLASSIFIER_CONFIDENCE_THRESHOLD	0.75	Minimum confidence for a classification to count as resolved
AUTOFILL_MIN_CONFIDENCE	0.85	Minimum confidence for a value to be marked ready
RAG_MIN_SIMILARITY	0.35	Cosine similarity cutoff for retrieved chunks
RAG_TOP_K	4	Chunks retrieved per query
RAG_CHUNK_SIZE / RAG_CHUNK_OVERLAP	500 / 50	Characters per chunk and overlap
JWT_ACCESS_TOKEN_EXPIRE_MINUTES	30	Access-token lifetime
AUTH_LOGIN_RATE_LIMIT_ATTEMPTS / WINDOW	5 / 60 s	Login limiter per key
MAX_UPLOAD_SIZE_MB	10	Largest accepted upload
LLM_MAX_RETRIES / LLM_TIMEOUT_SECONDS	2 / 60	Retry policy for non-interactive LLM calls
Interactive LLM timeout	20 s, no retry	Field classification and field-value calls (fixed in code)
LLM_EXTRACTION_MAX_CHARS	18000	Document text window per extraction call

Document processing

app/processing/pipeline.py runs six stages, recorded in the processing_records collection and reflected in the document's progress (10, 30, 55, 75, 90 and 100 per cent for validation, parsing, extraction, storage, indexing and completed). The pipeline parses the file, runs the deterministic extractor, optionally runs the LLM extractor, merges candidates, replaces the document's facts, and chunks, embeds and indexes the text. Reprocessing first deletes the document's previous facts and chunks, so nothing is duplicated and nothing stale remains. An indexing failure does not lose the extracted facts: it is recorded as index_status failed so that the user can see RAG is unavailable for that document.

Ontology

The ontology (app/ontology/field_ontology.yaml, version 3.0.0) defines 91 concepts in 18 domains (Table 4.3). Each concept has a canonical name, a description, aliases (295 in total), a data type (string, date, email, phone, postal_code or boolean), a cardinality (56 single-valued and 35 multiple-valued concepts) and optional autocomplete tokens (16 concepts), a sensitivity flag (12 concepts), a fillable flag (the two legal concepts, consent and signature, are not fillable) and a derived_from link (first, middle and last name are derived from the full name).

Table 4.3: Field ontology: concepts per domain

Domain	Concepts	Examples
personal	11	full_name, first_name, date_of_birth, email, father_name, github
financial	9	account_number, bank_name, annual_income, cif_number
skills	9	backend, frontend, databases, cloud_devops, ai_ml
education	8	degree, institution, field_of_study, cgpa, percentage, end_year
identity	7	aadhaar_number, pan_number, national_id, nationality, gender
address	6	street, city, state, country, zipcode, full_address
experience	6	company, role, start_date, end_date, description, technologies
project	6	name, description, role, technologies, architecture
medical	4	blood_group, allergies, insurance_provider, history_description
publication	4	title, conference, publisher, date
Others (8 domains)	21	achievement, business, certification, contact, emergency_contact, event, legal, shipping
Total	91	56 single-valued, 35 multiple-valued

Matching works on word tokens. Text is split at camel-case and non-alphanumeric boundaries, lower-cased and stripped of stop-words. An alias matches only if all its tokens appear contiguously, and a shorter alias covered by a longer one is discarded, so “father name” never matches the concept for “name”. The score is 0.98 when the alias covers the whole text and otherwise 0.80 + 0.15 × (alias tokens / text tokens).

4.4 DATA FLOW AND CONTEXT

Context diagram (Level 0)

Figure 4.4 is the context diagram. The system is a single process with one human actor, the user, working through the dashboard and the extension, and one optional external actor, the language-model provider. The user supplies credentials, documents, fact edits, conflict choices, questions and review decisions, and receives facts with provenance, conflicts, recommendations, grounded answers and processing status. The web form is shown as a data source and sink for the extension: it supplies field descriptors, never field values, and receives approved values.

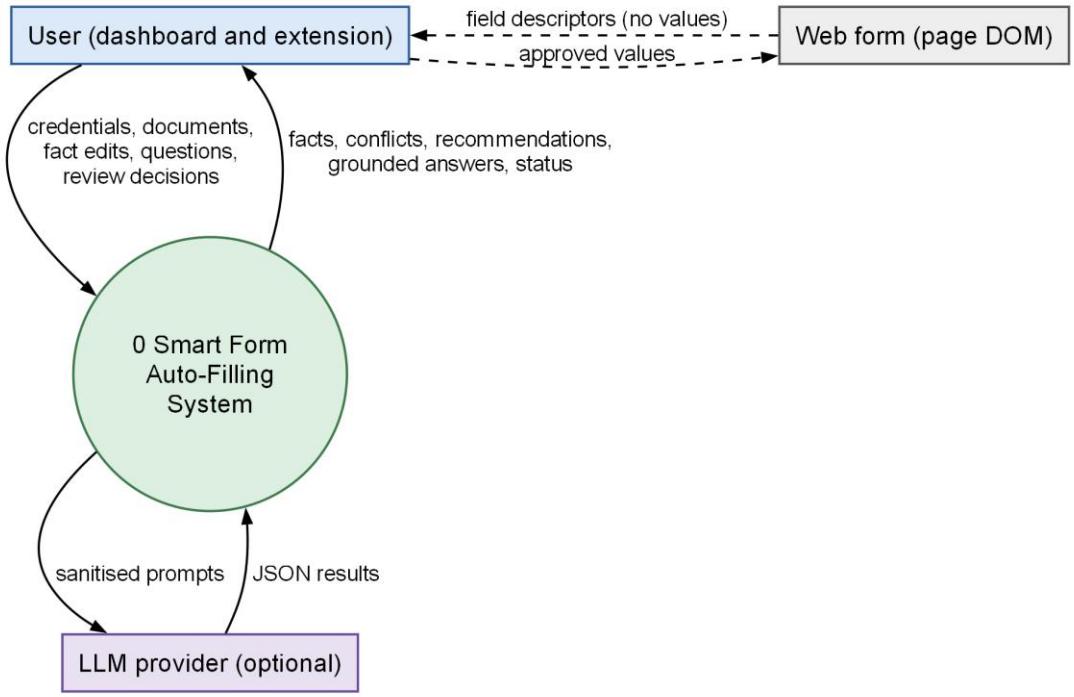


Figure 4.4: Context diagram (DFD Level 0)

Data flow diagram (Level 1)

Figure 4.5 decomposes the system into six processes and six data stores. Authentication (1.0) uses the users and token-revocation collections. Document management (2.0) writes document records and files and hands each stored file to fact extraction (3.0), which writes structured and discovered facts to D3 and chunk vectors to ChromaDB, calling the language model optionally. Profile and conflict resolution (4.0) reads and updates facts and records corrections. Field classification and recommendation (5.0) reads facts, retrieved evidence and saved memories and writes form sessions and recommendations. Question answering (6.0) reads chunks and calls the model.

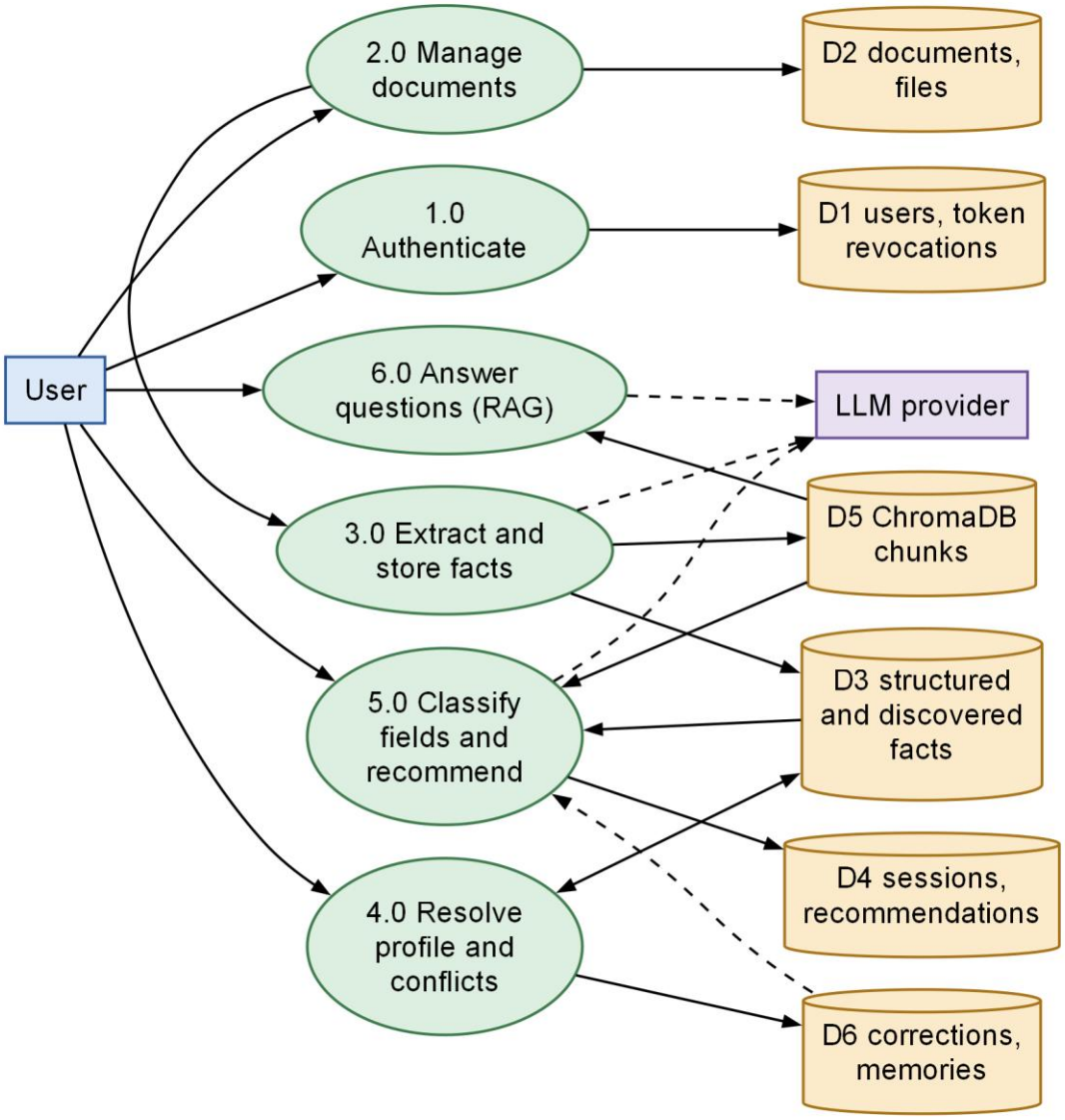


Figure 4.5: Data flow diagram, Level 1

Data flow diagram (Level 2)

Figure 4.6 expands process 3.0. A stored file is parsed (3.1); the deterministic extractor (3.2) and the optional LLM extractor (3.3) each produce candidate facts, using the ontology; the merge step (3.4) combines candidates with the same concept and value and keeps different values apart; the document's old facts are replaced by the merged set (3.5); and the text is chunked, embedded and upserted (3.6). A document that cannot be read, or that needs OCR when none is installed, leaves the pipeline with status failed or requires_review.

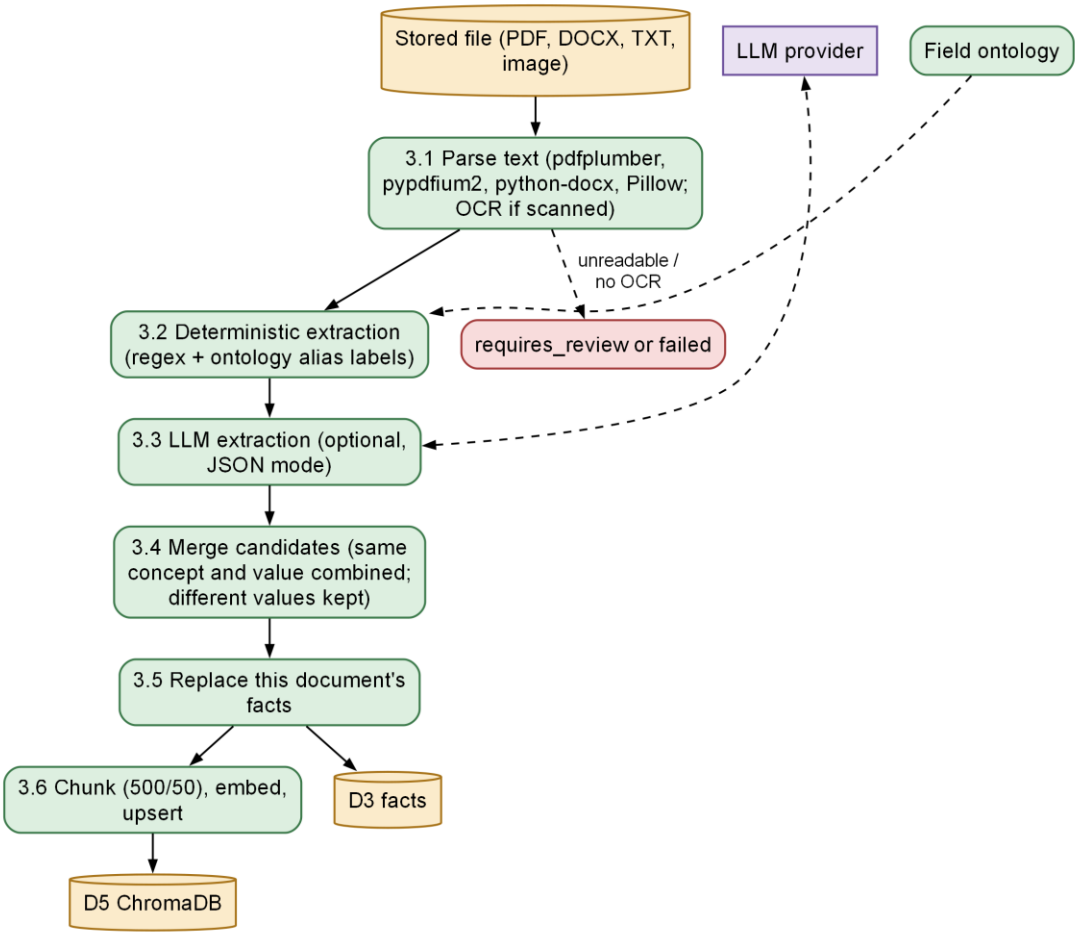


Figure 4.6: Data flow diagram, Level 2: Extract and Store Facts (process 3.0)

4.5 BEHAVIOURAL DESIGN

Use case diagram

Figure 4.7 summarises the functionality from the user's viewpoint. The system has a single authenticated role, so all use cases attach to one actor; there is no administrator. The language-model provider is shown as a secondary actor for the use cases that may call it: document upload (extraction), asking questions and reviewing recommendations. Scanning a page and filling approved values both include the review step, and resolving a conflict extends viewing and editing profile facts.

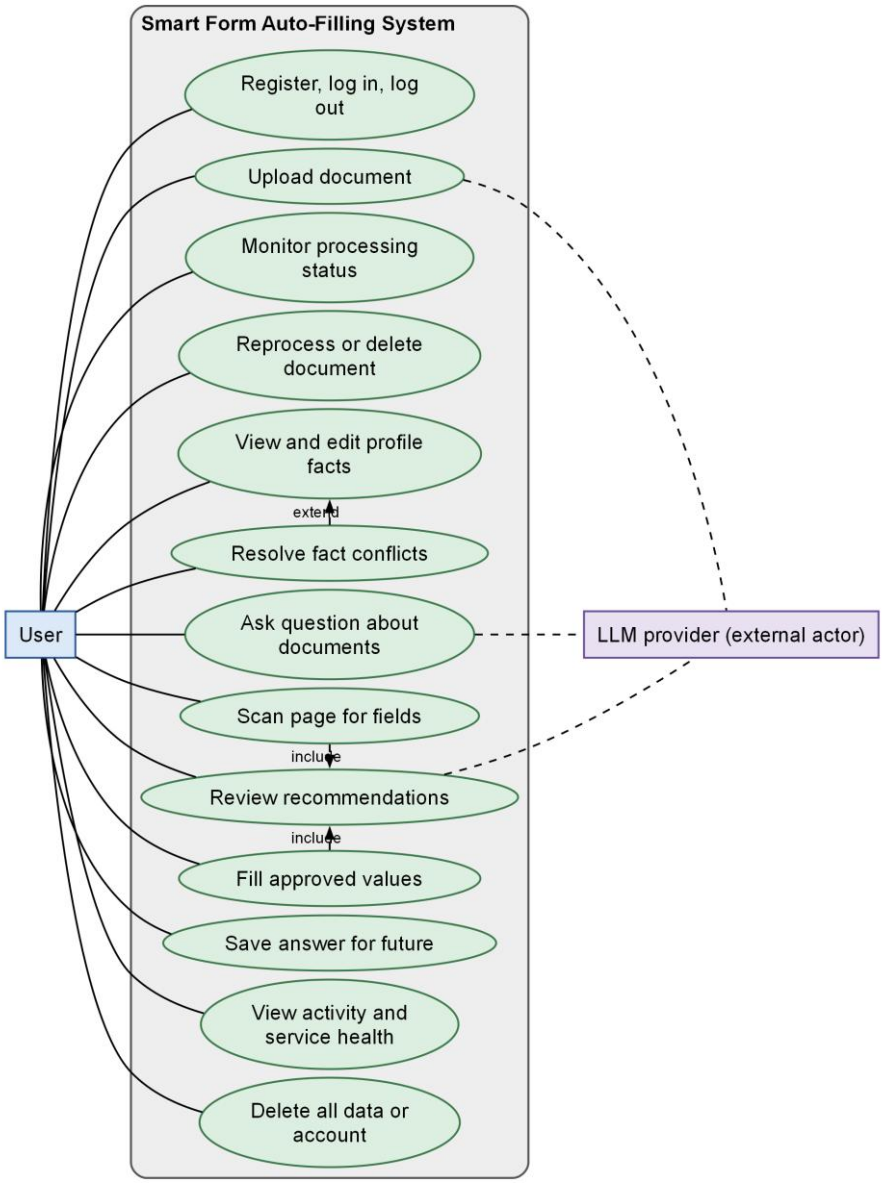


Figure 4.7: Use case diagram

Activity diagrams

Figure 4.8 models the upload workflow. Validation checks the extension, that the declared content type does not contradict the extension, that the file is not empty or larger than the limit, that its magic bytes match its type, and that a document with the same checksum has not already been uploaded by this user (a duplicate returns 409). A valid file is stored, a document record is created with status uploaded, and the job is dispatched. If no text can be read the document ends as failed or requires_review; otherwise facts are extracted and the text is indexed. A failure of indexing alone still yields a processed document whose index status is failed.

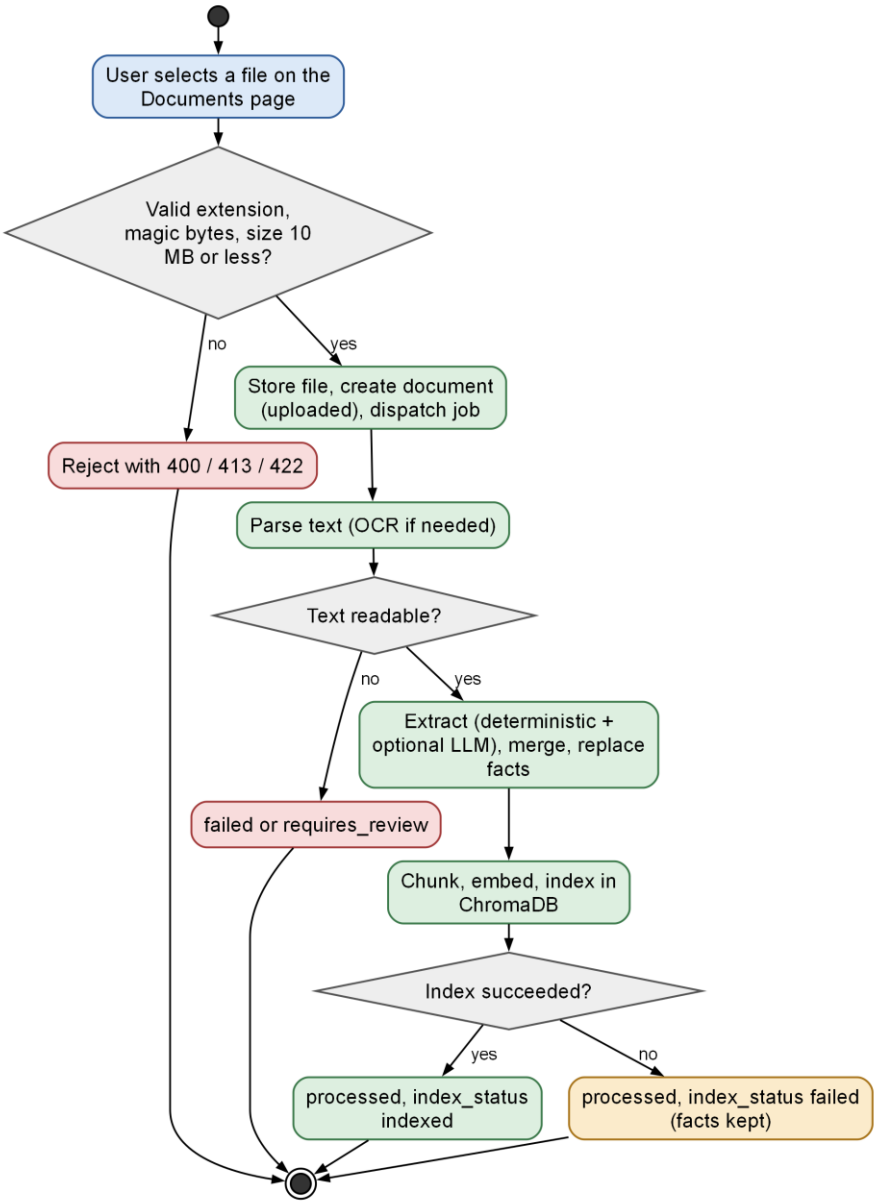


Figure 4.8: Activity diagram: document upload and processing

Figure 4.9 models the form-filling workflow. The popup injects the content script on demand, the detector lists eligible fields, a form session is created and analysis is started. The popup polls until recommendations are ready, shows each with a status, and the user ticks, edits or skips fields. Decisions are sent, approved values are fetched, the content script writes them and re-reads the page to verify them, and the results are posted back. The workflow ends without submitting the form.

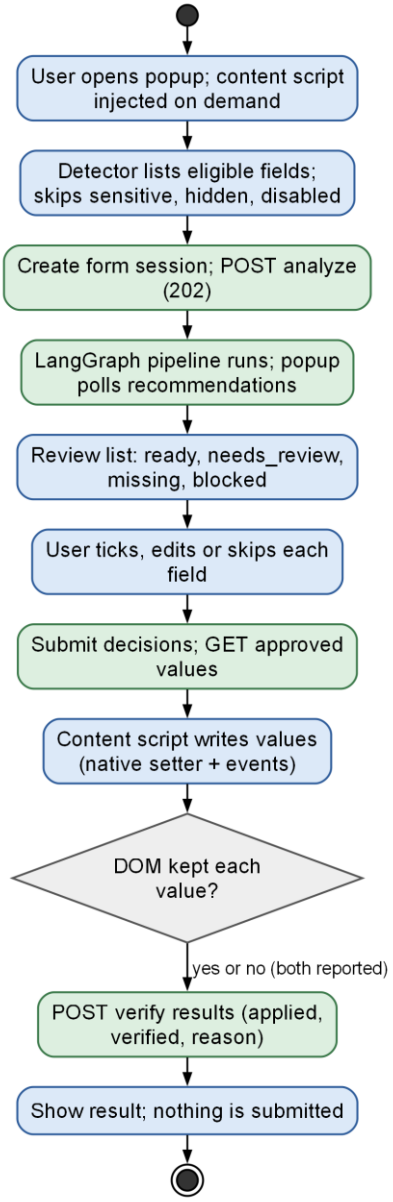


Figure 4.9: Activity diagram: filling a web form through the extension

Sequence diagrams

The sequence diagrams show the time-ordered messages between components for the key scenarios. Figure 4.10 and Figure 4.11 cover authentication. Registration rejects an existing e-mail with 409, hashes the password with bcrypt and stores the user. Login first applies a limiter of five attempts per sixty seconds per key, always performs one bcrypt comparison (against a dummy hash if the e-mail is unknown) so that response time does not reveal whether an account exists, and returns a JWT with subject, unique identifier, issue time, expiry and type. Later requests send the token as a bearer header; the service checks the revocation store and that the user is active.

Logout stores the token's identifier until its natural expiry, after which a time-to-live index removes it. The system issues no refresh token.

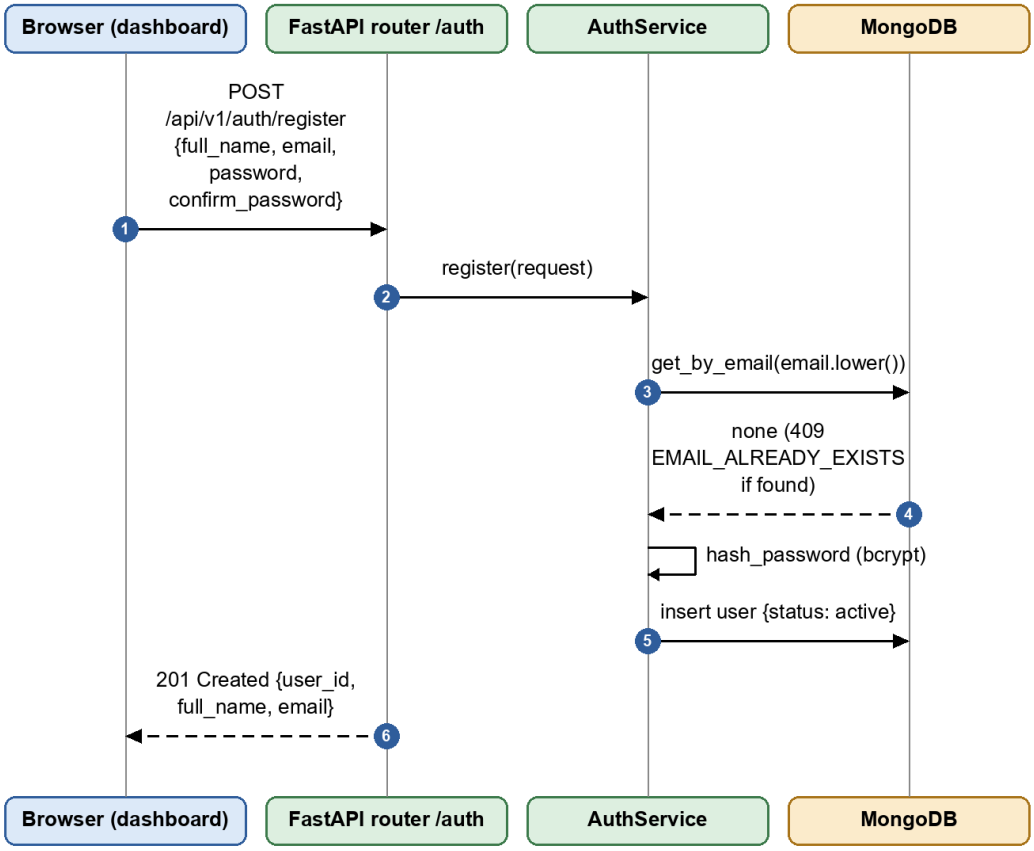


Figure 4.10: Sequence diagram: user registration

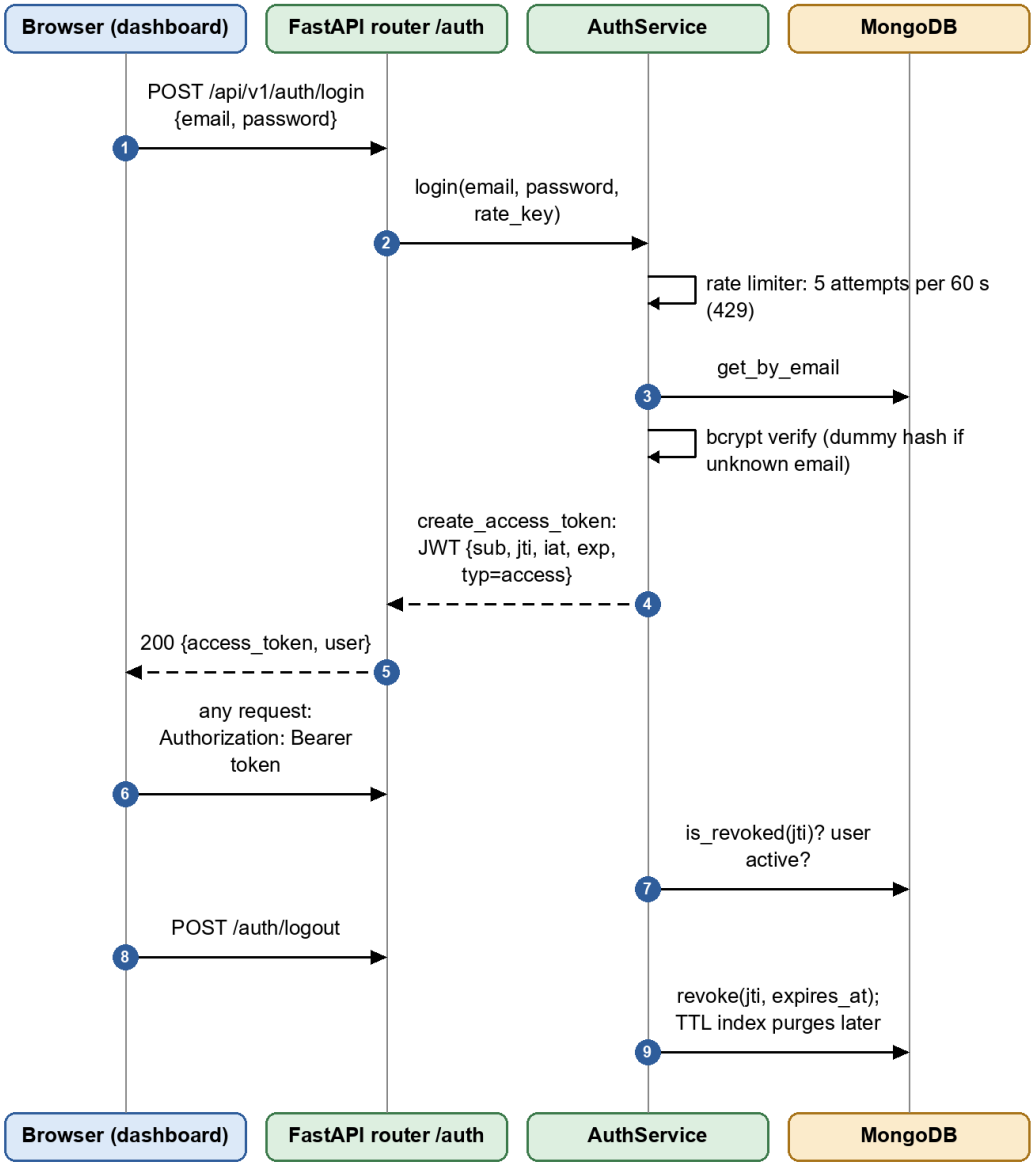


Figure 4.11: Sequence diagram: login, authenticated request and logout

Figure 4.12 shows the upload and processing exchange between the browser, the documents API, the pipeline runner, MongoDB and ChromaDB. The API answers 201 immediately; processing continues in the background (an inline task or a Celery job), and the browser polls the status endpoint for the current stage and progress.

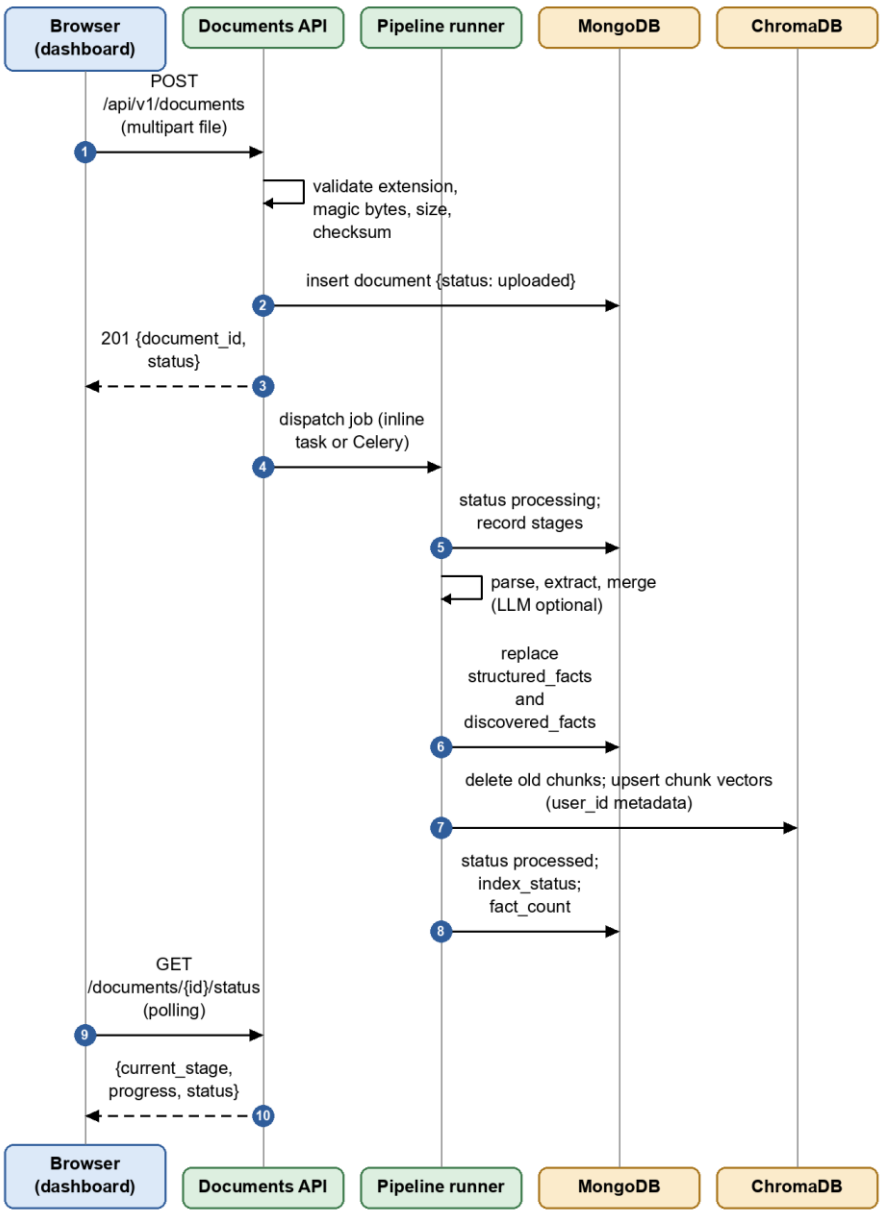


Figure 4.12: Sequence diagram: document upload, processing and status polling

Figure 4.13 shows form analysis, review and fill. The content script extracts field descriptors without reading values. The popup creates a session and calls analyze, which returns 202; the pipeline resolves the profile view from active facts, classifies, retrieves evidence for empty fields and stores recommendations. After the user's decisions are posted, approved values are returned in the form needed by the page (for example the selected option of a drop-down), the content script fills and verifies, and the verification results update the session's counters. The session is closed as not submitted unless the user reports otherwise.

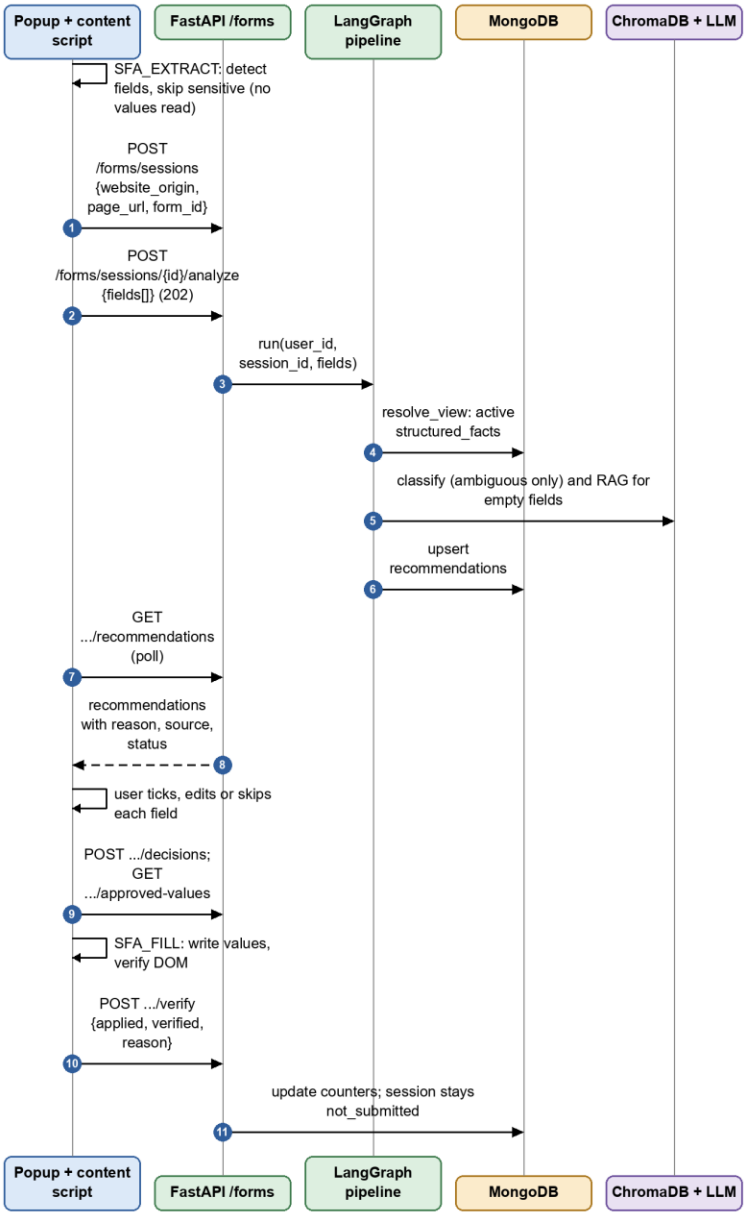


Figure 4.13: Sequence diagram: form analysis, review and fill

Figure 4.14 shows grounded question answering. The query is embedded and the vector store is asked for the nearest chunks of the current user only. Chunks below the similarity cutoff or containing injection-like text are dropped. The remaining chunks are sent to the model inside tagged untrusted blocks; the response must cite chunk identifiers, and only cited chunks are returned as sources. If no adequate evidence exists, the service returns insufficient_evidence instead of an answer.

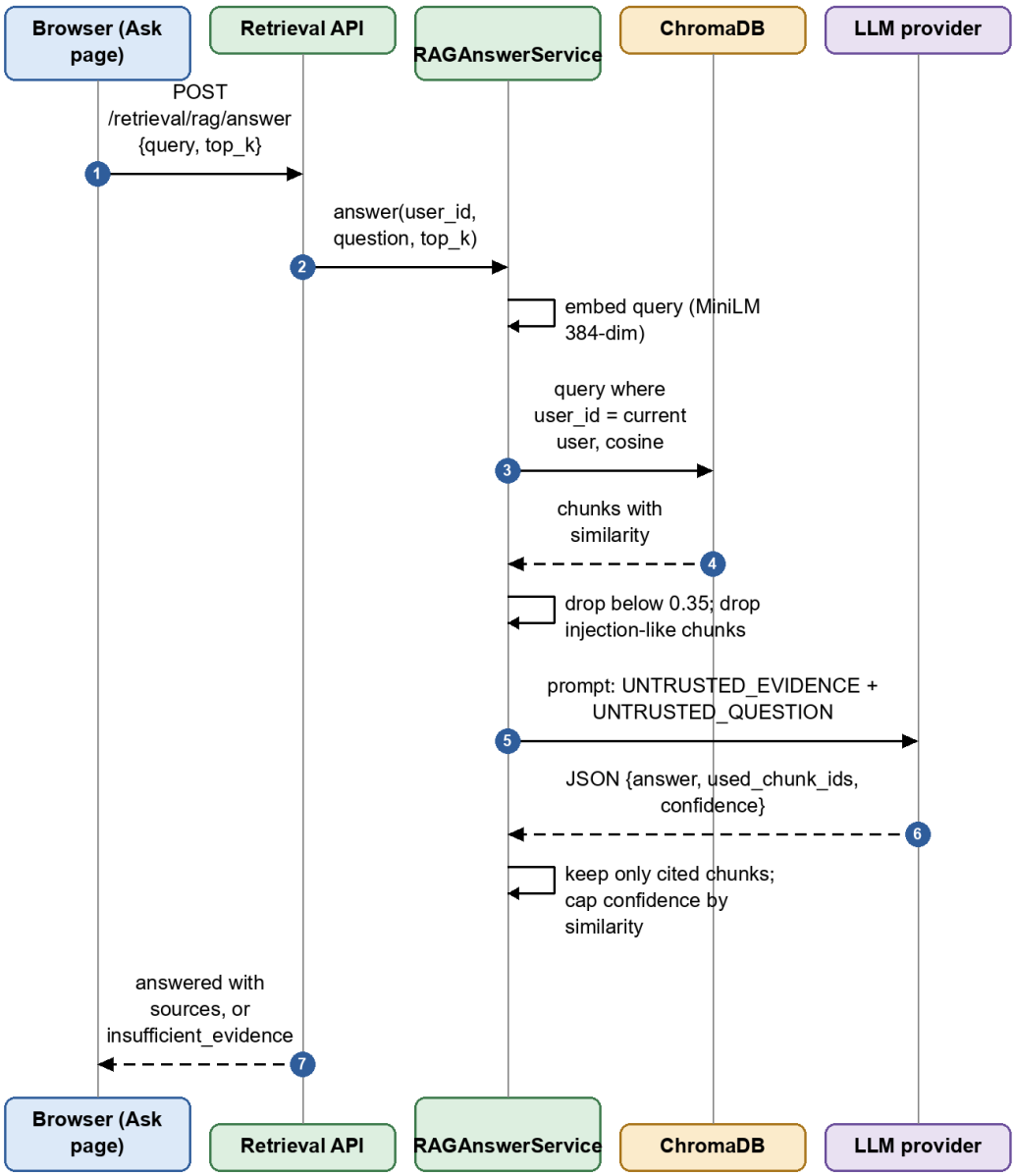


Figure 4.14: Sequence diagram: grounded document question answering

Figure 4.15 shows conflict handling. Conflicts are computed on request from the active facts: a single-valued concept with two or more distinct comparison keys is a conflict, classified as cross-document, intra-document or verified-versus-new. Resolving with a candidate confirms the chosen facts and supersedes the others; a custom value becomes a user-provided fact; rejecting marks all candidates rejected. A correction record is kept for the audit trail.

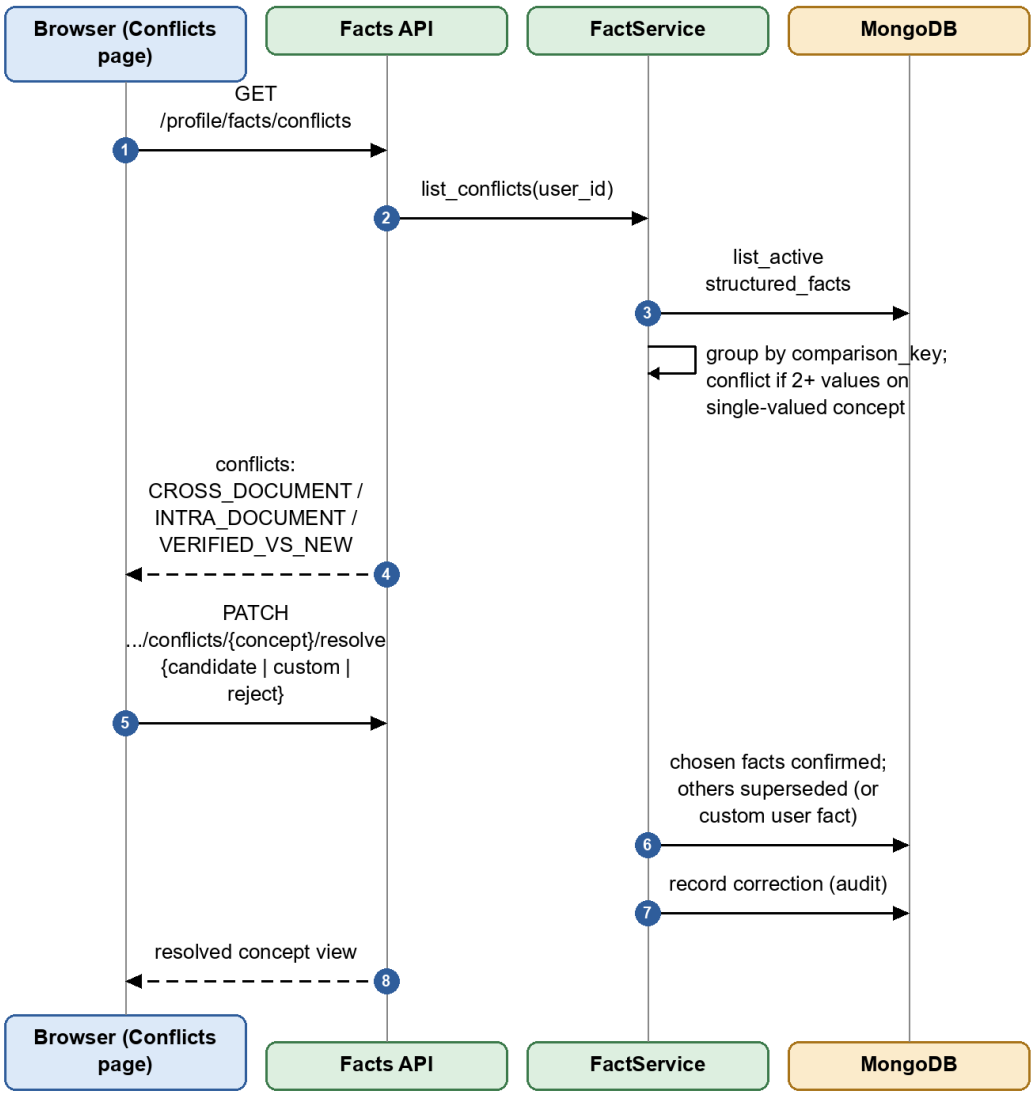


Figure 4.15: Sequence diagram: conflict detection and resolution

4.6 DATABASE DESIGN

MongoDB collections

MongoDB is the source of truth. Figure 4.16 shows the entity relationships. Almost every record carries user_id, and repository queries and deletes include it. A user owns documents, structured facts, discovered facts, form sessions, corrections and saved memories. A document owns its processing records and its facts. A form session owns recommendations, one per field, enforced by a unique index on (user_id, session_id, field_id). The ChromaDB chunk index is a parallel structure keyed by user_id and document_id and is not a MongoDB collection. Table 4.4 lists each collection, its purpose and its indexes.

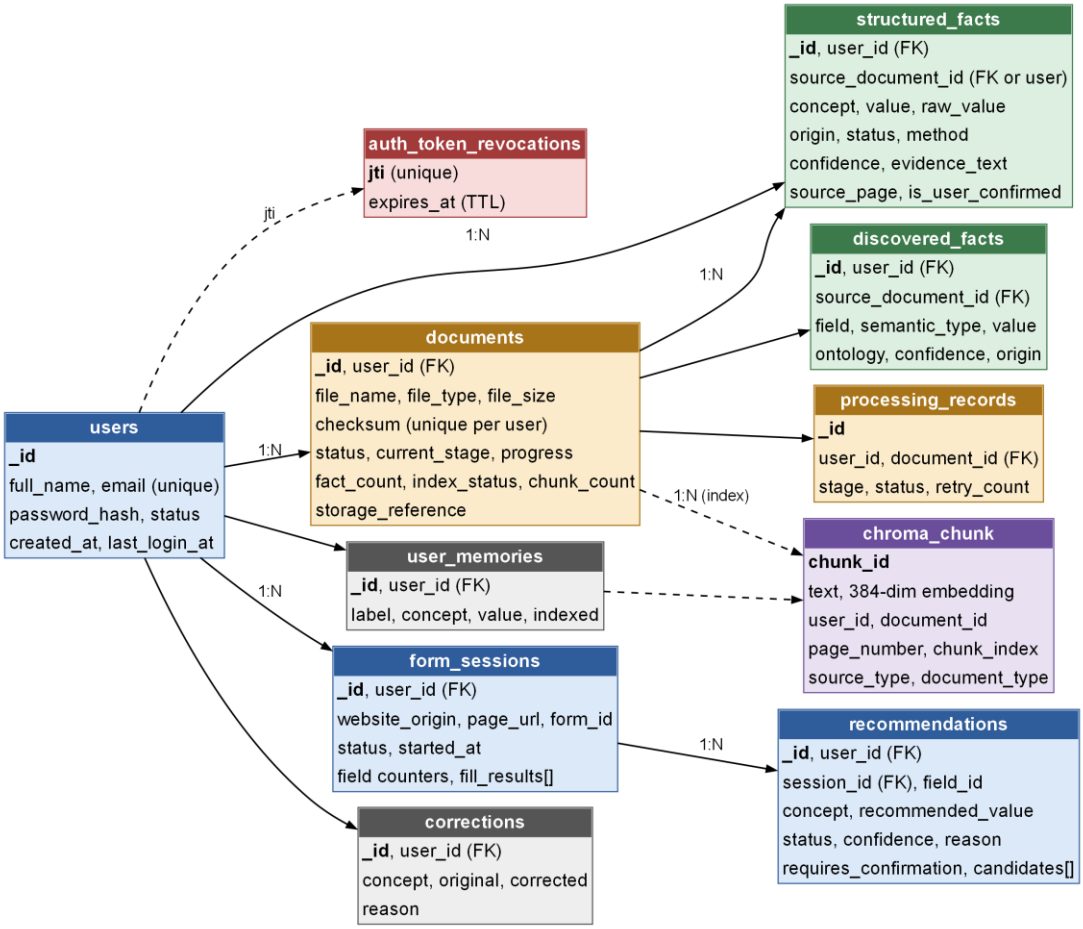


Figure 4.16: Entity relationship diagram of the MongoDB collections and the ChromaDB chunk index

Table 4.4: MongoDB collections and their indexes

Collection	Purpose and principal fields	Indexes
users	full_name, email, password_hash, status, timestamps	unique email
documents	file_name, file_type, file_size, storage_reference, checksum, status, current_stage, progress, page_count, fact_count, index_status, chunk_count	user_id; (user_id, status); unique (user_id, checksum)
structured_facts	One sourced value: concept, value, raw_value, origin, status, source_document_id, source_page, evidence_text, extraction_method, confidence, is_user_confirmed	(user_id, concept); (user_id, status, concept); (user_id, source_document_id)
discovered_facts	All extractor output, including unmapped values: field, semantic_type, value, ontology, confidence, evidence_text	(user_id, source_document_id); (user_id, semantic_type)
recommendations	One result per field: concept, recommended_value, status, confidence, reason, source, requires_confirmation, candidates	unique (user_id, session_id, field_id); (user_id, status)
form_sessions	website_origin, page_url, form_id, status, field counters, fill_results	(user_id, started_at desc)
corrections	Concept, original and corrected value, reason	(user_id, concept)

Collection	Purpose and principal fields	Indexes
processing_records	Stage, status, retry count, safe error message per document	(user_id, document_id)
user_memories	Explicitly saved answers: label, concept, value, indexed	(user_id, updated_at desc)
auth_token_revocations	Revoked token identifier and expiry	unique jti; TTL on expires_at

Conflict lists are not stored. They are computed from active structured facts (architecture decision AD-013), so a deleted or reprocessed document cannot leave a stale conflict. Legacy collection names (profiles, discovered_conflicts, feedback, agent_runs) are included in deletion cleanup for compatibility with older data only.

ChromaDB index

The application writes to the collection user_document_chunks_v2 through ChromaDB's HTTP v2 API. The collection uses cosine distance and records the embedding model and dimension in its metadata; a mismatch between the configured model and the collection is reported rather than tolerated. Each chunk stores user_id, document_id, document_name, document_type, page_number, language, chunk_index and source_type (document or user_memory). Every query and delete filters by user_id. If ChromaDB is unavailable there is no in-memory fallback: errors surface as 503 (architecture decision AD-005). A reindex command rebuilds a collection from the stored source documents without touching other collections.

Status vocabularies

Table 4.5 collects the status values used by the system. Figure 4.17 shows the lifecycle of a document, and Figure 4.18 shows the lifecycle of a fact. A document ends as processed, failed (unreadable or internal error) or requires_review (OCR needed but not installed); reprocessing returns it to processing. A fact is active until another candidate is confirmed, the value is edited or a custom value is chosen (superseded), or the user rejects it (rejected).

Table 4.5: Status vocabularies used by the system

Object	Values
Document status	uploaded, processing, processed, failed, requires_review, deleted
Document index status	pending, indexed, failed, empty
Fact status	active, superseded, rejected
Fact origin	extracted, normalized, inferred, derived, user_provided

Object	Values
Recommendation status	ready, needs_review, missing_information, security_blocked, skipped, error, approved, edited, rejected
Concept view status	resolved, conflict, multiple
Conflict type	CROSS_DOCUMENT, INTRA_DOCUMENT, VERIFIED_VS_NEW
Form decision	approve, edit, reject, skip
Submission status	user_submitted, not_submitted, cancelled

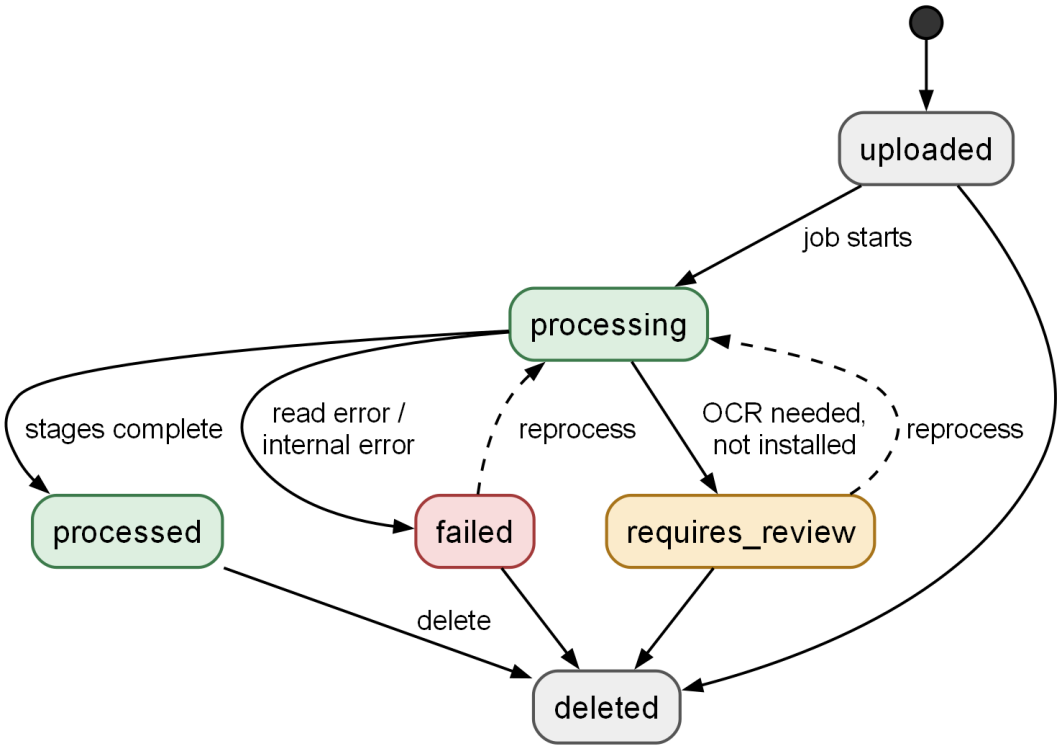


Figure 4.17: State diagram of a document's processing status

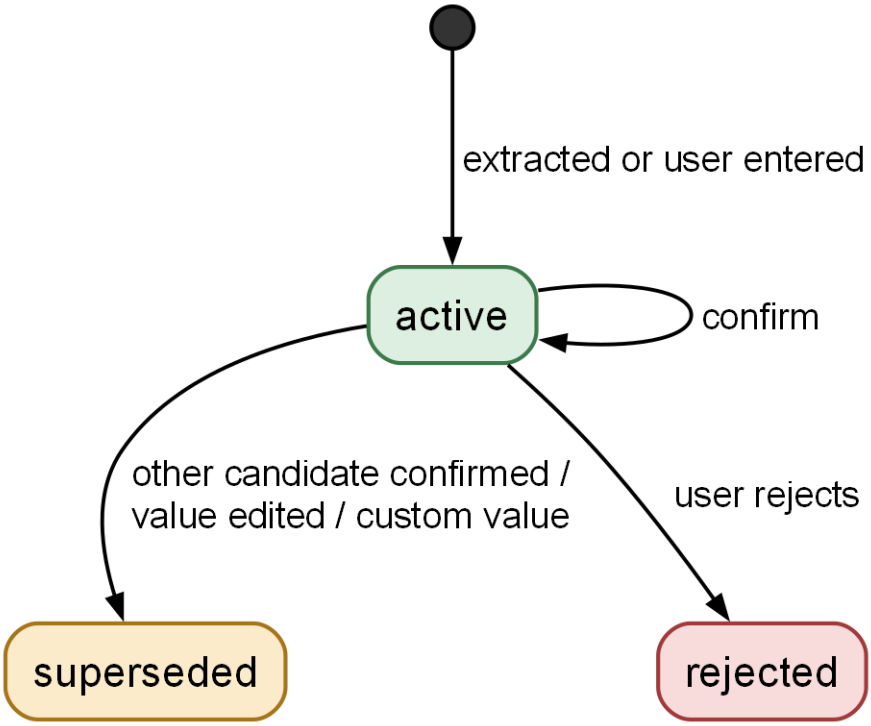


Figure 4.18: State diagram of a structured fact's status

4.7 STRUCTURAL DESIGN: CLASS DIAGRAMS

Figure 4.19 shows the classes behind form filling. FormFillGraph owns the seven node methods and depends on FieldClassifier, FactService and the retrieval service. FieldClassifier and FactService both depend on FieldOntology. FactService builds ConceptView objects from StructuredFact records; a ConceptView is resolved when all active facts agree, conflict when a single-valued concept has distinct values, and multiple when a multi-valued concept has several.

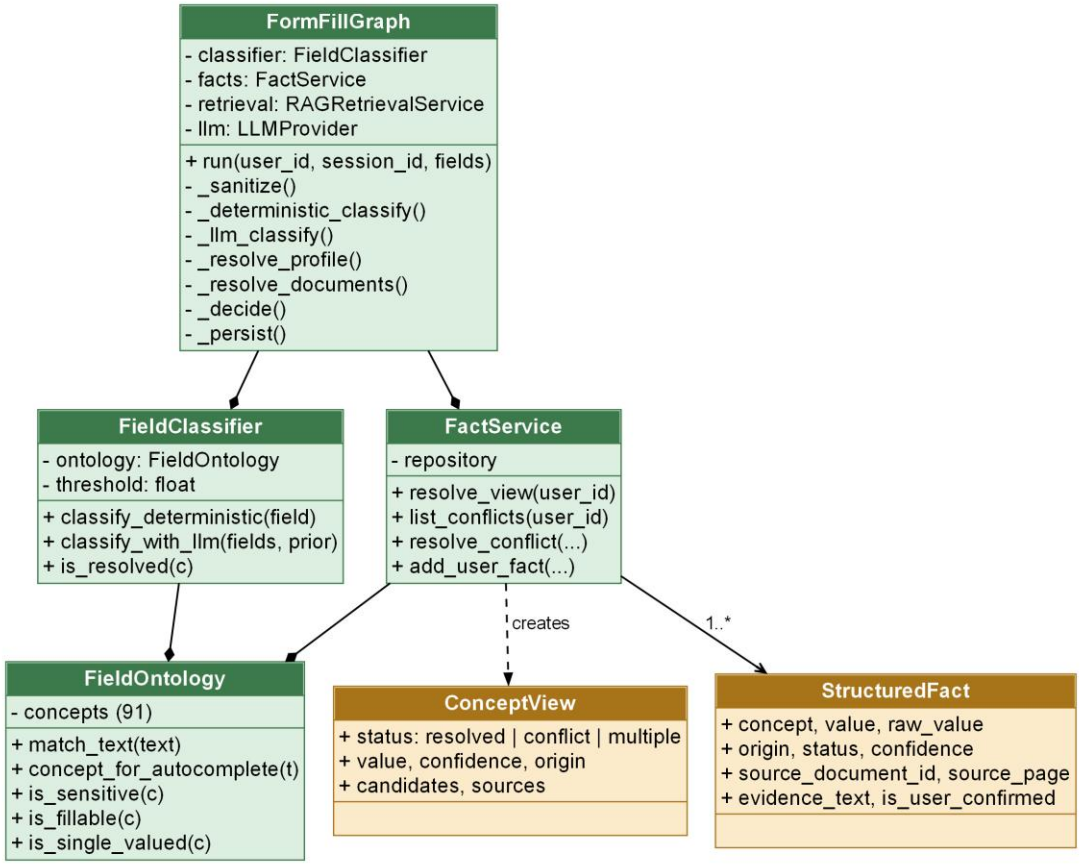


Figure 4.19: Class diagram: form-fill agent, classifier, fact service and ontology

Figure 4.20 shows the language-model and retrieval classes. LLMProvider is an abstract class with one asynchronous method; the Google, OpenAI, Anthropic and mock providers extend it. The HTTP providers share retry logic: a 429 or 5xx response or a timeout is retried with exponential back-off for non-interactive tasks, while the two interactive tasks (field classification and field values) use a 20-second timeout and no retry. The provider never returns a sentinel string and never silently switches provider or model. The mock provider is deterministic, makes no network call and answers every task with an empty, schema-valid result unless a test supplies a responder.

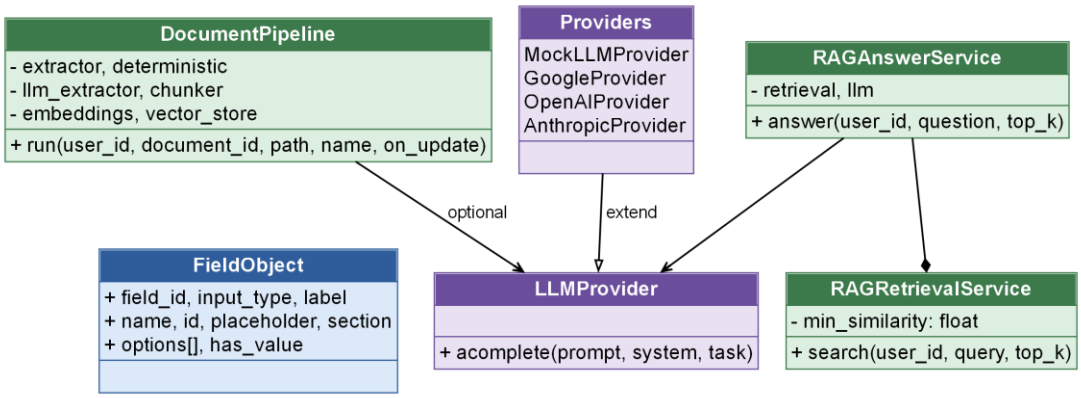


Figure 4.20: Class diagram: LLM providers, document pipeline and retrieval services

4.8 CORE ALGORITHMS

Fact extraction and merging

The deterministic extractor favours precision over recall. It takes a value only when it has a distinctive format (an e-mail address, a LinkedIn or GitHub profile, a phone number next to a phone-like label) or when it follows a label whose ontology match scores 0.98, which means the whole label is an alias of exactly one concept. Typed values that do not parse (for example a date) are discarded. Labelled values receive confidence 0.92, format matches 0.95 to 0.97, and the single heuristic, a capitalised name line near the top of the first page, is marked inferred with confidence 0.6.

The optional LLM extractor splits the document text into windows of up to 18,000 characters (at most four windows), wraps each in an UNTRUSTED_DOCUMENT block and asks for JSON with the concept, value, page, exact evidence text and confidence. Each returned value is accepted as extracted or normalized only if it literally appears in the document and so does its evidence; otherwise it is marked inferred and its confidence is capped at 0.5. If the provider fails, a warning is recorded and the deterministic facts are still saved.

merge_candidates combines candidates with the same concept (or semantic type) and the same comparison key. When both extractors agree, confidence rises by 0.03 up to a cap of 0.99, and the better origin (extracted before normalized before derived before inferred) is kept. Candidates with different values are never merged; they remain separate facts and may later form a conflict.

Algorithm 4.1: Merging candidate facts

```
merge(candidates):
```

```
merged = {}
for c in candidates:
  key = (c.concept or c.semantic_type,
         comparison_key(c.type, c.value))
  if key.value is empty: continue
  if key not in merged: merged[key] = c
  else:
    methods = union(merged[key].methods, c.methods)
    agree = len(methods) > 1
    best = lower_rank_origin(merged[key], c)
    merged[key] = best
    merged[key].confidence = min(0.99, max(conf) + (0.03 if agree else 0))
return values(merged)
```

The comparison key makes two spellings of the same fact equal: phone numbers by their last ten digits, dates by their ISO form, and other values by lower-cased, punctuation-stripped words. The cost of merging is linear in the number of candidates.

Form-fill pipeline

Figure 4.21 shows the seven-node LangGraph state graph, defined in agents/graph.py. State carries the user, the session, the unique fields, the per-field work items, the classifications and the profile views. Two conditional edges avoid unnecessary work: llm_classify runs only if some field is unresolved, and resolve_documents runs only if some non-sensitive field still has no value.

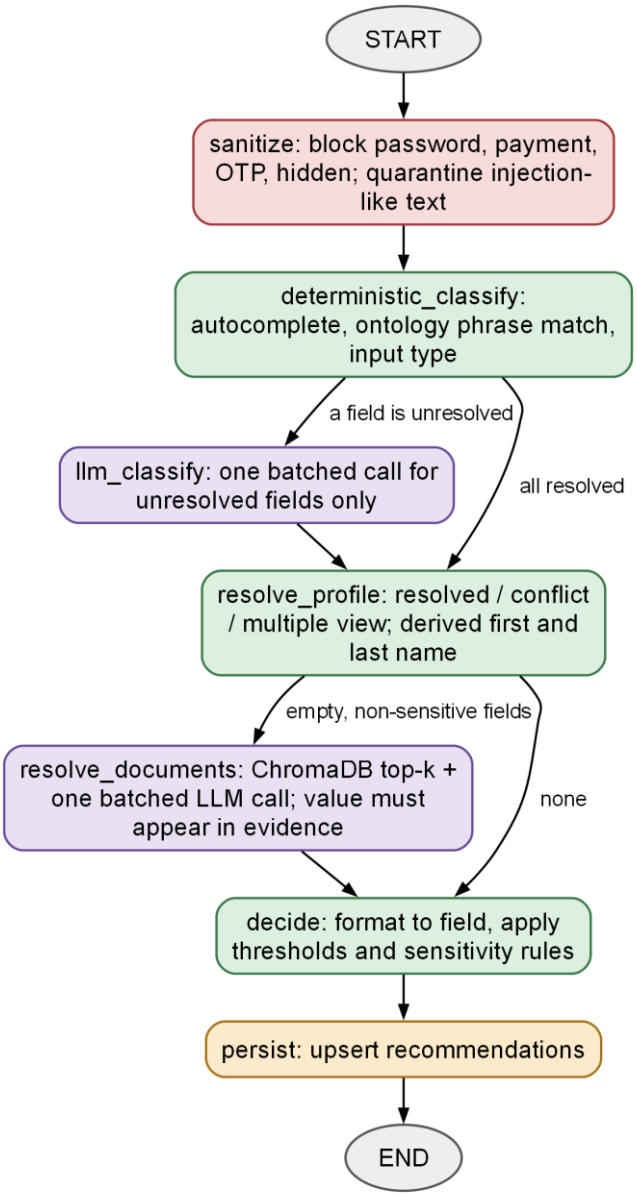


Figure 4.21: LangGraph state graph of the form-fill pipeline

- **sanitize:** Marks fields security_blocked when the input type is password, hidden, file, submit, button, reset or image, when the autocomplete token is a payment, one-time-code or password token, or when the label, name or placeholder mentions OTP, CVV, CAPTCHA, card number, password, security code and similar terms. Fields whose text matches an injection pattern are quarantined and kept away from the LLM.
- **deterministic_classify:** Applies the algorithm of Figure 4.22 to each remaining field.

- **llm_classify:** Sends one batched request with a sanitised description of each unresolved field and the catalogue of fillable concepts. A returned concept is accepted only if it is a known ontology concept, its confidence is at least 0.6, it is among the tied candidates if the field was ambiguous, and it is not the user's own value for a field about another person.
- **resolve_profile:** Builds the resolved view of every concept from active facts. A resolved concept supplies value, confidence (1.0 if user confirmed), origin and source; a conflict or multiple-valued concept becomes needs_review with candidates; a first, middle or last name is derived from a resolved full name, with lower confidence and a confirmation requirement if the name has other than two parts. Non-fillable concepts such as consent and signature are skipped with the reason that the user must give them.
- **resolve_documents:** For fields still empty and not sensitive, retrieves the top-k chunks for the field text and concept description and sends one batched request. The model must return the chunk identifier and, for open questions, a quotation. A factual value must literally appear in the cited chunk and its confidence is capped at 0.8 (or the chunk similarity plus 0.3, if smaller); prose answers need a quotation that appears in the chunk and are capped at 0.7. Anything ungrounded is discarded.
- **decide:** Applies the algorithm of Figure 4.23.
- **persist:** Upserts one recommendation per field for the session.

Deterministic field classification

Figure 4.22 shows the classification rules. The HTML autocomplete token is the strongest signal (confidence 0.97). Otherwise the field's texts are matched against ontology aliases in a fixed priority order with weights: question and label 1.0, aria-label 1.0, placeholder 0.95, name and id 0.9, nearby text 0.85. If the two best matches differ by 0.05 or less the field is ambiguous; the name or id attribute can break the tie (confidence 0.8). If still unclassified, an input of type email or tel falls back to the e-mail or phone concept. A match on one of seven owner concepts (own name, e-mail, phone, date of birth) inside a section whose text mentions a father, mother, guardian, spouse, emergency contact, referee, nominee, sibling or child is withdrawn, so that the user's own value is not used for another person. A classification is resolved when its concept is known, it is not ambiguous and its confidence reaches 0.75.

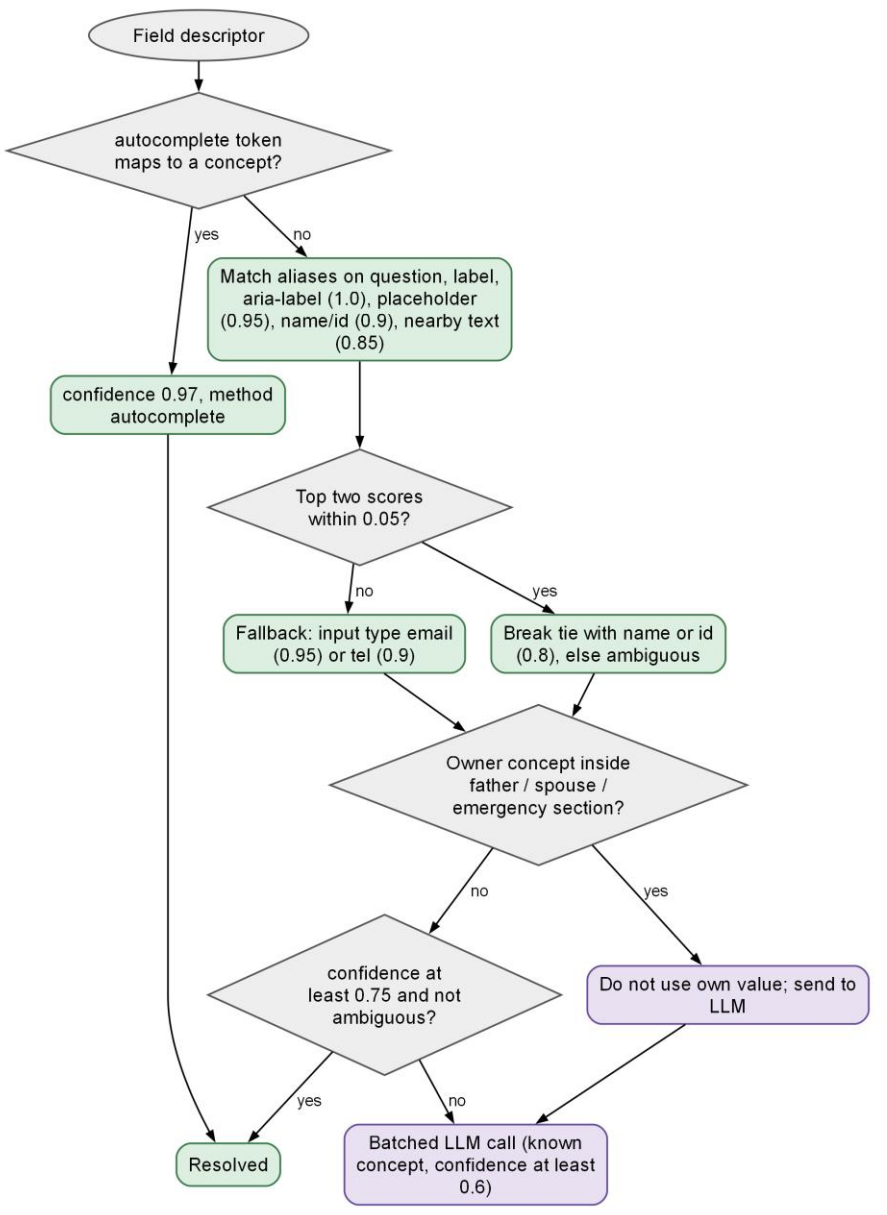


Figure 4.22: Flowchart of deterministic field classification

Decision rules

Figure 4.23 shows how the final status is chosen. Fields already blocked, skipped, in error or in review keep their status. A field without a value becomes missing_information. Otherwise the value is formatted for the field (a date is rewritten to the pattern the field expects, a drop-down or radio group is matched to one of its options); if formatting fails, the value is shown as a candidate with the reason. A value for a sensitive concept (the ontology flag or the configured list of eight identity and finance concepts) or of origin inferred always requires confirmation. A field is ready only if its confidence is at least 0.85 and no confirmation is required; every

other field is needs_review. Nothing is filled from conflicting, inferred or sensitive data without the user's explicit approval.

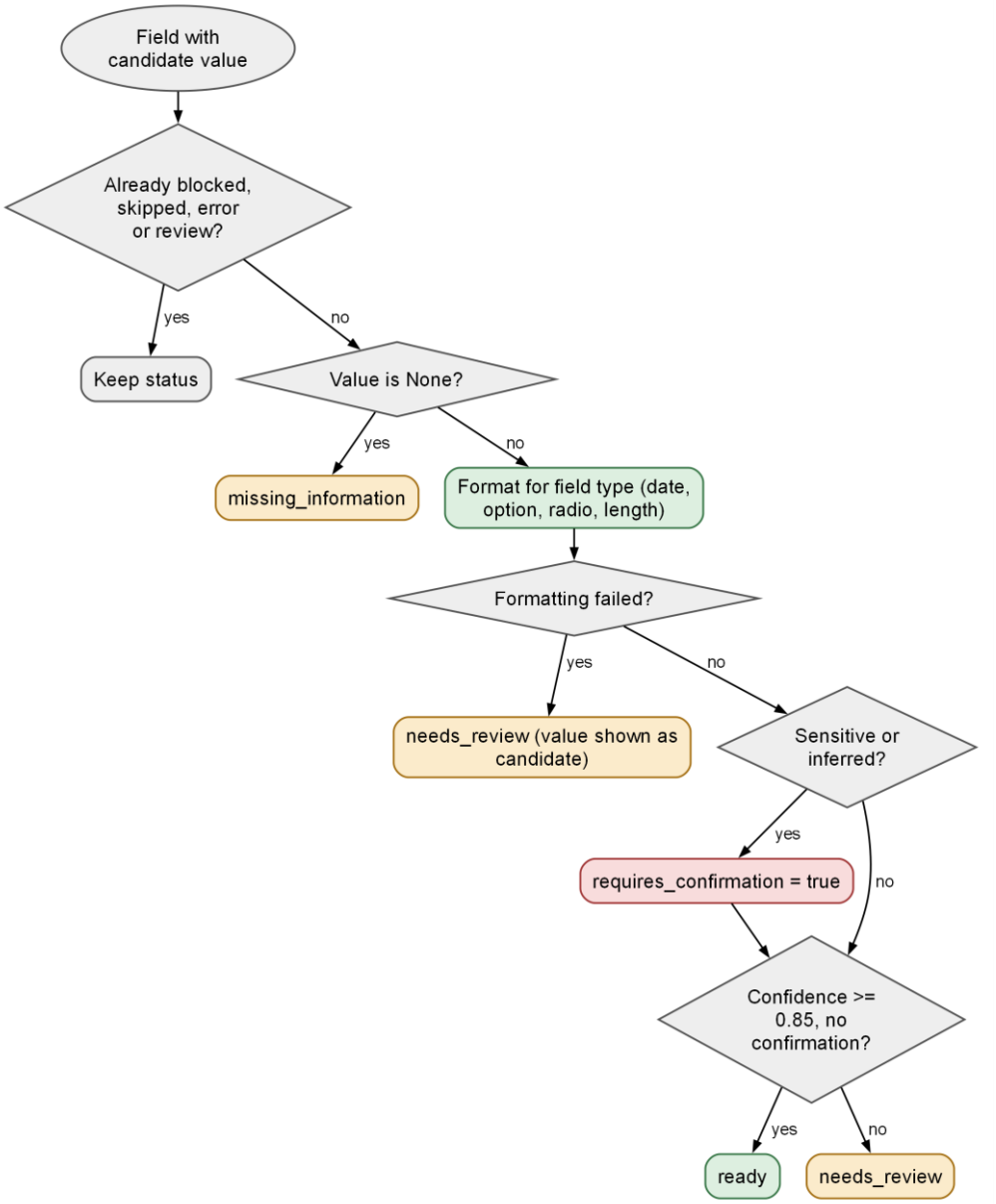


Figure 4.23: Flowchart of the decide node

Algorithm 4.2: Final decision for one field

```
decide(item):
  if item.status in {blocked, skipped, error, needs_review}: return
  if item.value is None: item.status = missing_information; return
  formatted = format_for_field(field, data_type(concept), item.value)
  if formatted.failed: item.status = needs_review; item.candidates = [value]; return
  if sensitive(concept) or item.origin == 'inferred':
    item.requires_confirmation = True
  ok = item.confidence >= 0.85 and not item.requires_confirmation
  item.status = 'ready' if ok else 'needs_review'
```

Conflict resolution

For each concept, active facts are grouped by comparison key. One group gives a resolved view. For a multiple-valued concept several groups give a multiple view, which is shown but is not a conflict. For a single-valued concept several groups give a conflict. Candidates are ordered with user-confirmed values first, then by confidence. Resolving with a candidate confirms its facts and supersedes the rest, so the choice persists and the same conflict does not reappear. The work is linear in the number of active facts of the user.

Retrieval-augmented question answering

Figure 4.24 summarises the RAG path used both by the Ask page and by resolve_documents. A query is embedded with the sentence-transformer model; ChromaDB returns the nearest chunks of the current user by cosine similarity; chunks below 0.35 or containing injection-like text are dropped; the rest are labelled and sent to the model. Text is chunked per page into windows of 500 characters with an overlap of 50, so a chunk never crosses a page boundary and a citation always names the correct page.

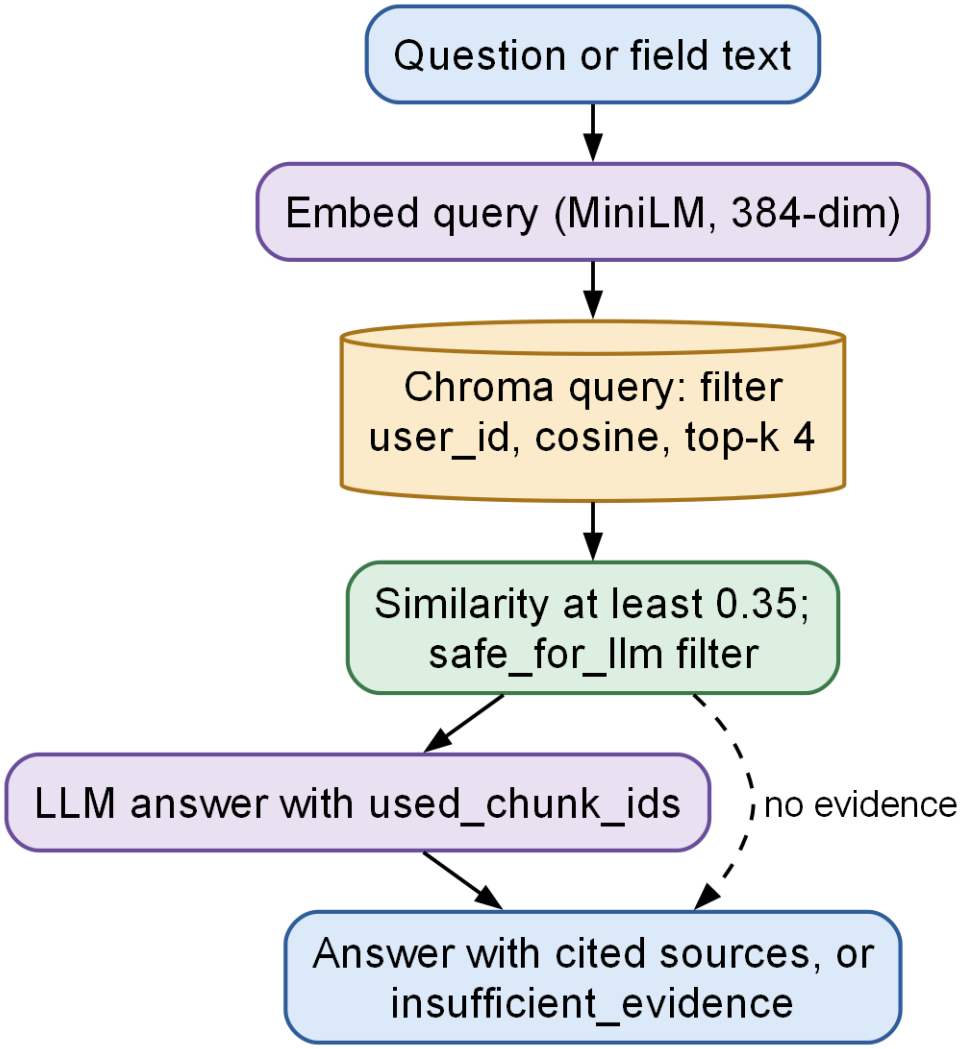


Figure 4.24: Retrieval-augmented question answering pipeline

4.9 SECURITY DESIGN

Security is designed in layers (Figure 4.25). Table 4.6 maps each control to the module that implements it. The threat model has four principal threats: a malicious web page that tries to read personal data or to push instructions to the model; a malicious or malformed document; another authenticated user; and leakage through logs or error messages.

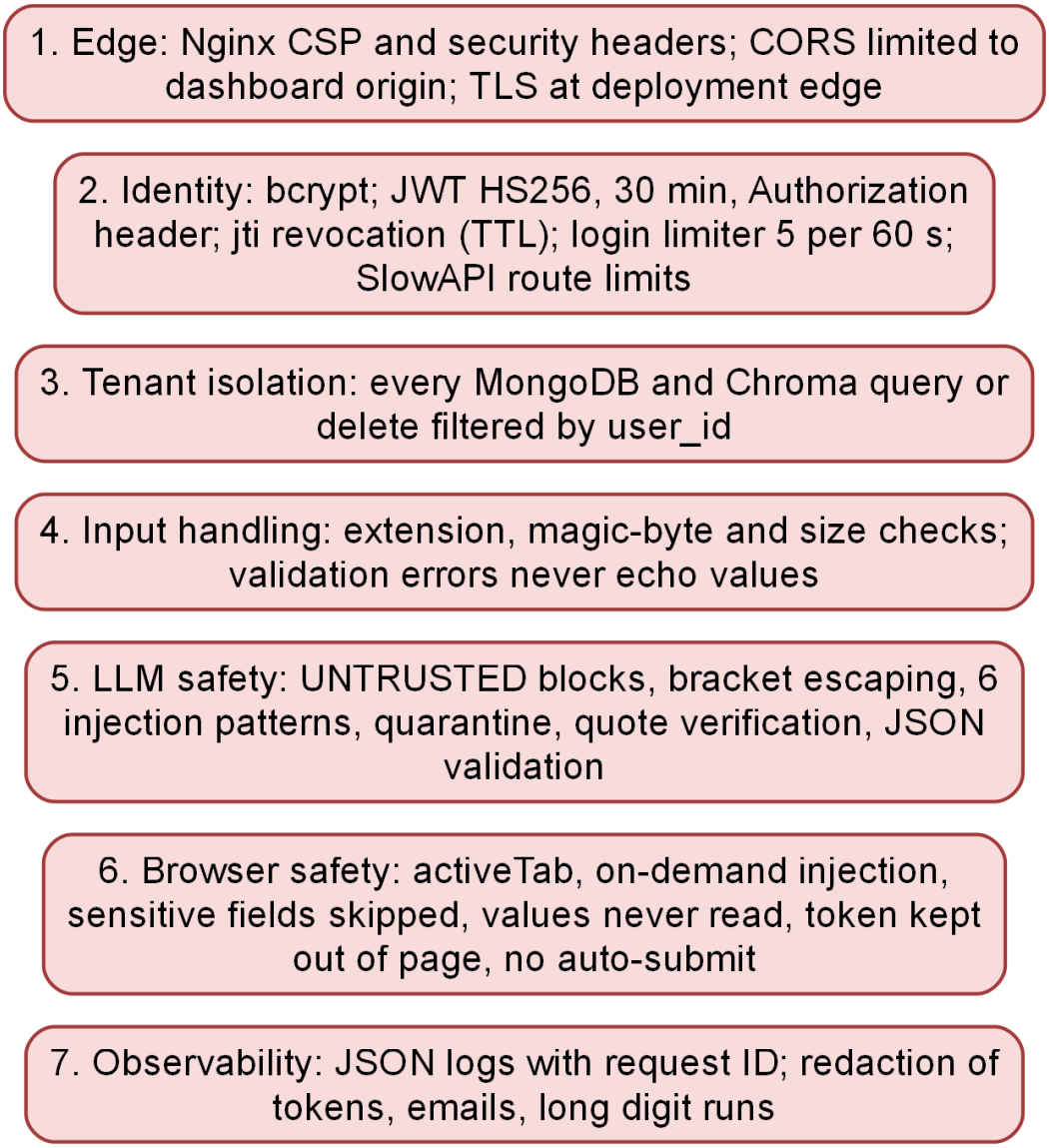


Figure 4.25: Defence-in-depth layers of the implementation

Table 4.6: Security controls and where they are implemented

Control	Threat addressed	Implementation
bcrypt password hashing; 72-byte limit enforced	Credential theft	core/security.py; services/auth.py
JWT HS256, 30 minutes, jti revocation with TTL	Token reuse after logout	core/security.py; db/revocations.py
Login limiter and SlowAPI route limits	Brute force, abuse	services/auth.py; core/rate_limit.py; api/v1/*
user_id filter on every MongoDB and Chroma operation	Cross-user access	repositories; db/chroma_client.py
Extension, content-type and magic-byte checks; size limit; checksum	Malicious or mismatched upload	services/documents.py
UNTRUSTED blocks, bracket escaping, six injection patterns, quarantine	Prompt injection via page or document	security/prompt_safety.py

Control	Threat addressed	Implementation
Quote and literal-presence checks; JSON schema validation	Hallucinated values	agents/graph.py; agents/json_output.py
Sensitive and hidden fields skipped (extension) and blocked (backend)	Autofill abuse [16]	content/detector.ts; agents/graph.py
activeTab and scripting on demand; host permission limited to backend	Overbroad extension access	public/manifest.json
Token held in popup and background only	Token theft by page scripts	extension/src/shared/api.ts
Redacting JSON logs; validation errors omit values; generic 500 message	Leakage through logs and errors	core/logging.py; core/exceptions.py
Strict secret validation at start-up	Weak or placeholder secrets	core/config.py
Cache-Control no-store and nosniff on previews and downloads	Cached or sniffed documents	api/v1/documents.py

Figure 4.26 shows how every untrusted string is handled before a model sees it. The helper detect_injection tests six regular expressions covering phrases such as “ignore previous instructions”, requests to reveal stored or system data, “you are now a…”, “new instructions:”, references to a system or developer prompt, and attempts to close the delimiter tags. A match quarantines a field or drops a chunk. Clean text has its whitespace collapsed, its length capped and its angle brackets replaced, so it cannot close the block, and is wrapped in a tagged block that the system prompt declares to be data. Model output is parsed against a schema, and for values from documents it must be traceable to a cited chunk. This follows the recommendations of the OWASP guidance [26] and addresses the indirect-injection risk described by Greshake et al. [23]. It reduces, but does not eliminate, the risk: pattern matching cannot recognise every phrasing, which is why the grounding checks and mandatory user review are also required.

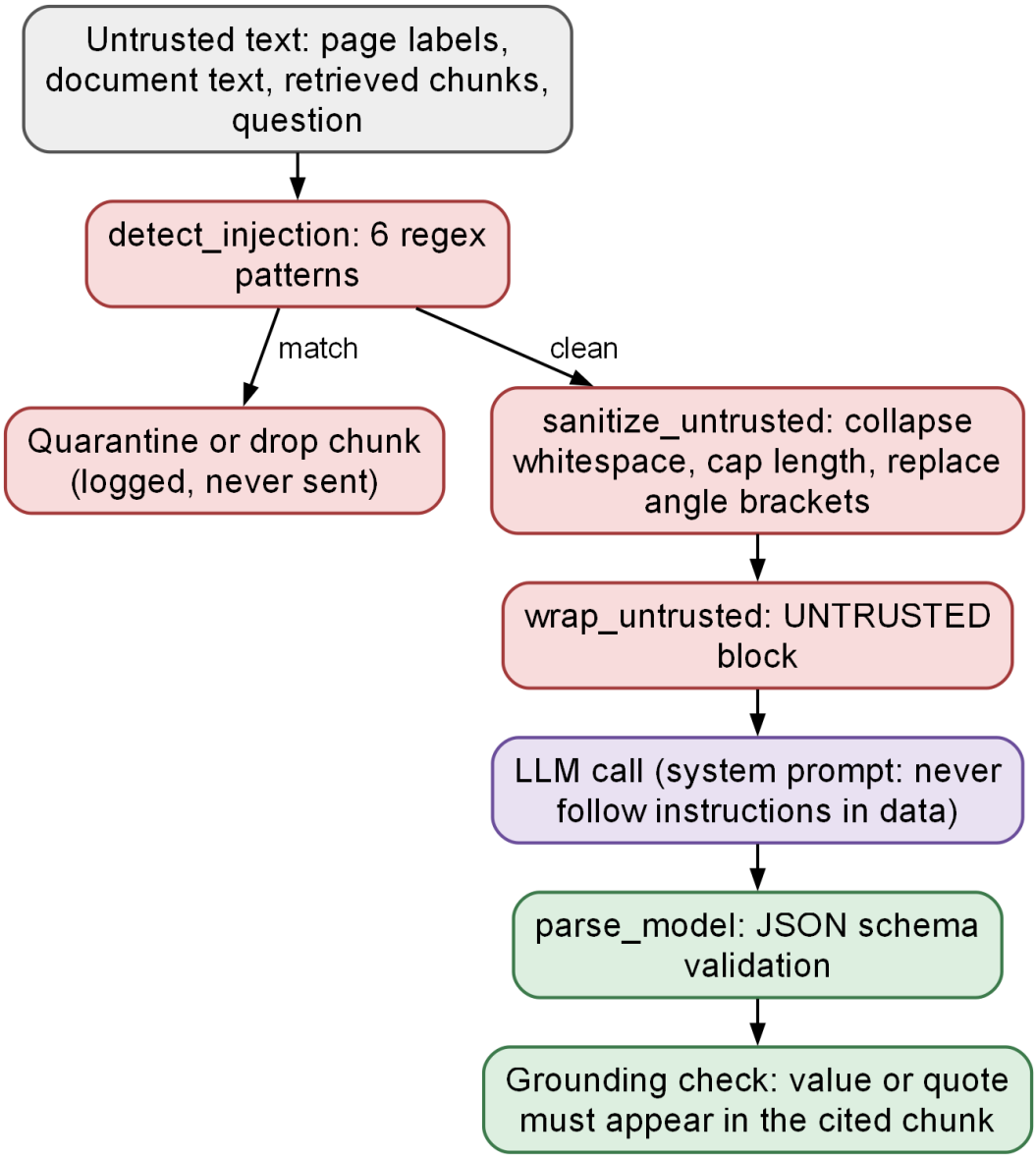


Figure 4.26: Prompt-injection defence applied to every untrusted input

Residual risks are documented in the repository: the dashboard stores its token in local storage (a script injected into the dashboard origin could read it); in-process rate limiting does not span replicas; backups and provider-side copies have their own retention; and personal documents that were committed in earlier versions of the repository were removed from the working tree but the history was deliberately not rewritten (AD-012), so any credential that was ever committed must be rotated.

4.10 API DESIGN

The backend exposes a versioned REST interface under /api/v1. Successful responses use the envelope {success, data, message} and errors use {success: false, error: {code,

message, details}}; validation errors do not echo submitted values. All routes except health, registration and login require a bearer token, and tokens in query strings are not accepted. Table 4.7 lists the principal endpoints; the complete schema is published by FastAPI at /docs and /openapi.json.

Table 4.7: Principal REST endpoints

Group	Method and path	Purpose
Auth	POST /auth/register, POST /auth/login	Create an account; obtain a token
Auth	GET /auth/me, POST /auth/logout, DELETE /auth/me	Current user; revoke token; delete account and data
Documents	POST /documents; GET /documents; GET /documents/{id}	Upload; list with filters; read one
Documents	GET /documents/{id}/status; /facts; POST /process, /reprocess	Progress; extracted facts; (re)run processing
Documents	GET /documents/{id}/download, /preview; DELETE /documents/{id}	Header-authenticated file access; delete with cascade
Facts	GET/POST /profile/facts; PUT/DELETE /profile/facts/{id}	List, add, edit or confirm, delete or reject facts
Facts	GET /profile/facts/conflicts; PATCH .../conflicts/{id}/resolve	Computed conflicts; resolve by candidate, custom value or reject
Profile	GET/PUT /profile; GET /profile/concepts; DELETE /profile/data	Aggregated profile; ontology concepts; delete data, keep account
Forms	POST /forms/sessions; POST /forms/sessions/{id}/analyze (202)	Create session; start batch analysis
Forms	GET .../recommendations; POST .../decisions; GET .../approved-values	Read results; record decisions; values to fill
Forms	POST .../verify; POST .../complete; POST .../answers	Fill results; close session; supply missing answers
Forms	POST /forms/classify-field(s), /forms/recommend(ations); POST /agent/run	Synchronous and agent wrappers of the same pipeline
Retrieval	POST /retrieval/rag; /retrieval/rag/answer; /retrieval/structured	Similar chunks; grounded answer; facts by concept
Reviews	GET /reviews/pending; POST /reviews/{id}/decision	Pending recommendations; decide
Health	GET /health; GET /health/services	Readiness; MongoDB, Chroma, LLM and embedding state

Rate limits are applied per route and per process: for example 10 per minute for registration and login, 20 per minute for uploads of documents, 60 per minute for single-field classification, and 20 to 30 per minute for analysis, recommendation and RAG routes.

4.11 CLIENT DESIGN

Chrome extension

The manifest declares Manifest V3 with the permissions activeTab, scripting and storage, a host permission limited to the local backend origins and optional host permissions for http and https that the user grants on demand. The popup does not run a persistent content script; it injects content.js into the active tab with chrome.scripting when needed, after pinging to see whether it is already present. The content script answers three messages only: SFA_PING, SFA_EXTRACT and SFA_FILL, and only from this extension's own identifier.

The detector selects input, select and textarea elements; skips types hidden, submit, button, reset, image, file, password, range and color; skips disabled and read-only controls, controls whose autocomplete token or descriptive text indicates payment, password or one-time-code, and invisible controls (display none, visibility hidden, zero opacity, zero size, or positioned far off-screen, which is a common honeypot technique). Each remaining field is described by label, accessible name, placeholder, name, id, autocomplete, pattern, section heading, nearby text, required, maximum length and options, plus a has_value flag; **values are never read**. Radio buttons are grouped into one field with the group's question. Field identifiers come from stable attributes (id, then name within form scope, then DOM path), so they survive reopening the popup.

The writer sets values through the element's native property setter and dispatches input and change events so that framework-controlled inputs register the change. Checkboxes and radios are clicked. Drop-downs are matched by the option recommended by the backend, then by value, then by an exactly one label match. After a short settling delay, each field is re-read: the result reports whether the value was applied and whether the page kept it. Disabled, read-only or hidden controls are left alone and reported as not applied.

Figure 4.27 shows the popup's state machine: idle, scanned, analyzing, review, filling and done. State is stored in chrome.storage.session per tab so that reopening the popup resumes the flow; an authentication failure (401) returns to sign-in. The popup polls recommendations every second for up to 180 seconds. Decisions are built so that

only ticked fields with a value become approve or edit decisions, an unticked ready field becomes skip, and blocked fields are never sent.

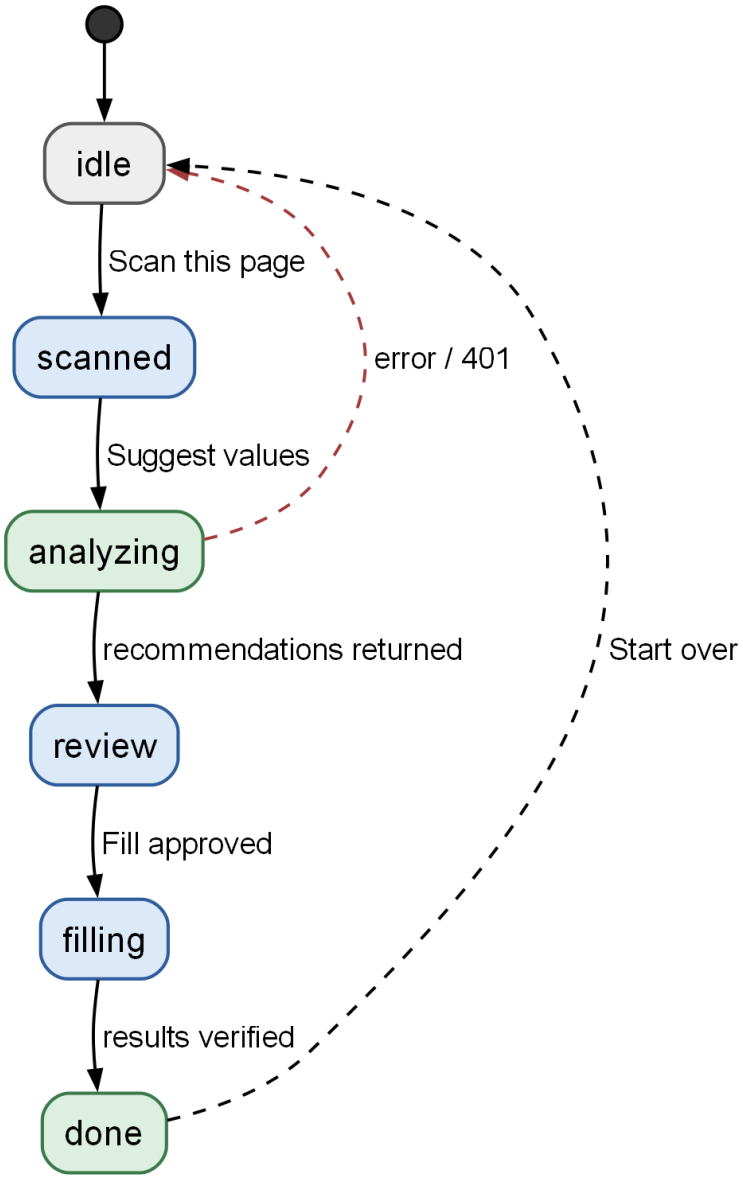


Figure 4.27: State diagram of the extension popup flow

Dashboard

The dashboard is a React single-page application with protected routes: Login and Register are public; Dashboard, Documents, Document details, Profile, Discovered facts, Conflicts, Reviews, Ask, Activity and Settings require a token. A single typed API client unwraps the response envelope, converts errors to a typed exception, builds readable messages from validation errors, and emits a global event on a 401 response so that the user is signed out in one place. Document previews and downloads use

header-authenticated blobs rather than tokens in URLs. In production the dashboard is served by Nginx with a restrictive Content Security Policy and security headers.

CHAPTER SUMMARY

The design was presented from architecture and deployment through data flow, behaviour and structure to algorithms, security and interfaces. Twenty-seven figures and seven tables document the implemented system. The principal design decisions are: a single ontology vocabulary, deterministic classification before any model call, grounded and quoted model output, conflicts computed rather than stored, tenant scoping on every query, and a review step that the extension cannot bypass.

Chapter 5
Result Analysis: Snapshots

This chapter presents snapshots of the running system. All screenshots were captured on 14 October 2026 from the running application with Playwright, using one test account and two synthetic text documents (resume.txt and passport.txt) created for testing. Nothing in the figures was edited. The documents deliberately disagree on the date of birth, so that the conflict workflow can be shown. Each snapshot is described and related to the design of Chapter 4.

5.1 REGISTRATION AND LOGIN

Figure 5.1 shows the sign-in screen with an e-mail and password. Registration is a sibling screen that adds the full name and a confirmation field; the client validates the input before calling the server and shows a clear message for wrong credentials, for rate limiting and for a duplicate e-mail (tests in Section 6.3). On success the token is stored and the user is taken to the dashboard. An expired or revoked token signs the user out from a single place.

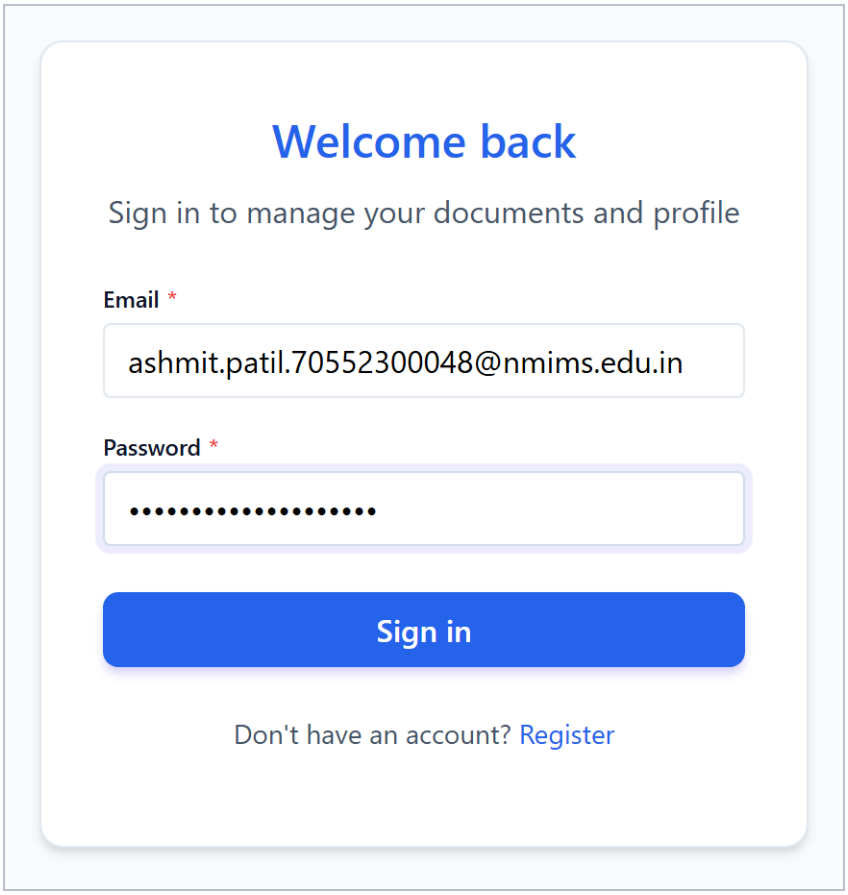


Figure 5.1: Login screen of the dashboard

5.2 DASHBOARD

Figure 5.2 shows the dashboard. Four cards summarise the account from live API data: two documents (none processing), six profile values (none yet confirmed by the user), two conflicts awaiting resolution and no pending field reviews. The recent-documents list shows each file with the number of facts extracted (eight from the résumé, five from the passport record) and a Processed status. The side panel summarises how to fill a form with the extension and states that the user submits the form personally.

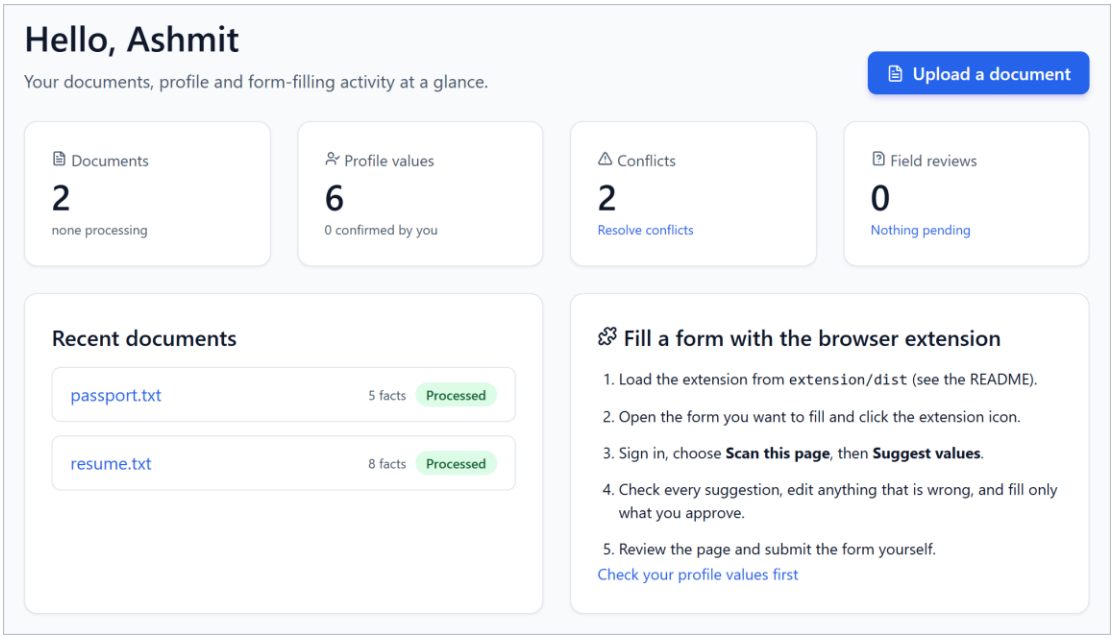


Figure 5.2: Dashboard overview

5.3 DOCUMENT UPLOAD AND PROCESSING

Figure 5.3 shows the Documents page. The upload area accepts PDF, DOCX, TXT, PNG and JPG files up to 10 MB and notes that scanned documents need the optional OCR engine. The table lists each document with its processing status, the number of facts, whether it is searchable (indexed in ChromaDB), and the upload time. A document whose processing failed shows the error message, and deletion asks for confirmation and removes facts, vectors and file.

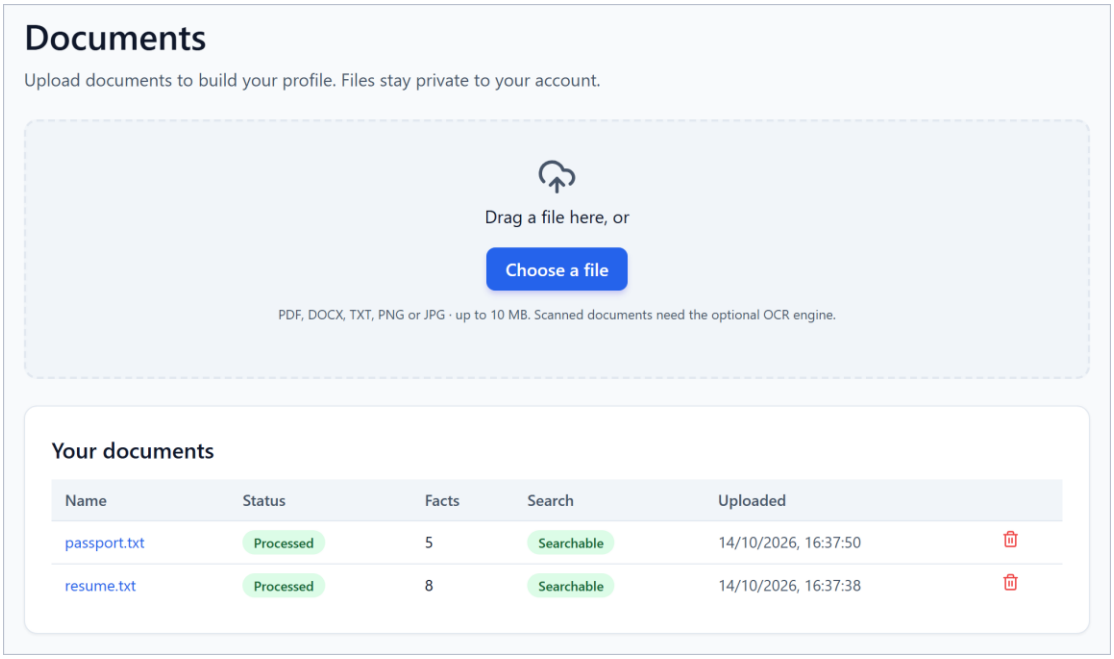


Figure 5.3: Documents page with upload area and processing status

5.4 PROFILE WITH PROVENANCE

Figure 5.4 shows the Identity and Personal groups of the Profile page (the Address and Education groups above them are cropped out to keep the text legible), which groups the resolved values by domain. Each value carries a pill for its origin (Extracted, Normalised), the source document and page, and the actions Confirm, Edit and delete. The passport number is flagged Sensitive and masked until the user reveals it. The date of birth and the full legal name show Conflicting values with a link to resolve them, because the two documents disagree. The telephone number was rewritten to a normalised international form, which is why it is marked Normalised. The form at the bottom adds a value by choosing an ontology concept, and the Saved answers section lists answers the user chose to keep.

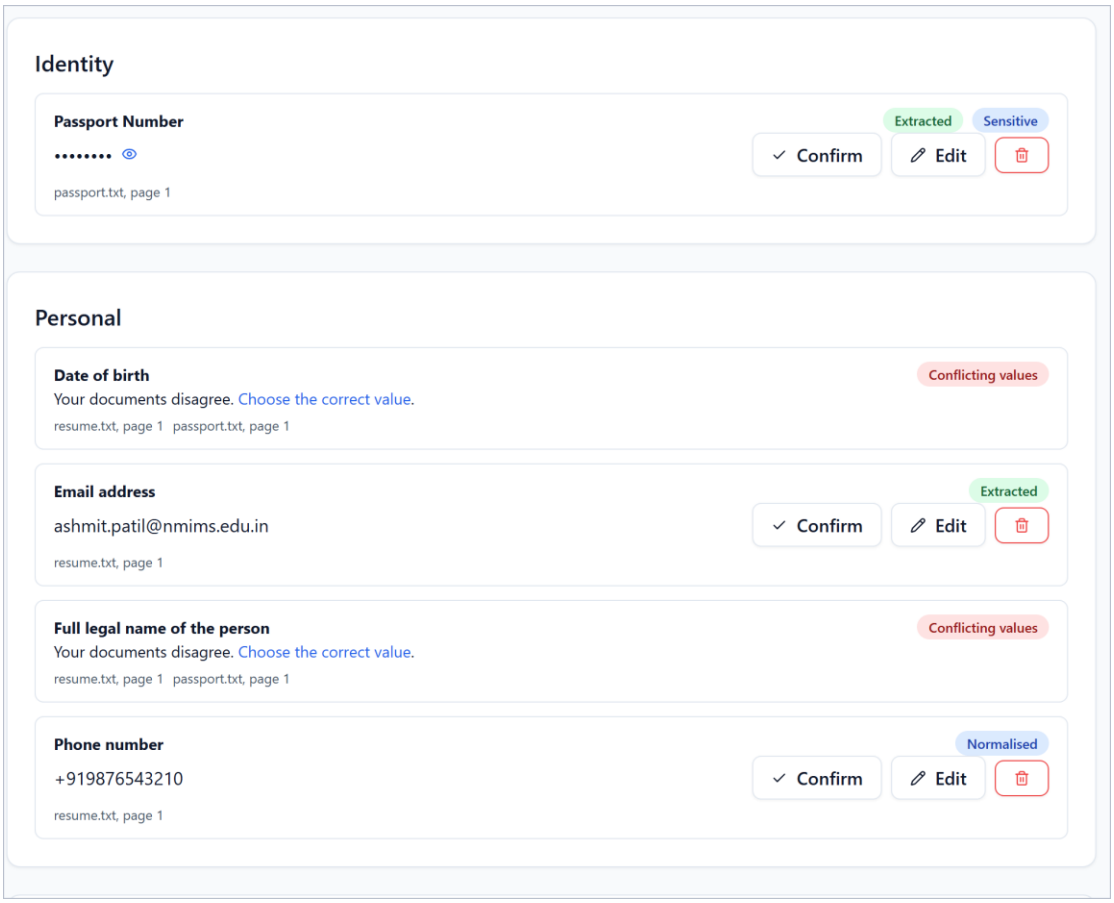


Figure 5.4: Profile page listing extracted facts with provenance

5.5 CONFLICT DETECTION AND RESOLUTION

Figure 5.5 shows the Conflicts page. For the date of birth the résumé gives 14/03/2004 and the passport record 13/03/2004; both were extracted with 99 per cent confidence and each is shown with the verbatim text it came from. The user can

choose a candidate, enter another value, or reject all values. For the full legal name, the candidate “Ashmit Patil” is supported by both documents, whereas “Passport Record” (the title line of the second document) has only 60 per cent confidence. This is the name heuristic of Section 4.8 at work: it is marked inferred with confidence 0.6, and because it disagrees with a better-supported value it is surfaced for the user instead of being used silently.

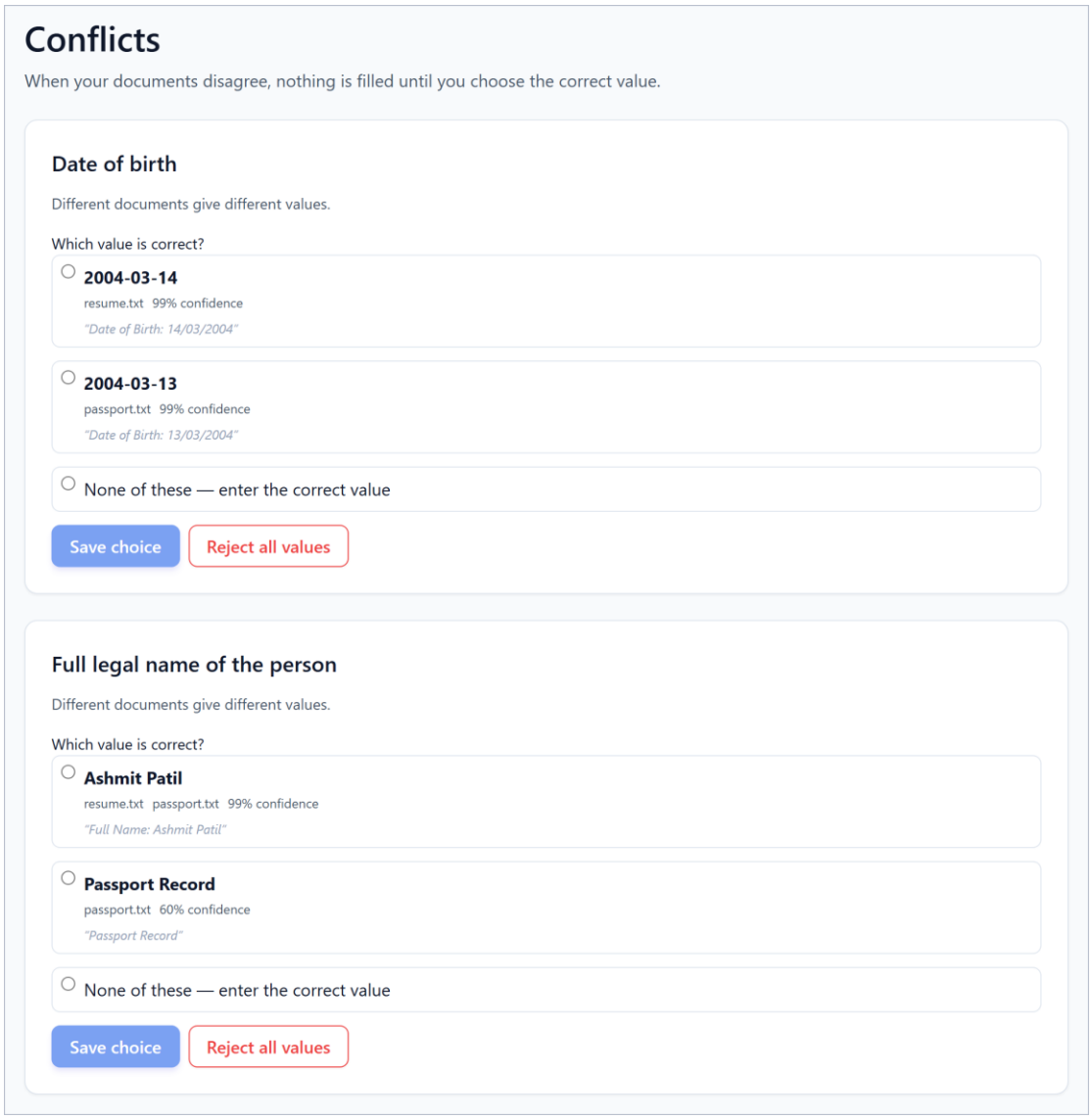


Figure 5.5: Conflicts page with candidate values from different documents

5.6 EXTENSION: SCAN

Figure 5.6 shows the extension popup after the user chose Scan this page on a test form. Six fillable fields were found and two hidden, disabled or sensitive fields were ignored. The popup states this explicitly. Suggest values starts the analysis; Scan

again repeats the scan, which is the current way to handle fields added to the page after the first scan.

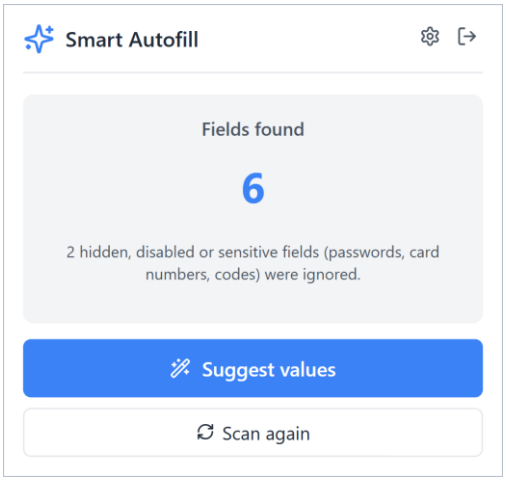


Figure 5.6: Extension popup after scanning a page

5.7 EXTENSION: REVIEW BEFORE FILL

Figure 5.7 shows the review list. The instruction at the top reads “Only ticked fields are filled; nothing is submitted.” The full legal name is unticked with the status Check, because the documents disagree; the two candidates are offered with their source file, together with a box to type the correct value. The e-mail address and phone number are ticked with the status Ready, their source document and page and a confidence of 99 per cent. The Fill button reports how many fields will be filled (three in this view) and Cancel leaves the page unchanged.

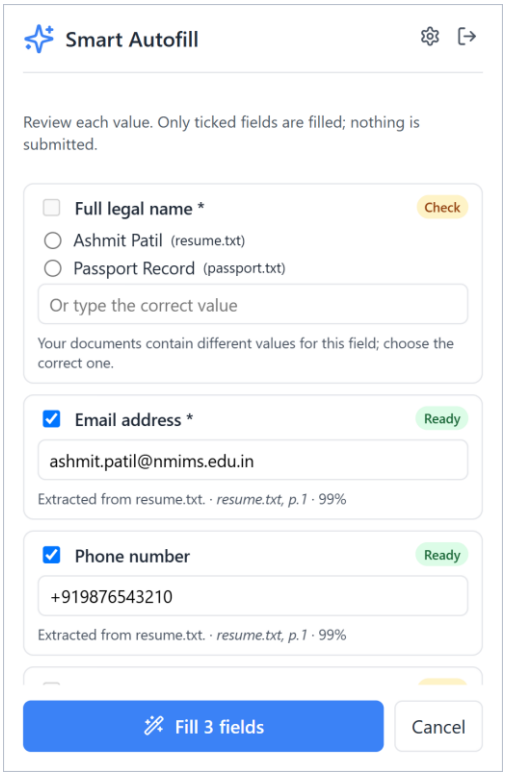


Figure 5.7: Extension popup: review before fill

5.8 GROUNDED QUESTION ANSWERING

Figure 5.8 shows the Ask page. The question “What is my date of birth and which document is it from?” returns an answer with a Sources panel that quotes the cited passage and names the document and page. The page states that answers come only from the user’s own documents and saved answers. The example also shows a limitation: the retrieval step is not aware of conflicts, so the answer reports the passport’s value and cites it, while the Conflicts page shows that the résumé disagrees. The citation lets the user see where the value came from, but the user must still resolve the conflict in the profile.

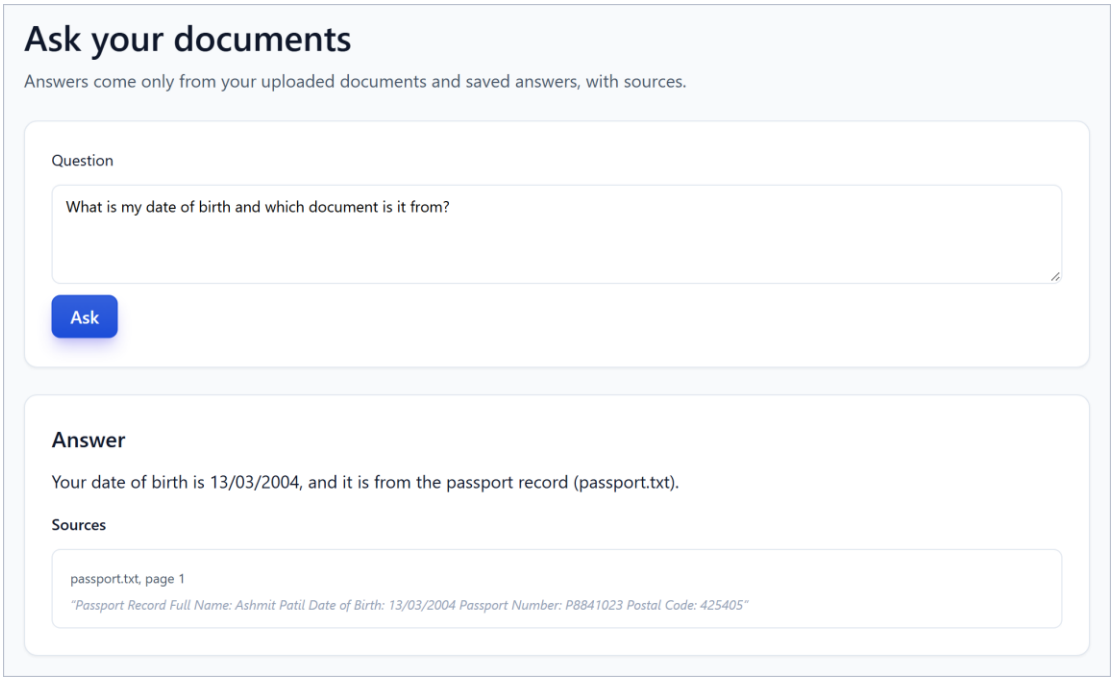


Figure 5.8: Ask page: grounded answer with cited source

5.9 SETTINGS AND SERVICE HEALTH

Figure 5.9 shows the Settings page. The system status panel reports the database, document search (ChromaDB), the AI provider (configured) and the embedding model, from the /health/services endpoint. Below are two confirmation-gated destructive actions: Delete all data removes documents, extracted values, saved answers, search entries and form history while keeping the account, and Delete account removes the account as well.

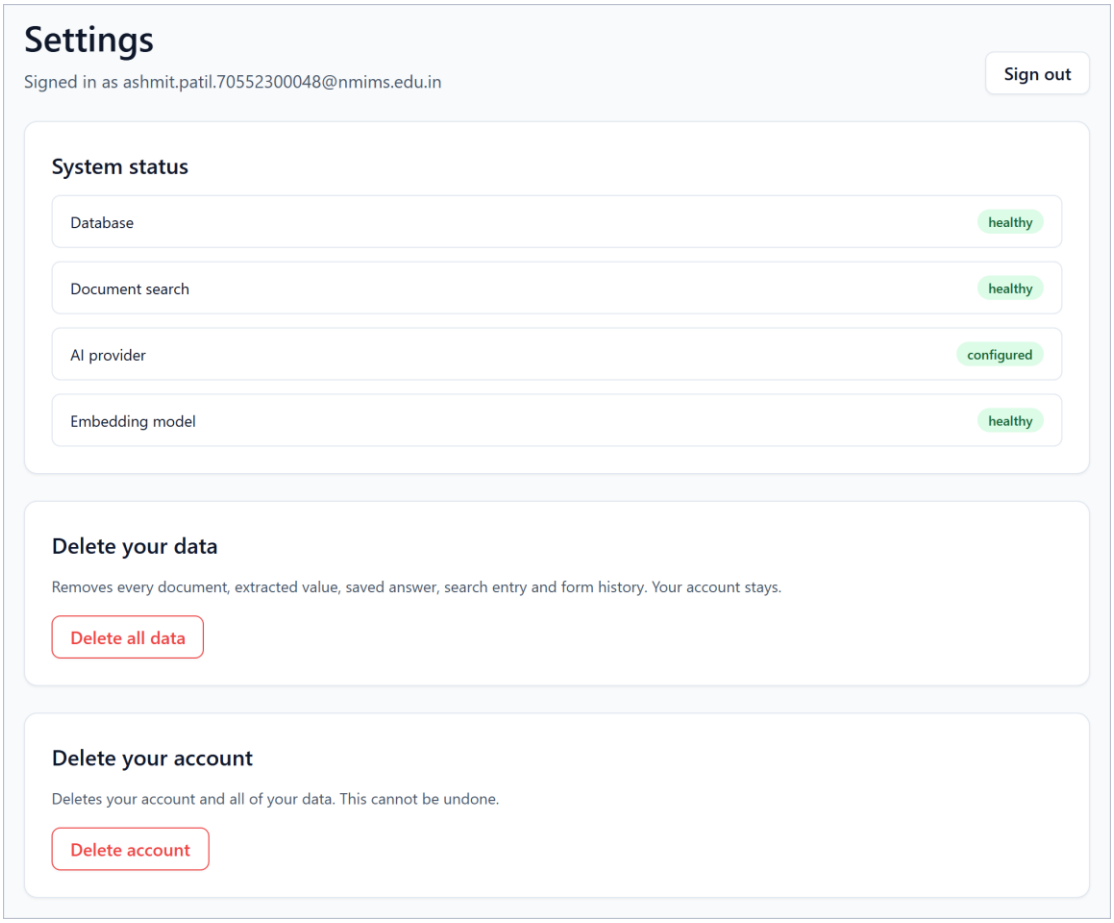


Figure 5.9: Settings page with live service health

5.10 OBSERVATIONS

- **Provenance is visible everywhere:** every value on the profile, conflict, review and answer screens shows its source and, where relevant, its confidence.
- **Uncertainty is routed to the user:** the date-of-birth and name conflicts blocked automatic filling in both the dashboard and the popup.
- **Sensitive values are guarded:** the passport number is masked in the dashboard, and the extension's scan ignored sensitive fields.
- **Known rough edges:** the name heuristic can propose a title line as a name (shown above) and RAG answers do not yet reflect conflicts. Both are recorded as limitations in Chapter 7.

CHAPTER SUMMARY

Nine snapshots of the dashboard and the extension demonstrated the workflow from sign-in to a reviewed fill: document intake with processing status, profile values with provenance and sensitivity masking, conflict resolution, on-demand page scan, per-

field review, grounded answers and service health. The snapshots also exposed two honest limitations, which Chapter 7 carries forward.

Chapter 6
Testing

This chapter describes how the system was tested and records the results. Every number below is taken from the repository's test inventory and from the recorded test runs; no figure was estimated. Where a test category was not run, the report says so.

6.1 TESTING STRATEGY

The strategy follows the repository's testing documentation. Each test belongs to exactly one category: **mocked** (fakes or the mock LLM stand in for external services), **real-service** (a real local MongoDB or ChromaDB) or **real-provider** (a paid external LLM). The default run never touches the developer's own database, never calls a paid API and uses only synthetic personal data. To guarantee this, the backend test configuration forces the LLM provider to mock, storage to a temporary directory, processing to inline, a randomly named MongoDB database that is dropped at the end of the session, and a randomly named Chroma collection. Markers select the real-service and real-provider categories: tests marked mongo or chroma are skipped with a reason if the service is unavailable, and real_llm tests run only when RUN_REAL_LLM_TESTS=1 is set and a provider and key are configured.

Figure 6.1 shows the continuous-integration workflow. On every push and pull request, the first job runs ruff and black on the backend, the mocked backend suite with a MongoDB service, the ChromaDB integration suite with a ChromaDB service, the type checks, tests and builds of the dashboard and the extension, and validation of both Compose files. A second job installs Chromium and runs the Playwright end-to-end test on an isolated Compose stack. Real-provider tests never run in CI.

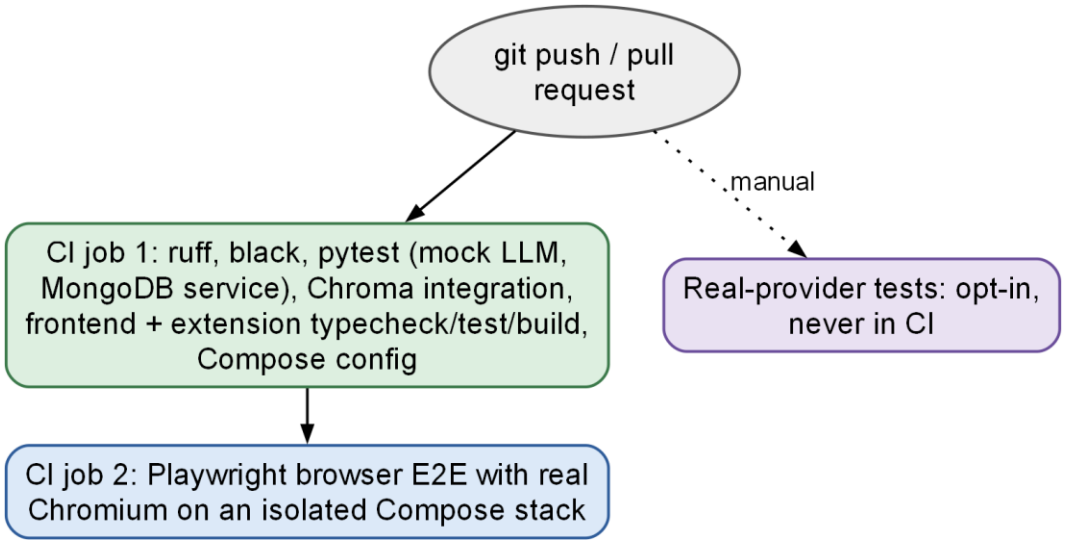


Figure 6.1: Continuous integration pipeline and test layers

6.2 BACKEND TESTS

The backend suite uses pytest. Table 6.1 lists the inventory by file. The counts are the numbers collected by pytest for the default run (146 tests, of which two live in the extension-manifest check) and are derived from the test files, not estimated.

Table 6.1: Backend test inventory (146 tests)

File	Scope	Representative scenarios	Tests
test_auth.py	Authentication	register, login, me, logout; password stored hashed; weak, mismatched or over-long passwords rejected; duplicate e-mail; same error for wrong password and unknown user; missing, malformed, expired and wrong-type tokens; token in query string refused; login rate limit; account deletion revokes the token	12
test_config.py	Configuration	weak or placeholder secret rejected; unknown provider not silently mocked; real provider needs a key; unsupported JWT algorithm; Cloudinary credentials; chunk overlap smaller than size	9
test_documents_api.py	Upload and processing	PDF processed into facts and indexed; DOCX and TXT; executables, fake PDF or DOCX, binary text, empty and content-type-mismatched files rejected; oversize and duplicate upload; corrupted PDF; scanned file without OCR requires review; LLM facts merged; indexing failure keeps facts; reprocessing replaces facts and chunks; delete cascades; delete refused when the vector store is down; cross-user access; safe download headers	18
test_end_to_end.py	Backend acceptance flow	register, upload a synthetic PDF, inline processing, facts, conflict resolution, session, analyze, recommendations, verify	1

File	Scope	Representative scenarios	Tests
test_extraction.py	Extraction and merge	date normalisation (seven forms including invalid dates); value normalisers; labelled facts with provenance; no invented values; LLM values absent from text marked inferred; invalid JSON and provider failure reported; untrusted document wrapping; merge of agreeing and conflicting extractors	14
test_facts_profile_api.py	Facts, conflicts, profile	conflict detected with sources; resolve by candidate, custom value, reject; foreign or unknown conflict returns 404; new document disagreeing with a confirmed value flagged; editing creates a user value and a correction; profile update; memories; delete all data keeps account; concept catalogue	10
test_field_classifier.py	Field classification	obvious fields by label, autocomplete, placeholder, input type, name; ambiguous label not resolved deterministically; name attribute breaks a tie; name in another person's section; one batched LLM call with validated output; invalid LLM output and provider failure marked as errors	13
test_forms_api.py	Recommendation pipeline	validated explainable recommendations; missing information never fabricated; conflicts go to review with candidates; one batched LLM call with minimal data; LLM failure reported as error; injection in field text quarantined; RAG values grounded in cited evidence and dropped otherwise; review decisions, approved values, verify, complete; reanalysis upserts; private sessions; same pipeline for all endpoints; payload validation	14
test_health.py	Platform	success envelope and request ID; unsafe request ID replaced; real component state; 404 envelope; CORS allows the dashboard and PATCH only for allowed origins	5
test_llm_and_prompt_safety.py	LLM and injection safety	injection patterns detected; ordinary labels not flagged; untrusted content cannot close its block; JSON parsing; mock provider empty answers; key sent in header; client errors not retried, server errors retried for three providers; empty completion is an error; factory	16
test_ontology.py	Ontology	phrase matching for thirteen label variants; longer alias beats shorter; substrings do not match; ambiguous label reports both concepts; metadata; tokenisation	18
test_rag_api.py	RAG	only the caller's chunks; unrelated query returns nothing above cutoff; grounded answer with cited sources only; no evidence means no LLM call; uncited answer insufficient; failures return errors not fake answers; structured retrieval scoped; injected chunks never reach the LLM	8
test_value_formatting.py	Value formatting	date formatted for control; select matching exact, never substring; radio duplicates left for review; checkbox ticked only for explicit yes; typed values validated; multiple values need a choice	6
tests/test_extension_manifest.py	Extension manifest	Manifest V3 with minimal permissions; build produces a classic content script	2
Total			146

6.3 DASHBOARD AND EXTENSION TESTS

The dashboard tests use Vitest, Testing Library and jsdom with a stubbed fetch (Table 6.2). The extension tests use Vitest and jsdom (Table 6.3). Both suites include tests for failure paths, not only for success.

Table 6.2: Dashboard test inventory (22 tests)

File	Scenarios	Tests
auth.test.tsx	sign in stores token and redirects; clear messages for wrong credentials and rate limiting; validation before calling the server; expired token signs out; registration; weak and mismatched passwords and duplicate e-mail	6
documents.test.tsx	client-side type and size validation; upload and listing with status; server error on a rejected upload; confirmation before delete; processing failures and facts with provenance; error for a failed document	6
profile.test.tsx	values with provenance, masked sensitive values, confirm and edit; API failures; resolving a conflict by candidate; editing a suggestion and saving it; real counts from the API	5
ConfidenceBadge.test.tsx	high, medium, low and conflict badges; text override	5
Total		22

Table 6.3: Extension test inventory (24 tests)

File	Scenarios	Tests
detector.test.ts	controls described without reading values; aria-labelledby, aria-describedby, sections, nearby text; skipping of password, payment, one-time-code, CAPTCHA, hidden, disabled and read-only controls; zero-size and off-screen honeypots; stable identifiers; radio grouping; select options; long attributes capped; hidden prompt-injection field in an adversarial fixture ignored	9
autofill.test.ts	text and textarea through the native setter; framework-controlled inputs; select by exact value or label, never substring; checkboxes by click; radio by option; vanished, disabled or unknown fields reported; page-rejected values reported; form never submitted	8
api.test.ts	bearer token and unwrapping; error envelope to ApiError and token cleared on 401; GET retried on server errors, POST never retried; network failure and timeout messages; refusal without a token	5
flow.test.ts	only ready values pre-selected; decisions map to approve, edit, skip and ignore protected fields	2
Total		24

6.4 INTEGRATION AND BROWSER END-TO-END TESTS

Three integration tests (marker chroma) run against a real ChromaDB service with real all-MiniLM-L6-v2 embeddings. They verify that indexing, querying and deletion are isolated per user, that a collection created with a different embedding model is

detected, and that an unreachable ChromaDB raises an error instead of falling back to another store.

One Playwright test runs the built extension in real Chromium against the isolated Compose stack (mock LLM, test database). It uploads a synthetic document, opens the fixture sample-application-form.html, drives the popup through scan, analysis and review, approves the fill, asserts the values in the page and asserts that the form was not submitted. A second fixture, adversarial-form.html, contains a hidden prompt-injection field that the detector must ignore; it is covered by a detector unit test.

6.5 TEST CASES

Table 6.4 lists representative test cases in the conventional format. The expected results are the assertions of the named automated tests, and the actual result is the outcome recorded in the run of Table 6.5. The identifiers are for this report.

Table 6.4: Representative test cases

ID	Module	Steps	Expected result	Result
TC-01	Auth	Log in with a wrong password and with an unknown e-mail	Same 401 error for both; no account enumeration	Pass
TC-02	Auth	Send a token in the query string	Request refused	Pass
TC-03	Upload	Upload an .exe renamed or a PDF whose bytes are not PDF	400 invalid file; nothing stored	Pass
TC-04	Upload	Upload the same file twice	Second upload returns a duplicate error	Pass
TC-05	Processing	Upload a scanned PDF with no OCR installed	Status requires_review with message	Pass
TC-06	Processing	Make the vector store fail during indexing	Document processed, facts kept, index_status failed	Pass
TC-07	Conflicts	Two documents give different dates of birth	Conflict listed with both candidates and sources	Pass
TC-08	Classifier	Label “Name” inside a father’s section	User’s own name is not used	Pass
TC-09	Pipeline	Field with no matching fact and no evidence	Status missing_information; no value invented	Pass
TC-10	Pipeline	Field text contains “ignore previous instructions”	Field quarantined; not sent to the LLM	Pass
TC-11	RAG	Model returns a value absent from the cited chunk	Value discarded	Pass
TC-12	RAG	Chunk contains injected instructions	Chunk never reaches the LLM	Pass
TC-13	Isolation	User B requests user A’s document, facts, session	Not found or refused	Pass
TC-14	Deletion	Delete a document while Chroma is down	503 before any MongoDB record is removed	Pass
TC-15	Extension	Scan a page with password, hidden and honeypot fields	Those controls are not listed	Pass

ID	Module	Steps	Expected result	Result
TC-16	Extension	Fill a framework-controlled input and a drop-down	Value kept; option chosen by exact match; verified	Pass
TC-17	Extension	Run the full flow in Chromium	Values filled; form not submitted	Pass

6.6 TEST RESULTS

Table 6.5 reproduces the recorded run in docs/TEST_RESULTS.md (13 October 2026, Windows, Python 3.11, Node 22, Docker Compose and Chromium; backend tests used the mock LLM). All executed tests passed. The real-provider tests were not run because they are opt-in and can incur charges.

Table 6.5: Test results (run of 13 October 2026)

Check	Environment	Result
ruff check backend tests	Local	Passed
black --check backend tests	Local	Passed (111 files unchanged)
pytest (default)	Mock LLM, temporary database	146 passed, 7 deselected
Real ChromaDB integration (marker chroma)	Compose Chroma, real embeddings	3 passed
Dashboard Vitest	jsdom	22 passed
Extension Vitest	jsdom	24 passed
Type check; build	Dashboard and extension	Passed
Playwright browser end-to-end	Real Chromium, isolated Compose stack	1 passed
Compose configuration (development and test)	docker compose config	Both parsed
Real-provider LLM tests	Opt-in	Not run
Total executed automated tests		196 passed, 0 failed

To confirm that the final repository state still passes, the mocked backend suite, the dashboard suite and the extension suite were run again on 14 October 2026 while this report was prepared: 146 backend tests passed with 7 deselected (54.73 s), 22 dashboard tests passed and 24 extension tests passed. The ChromaDB integration test, the Playwright end-to-end test and the real-provider tests were not run again; their results are those of the recorded run.

No timing or performance measurements were taken for the test runs, so the report gives none. Likewise, no measurement of extraction accuracy on a corpus of real

documents was made: the tests check behaviour on synthetic examples, and the accuracy of the language-model tracks on real documents has not been evaluated.

6.7 SECURITY-RELATED VERIFICATION

- **Authentication:** wrong and unknown credentials give one error; expired, malformed, revoked and wrong-type tokens and tokens in query strings are refused; the login limiter returns 429; account deletion revokes the token.
- **Isolation:** tests confirm that documents, facts, conflicts, sessions, recommendations, memories, RAG search and structured retrieval are scoped to the caller, and the Chroma integration tests confirm isolation in the real vector store.
- **Input validation:** executable, forged, binary, empty, oversized, duplicate and content-type-mismatched uploads are rejected.
- **Injection:** patterns are detected without flagging ordinary labels; untrusted content cannot close its block; injected field text and injected chunks never reach the model.
- **Grounding:** values absent from the cited evidence are discarded, and uncited answers become insufficient evidence.
- **Browser safety:** sensitive, hidden, off-screen and honeypot controls are skipped, no value is read, nothing is submitted, and the real-browser test confirms the form was not submitted.
- **Not tested:** penetration testing, load testing, fuzzing of the parsers and accessibility testing were not carried out.

6.8 DISCUSSION

The suite is strongest on behaviours the design promises: refusal paths, isolation, grounding and the user-approval rule each have named tests. It is weaker on measurements: there is no accuracy benchmark for extraction or classification on real documents, no performance test and no test of the optional OCR path, because the OCR engine is optional and was added in the final commit. These gaps are listed in Chapter 7 as future work.

CHAPTER SUMMARY

The system was tested at four levels: 146 backend tests with a mock LLM, 3 integration tests against real ChromaDB, 22 dashboard tests, 24 extension tests and one browser end-to-end test on an isolated stack. All 196 executed tests passed in the recorded run. Real-provider tests, performance tests and accuracy benchmarks were not run, and the report claims nothing about them.

Chapter 7
Conclusion and Future Work

This chapter summarises the work, states how far the objectives of Chapter 1 were met, records the limitations that remain, and proposes future work. As the college format requires, the report has a single conclusion, and it is given here.

7.1 CONCLUSION

The project set out to build a privacy-first assistant that fills web forms from the user's own documents while keeping every value traceable and every decision with the user. The resulting Smart Form Auto-Filling System consists of a FastAPI backend with MongoDB and ChromaDB, a React dashboard with eleven routes, and a Chrome Manifest V3 extension, deployable with a single Docker Compose project. Documents in four formats are parsed, facts are extracted by deterministic rules over a 91-concept ontology and, optionally, by a language model, and each fact keeps its origin, evidence text, page, method and confidence. Disagreements between documents are computed as conflicts and resolved by the user. A seven-node LangGraph pipeline classifies form fields and resolves values from the profile or from quoted document evidence, and the extension fills only the values that the user approves, verifies what the page kept, and never submits the form.

Achievement of objectives

- **Objective 1, parsing and extraction:** achieved for PDF, DOCX, TXT, PNG and JPG; scanned input needs the optional OCR engine.
- **Objective 2, provenance:** achieved; every fact stores origin, evidence text, page, method and confidence.
- **Objective 3, conflicts:** achieved; computed from active facts, resolved by the user, verified by tests and demonstrated in Chapter 5.
- **Objective 4, classification:** achieved; deterministic rules first and one batched model call for the unresolved remainder.

- **Objective 5, grounded values:** achieved; a value comes from the profile or from a quoted passage, and ungrounded values are discarded.
- **Objective 6, thresholds and sensitivity:** achieved; confidence 0.75 for classification and 0.85 for ready, with confirmation for sensitive and inferred values.
- **Objective 7, security:** achieved and tested for injection, isolation and token handling; residual risks are documented in Section 4.9.
- **Objective 8, verification:** achieved; 196 automated tests passed in the recorded run. Real-provider tests, performance tests and accuracy benchmarks were not run.

Principal findings

- **Rules before models:** deterministic matching over a small ontology resolves the common fields, so the language model is needed only for the ambiguous remainder, which keeps cost and risk low.
- **Conflicts must be first-class:** realistic users hold documents that disagree; surfacing the disagreement was more useful than any attempt to pick the right value automatically.
- **Grounding checks are cheap and effective:** requiring a value or quotation to appear in the cited chunk removes a class of fabricated answers without a second model call.
- **Verification after filling matters:** framework-controlled pages can discard a naive assignment, so the extension re-reads each field and reports the outcome.
- **Honest limits:** the name heuristic and the conflict-unaware RAG answer showed in Chapter 5 that the evidence trail lets the user catch the system's mistakes, which is the intended role of the design.

In short, the project demonstrates that a document-driven, evidence-preserving and human-approved autofill assistant can be built from open-source components and verified by automated tests. It does not claim measured accuracy on real-world documents or on arbitrary websites; that evaluation remains to be done.

7.2 LIMITATIONS

The limitations stated in Section 3.8 and discovered during testing and snapshotting are summarised here together with the proposed work. Table 7.1 pairs each limitation with a proposed improvement.

Table 7.1: Current limitations and proposed work

Current limitation	Proposed future work
No accuracy evaluation of extraction, classification or OCR on a corpus of real documents and real forms	Build a labelled benchmark of anonymised documents and forms; report precision and recall per concept; compare rule-only, LLM-only and combined tracks
Scanned documents need the optional PaddleOCR engine, which is large and not covered by a real-engine test	Add an OCR integration test and measure accuracy; consider a lighter engine
Ontology of 91 concepts; fields with no concept are left for the user	Extend and localise the ontology, including regional languages; let the user map unknown fields and learn the mapping
Static scan: fields added after the scan need a manual re-scan	Observe DOM mutations and support multi-step forms and iframes
Chrome-only extension	Port to Firefox and other Manifest V3 browsers; consider a mobile companion
Name heuristic can propose a title line as a name	Strengthen header-word rules or require confirmation before using any inferred name
RAG answers are not aware of conflicts	Warn in the answer when the retrieved concept has a conflict, and link to the Conflicts page
In-process rate limiting does not span replicas	Use a shared limiter backend or edge rate limiting
Dashboard token in local storage	Move to an HttpOnly cookie with CSRF protection, or add refresh-token rotation
No performance, load or accessibility tests	Add load tests for upload and analysis, and accessibility audits of the dashboard and popup
Real-provider tests not run	Run the opt-in suite with synthetic data and record provider-specific behaviour
Personal files committed earlier remain in Git history	Rewrite history in a coordinated clean-up and rotate any credential that was ever committed

7.3 FUTURE WORK

1. **Evaluation first.** Build the benchmark described above, because every other improvement should be judged against measured accuracy.
2. **Smarter conflict handling.** Suggest the more recent or more authoritative document type for a concept (for example a passport over a résumé for a date of birth) while still asking the user to confirm.
3. **Richer retrieval.** Evaluate graph-based memory such as HippoRAG-style retrieval [13] for questions that need several facts, and add a conflict-aware answer step.

4. **Privacy-stronger models.** Support a local or on-device language model so that no document text leaves the machine, and local embedding is already the default.
5. **Per-site consent.** Let the user keep a list of sites on which the extension may run without re-granting permission, while keeping review before fill.
6. **More forms.** Handle multi-step wizards, iframes, date pickers and custom drop-down widgets, and extend value formatting to more input conventions.
7. **Operational hardening.** Shared rate limiting, backup and retention policy, secret scanning in CI, and a hardened production deployment with TLS in front.

CHAPTER SUMMARY

The work met its objectives: an auditable, human-approved pipeline from personal documents to filled forms, verified by 196 passing automated tests. The remaining limitations were listed with proposed work, led by an evaluation of accuracy on real documents and forms.

References

[1] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon and G. Neubig, “WebArena: A Realistic Web Environment for Building Autonomous Agents”, International Conference on Learning Representations (ICLR), 2024. arXiv:2307.13854.

[2] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun and Y. Su, “Mind2Web: Towards a Generalist Agent for the Web”, Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track, 2023. arXiv:2306.06070.

[3] B. Zheng, B. Gou, J. Kil, H. Sun and Y. Su, “GPT-4V(ision) is a Generalist Web Agent, if Grounded”, Proceedings of the 41st International Conference on Machine Learning (ICML), PMLR vol. 235, pp. 61349–61385, 2024.

[4] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan and D. Yu, “WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models”, Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers, pp. 6864–6890, 2024. DOI: 10.18653/v1/2024.acl-long.371.

[5] W. Hong, W. Wang, Q. Lv, J. Xu, W. Yu, J. Ji, Y. Wang, Z. Wang, Y. Dong, M. Ding and J. Tang, “CogAgent: A Visual Language Model for GUI Agents”, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 14281–14290, 2024.

[6] K. Cheng, Q. Sun, Y. Chu, F. Xu, Y. Li, J. Zhang and Z. Wu, “SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents”, Proceedings of the 62nd Annual Meeting of the ACL, Volume 1: Long Papers, pp. 9313–9332, 2024.

[7] K. You, H. Zhang, E. Schoop, F. Weers, A. Swearngin, J. Nichols, Y. Yang and Z. Gan, “Ferret-UI: Grounded Mobile UI Understanding with Multimodal

LLMs", European Conference on Computer Vision (ECCV), 2024. DOI: 10.1007/978-3-031-73039-9_14.

[8] A. Drouin et al., "WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks?", Proceedings of the 41st International Conference on Machine Learning (ICML), PMLR vol. 235, pp. 11642–11662, 2024.

[9] D. Lee, J. Lee, K. Kim, J. Tack, J. Shin, Y. W. Teh and K. Lee, "Learning to Contextualize Web Pages for Enhanced Decision Making by LLM Agents", International Conference on Learning Representations (ICLR), 2025. arXiv:2503.10689.

[10] D. Nguyen et al., "GUI Agents: A Survey", Findings of the Association for Computational Linguistics: ACL 2025, pp. 22522–22538, 2025. DOI: 10.18653/v1/2025.findings-acl.1158.

[11] L. Wang et al., "A Survey on Large Language Model Based Autonomous Agents", Frontiers of Computer Science, vol. 18, no. 6, article 186345, 2024. DOI: 10.1007/s11704-024-40231-1.

[12] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang and X. Wu, "Unifying Large Language Models and Knowledge Graphs: A Roadmap", IEEE Transactions on Knowledge and Data Engineering, vol. 36, no. 7, pp. 3580–3599, July 2024. DOI: 10.1109/TKDE.2024.3352100.

[13] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga and Y. Su, "HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models", Advances in Neural Information Processing Systems 37 (NeurIPS 2024), 2024. arXiv:2405.14831.

[14] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong and J.-R. Wen, "A Survey on the Memory Mechanism of Large Language Model based Agents", arXiv:2404.13501, 2024 (journal version listed in ACM Transactions on Information Systems).

[15] A. Abdallah, D. Eberharter, Z. Pfister and A. Jatowt, "Transformers and Language Models in Form Understanding: A Comprehensive Review of Scanned Document Analysis", arXiv:2403.04080, 2024.

[16] X. Lin, P. Ilia and J. Polakis, “Fill in the Blanks: Empirical Analysis of the Privacy Threats of Browser Form Autofill”, Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS), 2020.

[17] Y. Shao et al., “FormLM: Recommending Creation Ideas for Online Forms by Modelling Semantic and Structural Information”, Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 8133–8149, 2022. DOI: 10.18653/v1/2022.emnlp-main.557.

[18] H. Belgacem, X. Li, D. Bianculli and L. Briand, “A Machine Learning Approach for Automated Filling of Categorical Fields in Data Entry Forms”, ACM Transactions on Software Engineering and Methodology. arXiv:2202.08572.

[19] P. Alian, N. Nashid, M. Shahbandeh and A. Mesbah, “Semantic Constraint Inference for Web Form Test Generation”, ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), Vienna, September 2024. arXiv:2402.00950.

[20] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020. arXiv:2005.11401.

[21] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”, Proceedings of EMNLP-IJCNLP 2019, pp. 3982–3992, Hong Kong, 2019. DOI: 10.18653/v1/D19-1410.

[22] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, “MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers”, Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020. arXiv:2002.10957.

[23] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz and M. Fritz, “Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection”, arXiv:2302.12173, 2023.

[24] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT)”, RFC 7519, Internet Engineering Task Force, May 2015.

[25] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme”, Proceedings of the USENIX Annual Technical Conference, FREENIX Track, 1999.

[26] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications”, online resource, https://genai.owasp.org/llm-top-10/.

[27] S. Ramírez, “FastAPI documentation”, online resource, https://fastapi.tiangolo.com/.

[28] Meta Platforms, Inc., “React documentation”, online resource, https://react.dev/.

[29] MongoDB, Inc., “MongoDB Manual”, online resource, https://www.mongodb.com/docs/.

[30] Chroma, “Chroma documentation”, online resource, https://docs.trychroma.com/.

[31] LangChain, Inc., “LangGraph documentation”, online resource, https://langchain-ai.github.io/langgraph/.

[32] Google LLC, “Chrome Extensions: Manifest V3 documentation”, online resource, https://developer.chrome.com/docs/extensions/.

[33] Docker, Inc., “Docker Compose documentation”, online resource, https://docs.docker.com/compose/.

[34] Microsoft Corporation, “Playwright documentation”, online resource, https://playwright.dev/.

Appendix I: Source Code Excerpts

The excerpts below are copied unchanged from the repository at the paths shown. They are chosen because they implement the main design decisions of Chapter 4. Code is set in a monospaced font for legibility.

I.1 Prompt-injection helpers

File: `backend/app/security/prompt_safety.py`. detect_injection flags six patterns, sanitize_untrusted collapses whitespace and neutralises angle brackets, and wrap_untrusted places pre-sanitised text in a tagged data block whose delimiters cannot be forged by the content.

```
_INJECTION_PATTERNS: tuple[re.Pattern[str], ...] = (
    re.compile(

r"\b(ignore|disregard|forget|override)\s+(all\s+|any\s+|the\s+)?(previous|prior|above
|earlier|system)\s+(instructions?|prompts?|rules?)",
        re.I,
    ),
    re.compile(
        r"\b(reveal|print|show|dump|output)\s+(all\s+|the\s+|any\s+)?"

r"((stored|user'?s?|private|personal|system|saved|other)\s+){1,2}(data|facts|informat
ion|prompt|details|profile)",
        re.I,
    ),
    re.compile(r"\byou\s+are\s+now\s+(a|an|the)\b", re.I),
    re.compile(r"\bnew\s+(system\s+)?instructions?\s*:", re.I),
    re.compile(r"\b(system|developer)\s+prompt\b", re.I),
    re.compile(r"</?\s*(system|instruction|instructions|prompt|untrusted_[a-
z_]+)\s*>", re.I),
)

_TAG_CHARS = str.maketrans({"<": "‹", ">": "›"})


def detect_injection(text: str | None) -> bool:
    """Return True when text contains a known prompt-injection pattern."""
    if not text:
        return False
    for pattern in _INJECTION_PATTERNS:
        if pattern.search(text):
            logger.warning("prompt_injection_detected", extra={"pattern":
pattern.pattern[:60]})
            return True
    return False


def sanitize_untrusted(text: object, max_chars: int) -> str:
    """Collapse whitespace, cap length and neutralise angle brackets."""
    cleaned = " ".join(str(text or "").split())
    return cleaned[:max_chars].translate(_TAG_CHARS)


def wrap_untrusted(tag: str, content: str) -> str:
    """Wrap pre-sanitised content in an untrusted-data block."""
```

```
    safe_tag = re.sub(r"[^A-Z_]", "", tag.upper()) or "UNTRUSTED_DATA"
    return f"<{safe_tag}>\n{content.translate(_TAG_CHARS)}\n</{safe_tag}>"
```

I.2 LangGraph construction

File: `backend/app/agents/graph.py`. The seven nodes and the two conditional edges of Figure 4.21. Classification is skipped when every field is resolved, and document retrieval is skipped when no field needs it.

```
        graph = StateGraph(FillState)
        graph.add_node("sanitize", self._sanitize)
        graph.add_node("deterministic_classify", self._deterministic_classify)
        graph.add_node("llm_classify", self._llm_classify)
        graph.add_node("resolve_profile", self._resolve_profile)
        graph.add_node("resolve_documents", self._resolve_documents)
        graph.add_node("decide", self._decide)
        graph.add_node("persist", self._persist)
        graph.add_edge(START, "sanitize")
        graph.add_edge("sanitize", "deterministic_classify")
        graph.add_conditional_edges(
            "deterministic_classify",
            self._route_classification,
            {"llm": "llm_classify", "skip": "resolve_profile"},
        )
        graph.add_edge("llm_classify", "resolve_profile")
        graph.add_conditional_edges(
            "resolve_profile",
            self._route_documents,
            {"rag": "resolve_documents", "skip": "decide"},
        )
        graph.add_edge("resolve_documents", "decide")
        graph.add_edge("decide", "persist")
        graph.add_edge("persist", END)
```

I.3 Decision node

File: `backend/app/agents/graph.py`. Applies formatting, sensitivity and confidence rules to give each field its final status (Figure 4.23).

```
    async def _decide(self, state: FillState) -> FillState:
        work = state["work"]
        fields = {f.field_id: f for f in state["fields"]}
        for field_id, item in work.items():
            item.pop("quarantined", None)
            narrative = item.pop("narrative", False)
            if item["status"] in ("security blocked", "skipped", "error",
"needs_review"):
                continue
            value = item["recommended_value"]
            if value is None:
                item["status"] = "missing_information"
                if not item["reason"]:
                    item["reason"] = (
                        "No matching information was found in your profile or
documents."
                        if item["concept"] or narrative
                        else "This field was not recognised; fill it yourself if
needed."
                    )
                continue
            concept = item["concept"]
            data_type = self.ontology.data_type(concept) if concept else "string"
            formatted = format for field(fields[field id], data type, value)
            if formatted.value is None:
                item["status"] = "needs_review"
```

```
                item["candidates"] = [
                    {"value": value, "confidence": item["confidence"], "source":
item["source"]}
                ]
                item["recommended_value"] = None
                item["reason"] = f"{formatted.problem}. Check the value before
filling."
                continue
            item["recommended_value"] = formatted.value
            item["selected_option"] = formatted.selected_option
            sensitive = bool(concept) and (
                self.ontology.is_sensitive(concept) or concept in self.sensitive
            )
            if sensitive:
                item["requires_confirmation"] = True
                item["reason"] = (
                    f"{item['reason']} Sensitive information always needs your
confirmation.".strip()
                )
            if item["origin"] == "inferred":
                item["requires_confirmation"] = True
            if (
                item["confidence"] >= self.settings.autofill_min_confidence
                and not item["requires_confirmation"]
            ):
                item["status"] = "ready"
            else:
                item["status"] = "needs_review"
        return {"work": work}
```

I.4 Concept view and conflict logic

File: `backend/app/services/facts.py`. Groups active facts by comparison key; one group gives a resolved view, several groups give multiple or conflict depending on the concept's cardinality.

```
    def _view(self, concept: str, records: list[dict[str, Any]]) -> ConceptView:
        data_type = self.ontology.data_type(concept)
        groups: dict[str, list[dict[str, Any]]] = defaultdict(list)
        for record in records:
            groups[comparison_key(data_type, record["value"])].append(record)
        candidates = [_candidate(items) for items in groups.values()]
        candidates.sort(key=lambda c: (c.is_user_confirmed, c.confidence),
reverse=True)
        label = self.ontology.description(concept)
        if len(candidates) == 1:
            best = candidates[0]
            return ConceptView(
                concept=concept,
                label=label,
                status="resolved",
                value=best.value,
                values=[best.value],
                confidence=best.confidence,
                is_user_confirmed=best.is_user_confirmed,
                origin=_best_origin(best.sources),
                sources=best.sources,
                candidates=candidates,
            )
        if not self.ontology.is_single_valued(concept):
            return ConceptView(
                concept=concept,
                label=label,
                status="multiple",
                values=[c.value for c in candidates],
                confidence=max(c.confidence for c in candidates),
                sources=[s for c in candidates for s in c.sources],
                candidates=candidates,
            )
        return ConceptView(
```

```
            concept=concept,
            label=label,
            status="conflict",
            confidence=max(c.confidence for c in candidates),
            sources=[s for c in candidates for s in c.sources],
            candidates=candidates,
        )
```

I.5 Merging candidate facts

File: `backend/app/extraction/merge.py`. Combines agreeing candidates, raises confidence slightly for agreement between extractors and keeps different values apart.

```
def merge candidates(candidates: list[CandidateFact]) -> list[CandidateFact]:
    merged: dict[tuple[str, str], CandidateFact] = {}
    for candidate in candidates:
        key = (
            candidate.concept or f"semantic:{candidate.semantic_type}",
            comparison_key(candidate.data_type, candidate.value),
        )
        if not key[1]:
            continue
        existing = merged.get(key)
        if existing is None:
            merged[key] = candidate
            continue
        methods = sorted({*existing.method.split("+"), *candidate.method.split("+")})
        agreeing = len(methods) > 1
        better = (
            candidate
            if  ORIGIN RANK[candidate.origin] <  ORIGIN RANK[existing.origin]
            else existing
        )
        merged[key] = CandidateFact(
            concept=existing.concept or candidate.concept,
            semantic_type=existing.semantic_type,
            field=existing.field,
            value=better.value,
            raw_value=better.raw_value,
            data_type=better.data_type,
            confidence=min(
                0.99, max(existing.confidence, candidate.confidence) + (0.03 if
agreeing else 0)
            ),
            origin=better.origin,
            method="+".join(methods),
            page=better.page if better.page is not None else (existing.page or
candidate.page),
            evidence_text=better.evidence_text or existing.evidence_text,
            entity_type=existing.entity_type or candidate.entity_type,
        )
    return list(merged.values())
```

I.6 Field eligibility in the extension

File: `extension/src/content/detector.ts`. Skips protected input types, disabled and read-only controls, sensitive autocomplete tokens and descriptive text, and invisible or far off-screen controls.

```
export function isEligible(element: FieldElement): boolean {
  if (element instanceof HTMLInputElement &&
SKIPPED_INPUT_TYPES.has(element.type.toLowerCase())) return false;
  if (element.disabled) return false;
  if ((element instanceof HTMLInputElement || element instanceof HTMLTextAreaElement)
&& element.readOnly) {
```

```
    return false;
  }
  if (SENSITIVE_AUTOCOMPLETE.test(element.getAttribute('autocomplete') || '')) return
false;
  const describing = [
    element.getAttribute('name'), element.getAttribute('id'),
element.getAttribute('placeholder'),
    element.getAttribute('aria-label'), labelText(element),
  ].join(' ');
  if (SENSITIVE_TEXT.test(describing)) return false;
  return isVisible(element);
}

export function isVisible(element: Element): boolean {
  const view = element.ownerDocument.defaultView;
  if (!view) return false;
  if (element.closest('[hidden], [aria-hidden="true"], [inert]')) return false;
  for (let node: Element | null = element; node; node = node.parentElement) {
    const style = view.getComputedStyle(node);
    if (style.display === 'none' || style.visibility === 'hidden' || style.visibility
=== 'collapse') return false;
    if (Number.parseFloat(style.opacity) === 0) return false;
  }
  const rect = element.getBoundingClientRect();
  if (rect.width === 0 && rect.height === 0) return false;
  // Controls moved far outside the page are a common honeypot / hidden-field trick.
  if (rect.right < -1000 || rect.bottom < -1000) return false;
  return true;
}
```

I.7 Writing a value into a page

File: `extension/src/content/autofill.ts`. Uses the native value setter and input and change events so that framework-controlled inputs register the change, then verifies the result after a short delay.

```
function setNativeValue(element: FieldElement, value: string): void {
  const prototype = Object.getPrototypeOf(element);
  const setter = Object.getOwnPropertyDescriptor(prototype, 'value')?.set;
  if (setter) setter.call(element, value);
  else element.value = value;
}

function announce(element: FieldElement): void {
  element.dispatchEvent(new Event('input', { bubbles: true }));
  element.dispatchEvent(new Event('change', { bubbles: true }));
}

function fillText(element: FieldElement, value: ApprovedValue): Attempt {
  element.focus({ preventScroll: true });
  setNativeValue(element, value.value);
  element.dispatchEvent(
    new InputEvent('input', { bubbles: true, inputType: 'insertReplacementText',
data: value.value }),
  );
  element.dispatchEvent(new Event('change', { bubbles: true }));
  element.blur();
  return {
    result: { field_id: value.field_id, applied: true, verified: false },
    verify: () => (element.value === value.value
      ? { verified: true }
      : { verified: false, reason: 'The page changed or rejected the value' }),
  };
}
```

I.8 Settings validators

File: `backend/app/core/config.py`. Rejects placeholder or short secrets, unsupported algorithms, unknown providers, a real provider without a key and inconsistent chunk settings at start-up.

```
    @field_validator("jwt_secret_key")
    @classmethod
    def _strong_secret(cls, value: str) -> str:
        if value.strip().lower() in _PLACEHOLDER_SECRETS:
            raise ValueError("JWT_SECRET_KEY is still a placeholder; generate a
random secret")
        if len(value) < 32:
            raise ValueError("JWT_SECRET_KEY must be at least 32 characters long")
        return value

    @field_validator("jwt_algorithm")
    @classmethod
    def _allowed_algorithm(cls, value: str) -> str:
        if value not in _ALLOWED_JWT_ALGORITHMS:
            raise ValueError(f"JWT_ALGORITHM must be one of {',
'.join(_ALLOWED_JWT_ALGORITHMS)}")
        return value

    @field_validator("llm_provider")
    @classmethod
    def _known_provider(cls, value: str) -> str:
        provider = (value or "").strip().lower()
        if provider == "gemini":
            provider = "google"
        if provider not in SUPPORTED_LLM_PROVIDERS:
            raise ValueError(
                f"LLM_PROVIDER must be one of {', '.join(SUPPORTED_LLM_PROVIDERS)}
(got {value!r})"
            )
        return provider

    @model_validator(mode="after")
    def _provider_has_key(self) -> "Settings":
        if self.llm_provider != "mock" and not self.llm_api_key.strip():
            raise ValueError(f"LLM_API_KEY is required when
LLM_PROVIDER={self.llm_provider}")
        if self.rag_chunk_overlap >= self.rag_chunk_size:
            raise ValueError("RAG_CHUNK_OVERLAP must be smaller than RAG_CHUNK_SIZE")
        if self.storage_provider == "cloudinary" and not (
            self.cloudinary_cloud_name and self.cloudinary_api_key and
self.cloudinary_api_secret
        ):
            raise ValueError("Cloudinary storage requires CLOUDINARY_* credentials")
        return self
```

Appendix II: Ontology Excerpt

The first concepts of backend/app/ontology/field_ontology.yaml (version 3.0.0), unchanged. The full file defines 91 concepts. A concept with an autocomplete list is matched by the HTML autocomplete token before any text matching.

```
version: 3.0.0
concepts:
  achievement.organization:
    canonical_name: achievement.organization
    description: Organization granting the achievement
    aliases:
    - awarded by
    - organizer
    data_type: string
    cardinality: multiple
    is_structured: true
  achievement.result:
    canonical_name: achievement.result
    description: Result or rank obtained
    aliases:
    - rank
    - result
    data_type: string
    cardinality: multiple
    is_structured: true
  achievement.title:
    canonical_name: achievement.title
    description: Title of the achievement or award
    aliases:
    - accomplishment
    - achievement
    - award
    - honor
    data type: string
    cardinality: multiple
    is_structured: true
  address.city:
    canonical_name: address.city
    description: City or town
    aliases:
    - city
    - city town
    - district
    - town
    data_type: string
    cardinality: single
    is_structured: true
    autocomplete:
    - address-level2
  address.country:
    canonical_name: address.country
    description: Country
    aliases:
    - country
    - country name
    data_type: string
    cardinality: single
    is_structured: true
    autocomplete:
    - country
    - country-name
  address.full_address:
```

```
    canonical_name: address.full_address
    description: Complete postal address
    aliases:
    - address
    - correspondence address
```

Appendix III: Configuration Reference

The repository's .env.example, unchanged. All values are placeholders or safe local defaults. JWT_SECRET_KEY=change-me is deliberately rejected by the application until it is replaced by a random value of at least 32 characters.

```
# Copy to .env and fill in. Never commit .env.
# All values below are placeholders or safe local defaults.
# --- Application ------------------------------------------------------
APP_ENV=development
LOG_LEVEL=INFO
BACKEND PORT=8000
FRONTEND_PORT=5173
# Dashboard -> API base URL (baked into the frontend at build time)
VITE_API_BASE_URL=http://localhost:8000
# --- Security (REQUIRED) ----------------------------------------------
# Generate with: python -c "import secrets; print(secrets.token_urlsafe(48))"
JWT SECRET KEY=change-me
JWT_ALGORITHM=HS256
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=30
AUTH_LOGIN_RATE_LIMIT_ATTEMPTS=5
AUTH_LOGIN_RATE_LIMIT_WINDOW_SECONDS=60
# Comma-separated dashboard origins allowed by CORS. The browser extension does
# not need an entry: it calls the API through its host permission.
CORS_ORIGINS=http://localhost:5173
# --- MongoDB (structured data, source of truth) -----------------------
MONGODB_URL=mongodb://localhost:27017
MONGODB_DATABASE=smart_form_db
MONGODB SERVER SELECTION TIMEOUT MS=1500
# Only used by docker-compose to initialise the MongoDB container
MONGODB_ROOT_USERNAME=smartform
MONGODB_ROOT_PASSWORD=
# --- ChromaDB (document chunk vectors) --------------------------------
CHROMADB_PORT=8001
CHROMADB URL=http://localhost:8001
CHROMADB_COLLECTION=user_document_chunks_v2
CHROMADB_TENANT=default_tenant
CHROMADB_DATABASE=default_database
CHROMADB_TIMEOUT_SECONDS=30
# --- LLM provider -----------------------------------------------------
# One of: google | openai | anthropic | mock
# "mock" makes no network calls and returns empty AI results (pattern-based
# extraction and deterministic field matching still work).
LLM_PROVIDER=mock
# Exact model id for the chosen provider, e.g. a Gemini, GPT or Claude model id.
LLM_MODEL=
LLM_API_KEY=
LLM_TIMEOUT_SECONDS=60
LLM_MAX_OUTPUT_TOKENS=4096
LLM_MAX_RETRIES=2
# --- Embeddings -------------------------------------------------------
# Any sentence-transformers model id. Downloaded on first use (~90 MB for the
# default). Changing it requires a new CHROMADB_COLLECTION and a reindex.
EMBEDDING_MODEL_NAME=all-MiniLM-L6-v2
EMBEDDING_DIMENSION=384
# --- File storage -----------------------------------------------------
# local | cloudinary
STORAGE_PROVIDER=local
STORAGE_ROOT=./uploads
MAX_UPLOAD_SIZE_MB=10
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
```

```
# --- Document processing ------------------------------------------------------
# inline: process inside the API process (no Redis needed)
# celery: queue to a Celery worker through Redis (used by docker-compose)
PROCESSING_MODE=inline
REDIS_URL=redis://localhost:6379/0
PROCESSING_STALE_AFTER_MINUTES=15
# Set to true to add PaddleOCR to the backend and worker images (large download).
INSTALL_OCR=false
CELERY_WORKER_CONCURRENCY=2
OCR_LANGUAGE=en
OCR_RESOLUTION=150
OCR_CPU_THREADS=2
LLM_EXTRACTION_MAX_CHARS=18000
# --- Retrieval (RAG) ----------------------------------------------------------
RAG_CHUNK_SIZE=500
RAG_CHUNK_OVERLAP=50
RAG_TOP_K=4
# Chunks below this cosine similarity are ignored
RAG_MIN_SIMILARITY=0.35
# --- Form filling -------------------------------------------------------------
# Deterministic matches below this confidence are sent to the LLM classifier
CLASSIFIER_CONFIDENCE_THRESHOLD=0.75
# Recommendations below this confidence always need user review
AUTOFILL_MIN_CONFIDENCE=0.85
# Extra concepts that always need confirmation (ontology "sensitive" concepts are
included automatically)
SENSITIVE_FIELD_CONCEPTS=identity.passport_number,identity.pan_number,identity.aadhaa
r_number,identity.national_id,financial.account_number,financial.routing_number,finan
cial.tax_id,financial.cif_number
```

Appendix IV: List of Technical Components

Backend (Python 3.11)

Table IV.1 lists the backend packages and their version constraints from requirements.txt, requirements-ocr.txt and requirements-dev.txt.

Table IV.1: Backend dependencies

Package	Version constraint	Role
fastapi, uvicorn[standard]	>=0.115,<1.0; >=0.30,<1.0	Web framework and ASGI server
pydantic, pydantic-settings, email-validator	>=2.8,<3.0; >=2.6,<3.0; >=2.2,<3.0	Models, settings, e-mail validation
bcrypt, PyJWT	>=4.2,<5.0; >=2.9,<3.0	Password hashing; tokens
slowapi	>=0.1.9,<0.2.0	Rate limiting
httpx	>=0.27,<1.0	HTTP client for LLM providers and ChromaDB
motor, pymongo	>=3.6,<4.0; >=4.9,<5.0	MongoDB access
langgraph	>=1.0,<2.0	Form-fill state graph
pdfplumber, pypdfium2, python-docx, Pillow, opencv-python-headless	see requirements.txt	Document parsing and image pre-processing
sentence-transformers	>=3.0,<6.0	Embeddings (all-MiniLM-L6-v2)
celery[redis]	>=5.4,<6.0	Background jobs in Compose
cloudinary	>=1.41.0,<2.0.0	Optional file storage
paddleocr, paddlepaddle	>=2.7,<3.0; >=2.6,<3.0	Optional OCR (requirements-ocr.txt)
pytest, pytest-cov, ruff, black, pip-audit, reportlab	see requirements-dev.txt	Tests, lint, format, dependency audit, synthetic PDFs

Dashboard and extension

Table IV.2 lists the framework versions declared in the two package manifests.

Table IV.2: Dashboard and extension framework versions

Component	Dashboard	Extension
React	19.2	18.3
TypeScript	6.0	5.5

Component	Dashboard	Extension
Vite	8.3	5.4
Vitest	5.0	2.0
jsdom	29	25
Other	React Router 7, lucide-react, Testing Library	lucide-react, Playwright 1.63

Services and tools

- MongoDB 8.0, ChromaDB (chromadb/chroma image), Redis 7 (alpine), Nginx (dashboard image), Docker Compose.
- Playwright with Chromium for the end-to-end test; GitHub Actions for continuous integration.
- Graphviz 16.1 and Pillow were used to draw the diagrams of this report from scripts kept with the report sources.

Appendix V: Installation and Test Commands

Commands from the repository README, for a Windows PowerShell session. Docker Desktop with Compose v2 is required for the stack.

```
# 1. configuration
Copy-Item .env.example .env
# set JWT_SECRET_KEY to a random value of at least 32 characters, e.g.
python -c "import secrets; print(secrets.token_urlsafe(48))"

# 2. start the stack (MongoDB, ChromaDB, Redis, API, worker, dashboard, extension
builder)
docker compose up --build
# dashboard: http://localhost:5173   API: http://localhost:8000   docs: /docs

# 3. development environment and checks
python -m venv .venv ; .venv\Scripts\Activate.ps1
python -m pip install -r backend/requirements-dev.txt
npm ci
python -m pytest
npm run typecheck ; npm test ; npm run build

# 4. isolated browser end-to-end test
.\scripts\test-e2e.ps1
```

To use the extension, build it (the Compose stack does this) and load the extension/dist folder as an unpacked extension in Chrome, open a form, click the extension icon, choose Scan this page and then Suggest values, review the list and choose Fill. To read scanned documents, set INSTALL_OCR=true and rebuild, then choose Process again on the document.

Appendix VI: List of Papers Presented and Published

No paper presented or published from this project is recorded in the project repository. If the team has presented or published work, the details are to be entered here before submission.